

1. C言語マスター 目次

(0) C言語の実習を始める前に (準備)	… P. 2
(1) 画面に文字を表示する (関数printf()の使用法)	… P. 3
(2) プログラムをわかりやすくする (コメントの書き方)	… P. 6
(3) 変数と定数とはなにか (変数・定数について)	… P. 9
(4) 実際に計算をおこなうために (演算子について)	… P. 18
(5) プログラムの構造 (制御構造、論理・関係演算子)	… P. 23
(6) 三角関数などの使用方法 (数学関数・数学定数)	… P. 43
(7) まとまったデータの扱い方 (配列変数)	… P. 46
(8) 文字データの扱い (文字・文字列)	… P. 52
(9) 二次元配列の使用法 (二次元配列、文字列の配列)	… P. 57
(10) データの入力方法 (文字単位の入出力、数値を正しく入力する)	… P. 64
(11) わかりやすいプログラムのスタイル (インデント、マクロ機能)	… P. 75
(12) プログラムの部品を作る (関数の利用)	… P. 80
(13) 変数の有効範囲 (変数のスコープ)	… P. 87
(14) 変数の生存期間 (記憶クラス)	… P. 91
(15) 高度なデータ型を使う [1] : 構造体	… P. 103
(16) 高度なデータ型を使う [2] : 自分だけの新しい型を作る (typedef)	… P. 108
(17) ポインタという考え方 (ポインタ[1])	… P. 111
(18) ポインタの高度な使い方 (ポインタ[2])	… P. 120
(19) プログラムにコマンドラインを取り込む (main()関数への引数)	… P. 127
(20) 動的なメモリ確保	… P. 132
(21) 自分で作った関数・マクロをいつでも使えるようにする (ライブラリとヘッダファイル)	… P. 138
(22) ファイル処理 (テキスト・バイナリファイルの取り扱いとそのエラー処理)	… P. 149
(23) 特定の範囲の整数データに名前を付ける (列挙型, enum)	… P. 169
(24) ビット単位でのデータ操作 (ビット演算子)	… P. 172

1. C マスター実習

(0) C 言語の実習を始める前に (準備)

① はじめに

プログラミング技術の授業では、C 言語によるプログラムをいくつか作成していきます。しかし、そこではアルゴリズムの学習に重点をおいており、プログラミング技法に関する細かな内容には触れません。そこで、この実習では、C 言語の基本文法を中心とするプログラミング技法の学習に主眼をおきます。

また、実習に臨むにあたっては、次のコラムに挙げた (i) 考えること (ii) 覚えること (iii) やってみること の3点に心がけて学習してください。

~~~~ コラム 1 ~~~~~

i) 考えること

とにかく暗記をすればC言語がマスターできるかという、そうではありません。覚えるだけではなく、自分の頭を使って考えることが必要です。

ii) 覚えること

C言語はコンピュータが理解する言葉です。コンピュータは君たちがC言語を使って書いたプログラムを読んで動作します。言葉を使うためには、どうしてもさまざまなことを覚えなくてはなりません。C言語の文法、ライブラリ関数の使い方、それに専門用語を覚えなくてはなりません。

iii) やってみること

プログラミングの学習では、特にこの「やってみる」ことが重要です。実際に自分でコンピュータ上でプログラムを作ってみることが、C言語マスターの近道といえます。やってみれば、何か間違いをやらかしたらコンピュータが指摘してくれることもあるだろうし、プログラムがうまく動作しないことで「何かが間違っている」ことがわかることもあります。

② 準備

ホームディレクトリの配下に実習用のディレクトリを作成し、そこをカレントディレクトリとします。以降、本実習においてはそのディレクトリを使用します。

```
$ mkdir c-lang [↵] ... c-lang ディレクトリの作成
$ cd c-lang [↵] ... c-lang ディレクトリに移動
```

また、例題プログラム (必要なものについては問題プログラムも)
/home/pub/practice/c-master ディレクトリに rei??-??.(c (課題は toi??-??.(c
) というファイル名で置いてありますから、各自コピーして利用すること。

```
$ cp /home/pub/practice/c-master/rei01-01.c . [↵]
... 例(1)-1のプログラムをカレントディレクトリにコピー
$ cp /home/pub/practice/c-master/toi01-01.c . [↵]
... 問題(1)-1のプログラムをカレントディレクトリにコピー
```

③ C言語プログラムの基本形

C言語プログラムは基本的に次のような形で記述します。

<pre>#include <stdio.h> main() { 文1; 文2; ⋮ 文n; return(0); }</pre>	… プログラム先頭につけるおまじない … 実行を開始するブロックの名称（絶対必要） … ブロックの先頭 … プログラムの実行文₁ … プログラムの実行文₂ … プログラムの実行文_n … プログラムの実行終了 … ブロックの終端
--	--

ここで、“main()”（メイン関数と呼びます）は、プログラムの実行が最初に開始される部分です。プログラムの本体はこの後続く“{}”内に記述していきます。“{}”に囲まれた範囲をブロックといいます。このブロックの最後に記述した“return(0)”を実行すると、プログラムの実行を終了し制御がOSに戻ります。また、プログラムの各文の末尾には“;”（セミコロン）をつけます。

なお、プログラムの先頭にある“#include <stdio.h>”は、「stdio.h というヘッダファイルを読み込みなさい」というコンパイラへの指示命令ですが、当分の間、入出力を伴うプログラムには必ず記述する（入出力のないプログラムは基本的に存在しませんから、必ず書きます）「おまじない」だと思っていて構いません（実習の後半で改めて扱います）。

(1) 画面に文字を表示する（関数printf()の使用法）

C言語プログラムにおいて、出力命令として必ずといっていいほど使用されるprintf()という関数について学習します。

次のプログラムを実行してみましょう。

---- 例(1)-1 -----

```
#include <stdio.h>
main()
{
    printf("Hello world. \n");
    return(0);
}
```

まずは、単に文字列を出力するだけの小さなプログラムを、どう書くか確認します。このプログラムを実行すると画面に「Hello world.」と表示されます。これは、最も簡単なCのプログラムです。

では、次のプログラムを同様に実行してみましょう。

----- 例(1)-2 -----

```
#include <stdio.h>

main()
{
    printf("1 + 2 = %d\n", 1 + 2);
    return(0);
}
```

ここで、printf文中の「%d」は「1+2」の結果を10進整数表示するものです。

printf() の書式

printf(表示文字列, 出力するデータの並び);

表示文字列 : 表示させたい文字や文章、%d, %cなど

出力するデータの並び : 変数や定数などのデータを順番に記述したもの

使用例 : printf("%d * %d = %d\n", 2, 3, 2 * 3);

さて、前述の2つのプログラムのprintf文中に'\n'という記号がありました。これは、エスケープシーケンス(escape sequence)と呼ばれ、そのままでは画面に表示できない文字・記号等を表示できるようにするための記号です。これ以外にも次表のようなものが用意されています。

エスケープシーケンス	意味	ASCII端末での表示	日本語JIS端末での表示
\n	改行	\n	¥n
\"	"を表示する	\"	¥"
\'	'を表示する	\'	¥'
\\	\を表示する(日本語端末では¥)	\\	¥¥
\t	タブ	\t	¥t
\a	ベル	\a	¥a
%%	%を表示する	%%	%%

[表1-1] 主なエスケープシーケンス (一部)

では、次のプログラムを実行してみましょう。

----- 例(1)-3 -----

```
#include <stdio.h>

main()
{
    printf("aaaaa\nbbbb\nccccc\n");    ... ①
    printf("AAAAA\n\nBBBBB\n");        ... ②
    printf("CCCCC");                    ... ③
    printf("DDDDD\n");                  ... ④
    printf("\tEEEEEE\n");               ... ⑤
    printf("\"fffff\"");                 ... ⑥
    printf("\aerror\n");                ... ⑦
    return(0);
}
```

- ① 1つのprintfで、3行分の出力をおこなう。
- ② 「AAAAA」と「BBBBB」の間に1行空行を作る。
- ③ 「\n」がないため、改行されない。
- ④ ③と同じ行（その続き）に出力される。
- ⑤ 「EEEEEE」を8桁目から出力する。
- ⑥ 「ffffff」を「"」（ダブルクォーテーション）でくくる。
- ⑦ ピッと鳴った後、「error」と表示する。

ポイント

☆printf()は_____する関数である。

☆printf()の書式

☆エスケープシーケンスとは

問題(1)-1

次のプログラムの間違いを見つけ、正しく直しなさい。(toi01-01.c)

```
----- toi01-01.c -----
#include <stdio.h>

main()
{
    print("Hello world.\n")

    return(0);
}
```

問題(1)-2

次のように画面表示されるプログラムを作りなさい。(toi01-02.c)

```
静岡県立浜松工業高等学校
[tab]  情報技術科
[tab]  1年!"[氏名]"
```

※[tab]はタブコード出力の意味

[氏名]の部分には、自分の名前を表示させること

(2) プログラムをわかりやすくする（コメントの書き方）

よいプログラムを作るためには、ただ命令（コード）を並べるだけでは不十分です。上手に書かれたプログラムは、正しく機能するだけでなく、シンプルでわかりやすいものです。コメントによって、プログラマはプログラムの中に説明書きを含めることができます。書き方が明快で、かつコメントがしっかり書かれているプログラムがよい作品といえます。

プログラマは、利口ぶったトリックを避けるべきです。次のような、2つのプログラムを考えてみましょう。

- (a) 利口ぶったプログラマがあらゆるトリックを駆使して書いている。
コメントはないが、とにかく動く。
- (b) コメントがしっかり書かれていて、見事に構造化されているが、動かない。

この両者のどちらが実際に役立つでしょうか？

長期的に見ると、後者(b)の方が役に立ちます。動かない部分は直すことができます。(a)の方は、今は動くとしても、プログラムというものはいずれは変更が加えられる可能性があります。理解しにくく書かれたプログラムの変更作業は、プログラマにとって最悪の仕事となってしまいます。

よいプログラムの考え方

- (a) わかりやすいプログラムを書く工夫
- (b) 修正しやすいプログラムを書く工夫
- (c) 効率のよいプログラムを書く工夫
- (d) 移植性の高いプログラムを書く工夫
- (e) 実行時、エラーの少ないプログラムを書く工夫
- (f) デバッグしやすいプログラムを書く工夫

プログラムに説明を加えることの大切さ

もう一度、例(1)-1 のプログラムを見てみましょう。「よいプログラム考え方」の観点から見た場合、これには大きな欠陥があります。それは、多くのプログラマが今でも犯している事柄であり、他のどんな問題よりもトラブルの原因となるものです。その欠陥というのは、

＜このプログラムにはコメントがない＞

ということです。

動くけれどもコメントのないプログラムは、いつかは爆発する時限爆弾のようなものです。遅かれ早かれ、誰かがそのプログラムの変更や改良をすることになり、そうするとコメントがないため作業が何倍も困難になります。コメントがしっかり書かれたシンプルなプログラムは、芸術作品といってもよいでしょう。コメントの書き方の勉強は、正しいコードの書き方の勉強と同じくらい重要なのです。

C言語のコメント

- ・複数行コメント … 「/*」で始まり「*/」で終わる。
- ・単一行コメント … 「//」で始まりこれ以降、行末までがコメント

例(1)-1 を書き直した例を次に挙げます。

例(2)-1

```
/******  
 * 例(2)-1 *  
 * *  
 * ファイル名 : rei2-1.c *  
 * *  
 * 作者      : 浜工 太郎 *  
 * *  
 * 作成日    : 2014. 12. 25 *  
 * *  
 * 説明      : 簡単なプログラムの例 *  
 *            プログラムを実行すると *  
 *            メッセージが表示される *  
*****/  
  
#include <stdio.h>  
main()  
{  
    printf("Hello world.\n"); // 世界へ向けて挨拶する (ただし、このコメントは不適切)  
    return(0);  
}
```

プログラムは、簡潔で読み易くなければなりません。プログラムは、コンピュータの命令を集めたものですが、それに用いているアルゴリズムやデータを説明している参考文献でもあるのです。文書化する必要のあることは、すべてコメントとして書くべきです。

コメントには次の2つの役割があります。

- ・プログラムの理解を助ける。
- ・自分が何を何のためにやったかという記憶を助ける。

ただし、ソースコードを見れば自明であるようなコメントは書くべきではありません。そのようなコメントは、かえってプログラムを読みにくくするだけです。

ポイント

☆コメントの2つの役割

- ・ _____
- ・ _____

この指導書では紙面の関係上、必要以外のコメントは記述しませんが、雛形となるプログラム例にはコメントが記述されているので、そちらを参考にしてください。

コラム 2

◆◆◆ コメントの様々なスタイル ◆◆◆

活字印刷では、ゴシック体や斜体文字など様々な書体を使って、文字に変化をつけることができます。プログラムのコーディングでは、通常種類の書体しか使えませんが、うまく工夫すればその制約を克服することができます。以下は、コメントの様々なスタイルの例です。

```
/******
*****
***** 警告：これはプログラマの注意を引く *****
*****          警告メッセージの          *****
*****          書き方の例          *****
*****
*****/

/*----->      これも軽い警告文の書き方      <-----*/

/******
*   ボックス型のコメント                               *
*   プログラムの頭の部分に使うとよい                   *
*****/

/*
*   これは1つの節の始まり
*   ~~~~~
*   ここでこの節で行う処理について説明を書く
*/

/*-----*
*   これもボックスの書き方の例                           *
*-----*/

/*>>>>>>>>>      大きな節単位のヘッダ      <<<<<<<<<<*/

/*
*   以降の10数行に関する説明を書くための中レベルのコメント
*   ゴシック書体を使えなくてもこうして 言葉 を強調できる
*/

/* 次の行について説明する簡単なコメント */
```

左右にもアスタリスクなどのある、いわゆる「ボックス型」のコメントは、コメントを書き換える時などに、非常に扱い難いものです。そこで、コメントの編集をし易くするために上下だけに飾り罫を設ける、次のようなスタイルもよく使われます。

```
/*-----
      コメント本文
-----*/

/*=====
      コメント本文
=====*/
```

◆◆◆ 単一行コメント // ~ ◆◆◆

```
// コメント文1
// コメント文2
```

単一行コメントは、本来C++言語のコメントで、以前はC言語で使えませんでした。しかし、新しいC99という規格以降のC言語は、これに対応しています。本校のCコンパイラもC99規格ですので、このコメントが使えます。

(3) 変数・定数とはなにか（変数・定数について）

プログラムでデータを扱う場合には、「変数」や「定数」が必要になります。

変数(variable)：値を入れておく箱のようなものですが、その実体は名前で参照できるメモリ番地です。変数に格納された値は、代入や単項演算子などによって変化します。

定数(constant)：プログラム内に直接記述する値です。

① 変数名(variable name)

変数に付ける名前を変数名といいます。この変数名の使い方には、どのような規則があるのでしょうか。

次のプログラムを実行してみましょう。

---- 例(3)-1 -----

```
#include <stdio.h>

main()
{
    int data, Data, DATA;    // int型変数の宣言

    data = 100;
    Data = 200;
    DATA = 300;

    printf("data = %d Data = %d DATA = %d\n", data, Data, DATA);

    return(0);
}
```

この例では、変数名に「data」「Data」「DATA」と、大文字と小文字を混在した変数を使用しています。

アルファベットの並びからいえば全く同じであるのに、結果として別々の値が代入されます。つまり、C言語では大文字と小文字を明確に区別しているのです。

変数名の命名規則

C言語における変数名の命名規則は次の通りです。

(但し、ANSI規格の場合)

- (a) 先頭文字は英字(a～z, A～Z)か下線(_)でなくてはならない。
- (b) 2番目以降の文字は英字か下線か数字でなくてはならない。
- (c) 大文字と小文字は区別される。
- (d) 変数名の長さには制限はないが、31文字までが有効である。
- (e) 途中に英字・下線・数字以外の文字があってはいけない。
- (f) 途中に空白があってはいけない。
- (g) 予約語を使用してはならない。

※ ANSI … American National Standards Institute (アメリカ規格協会)：アメリカの標準化団体

② 変数・定数の型

(a) 変数の型及び宣言

変数や定数には、文字や整数などの種類がいろいろあります。この種類のことを型(type)と呼びます。本校で使用しているC言語処理系では、次のような型が利用できます。

型	宣 言	意 味	データの範囲	バイト長
整数型	short	整数(符号あり $\leq$ int)	$-2^{15} \sim 2^{15}-1$	
	unsigned short	整数(符号なし $\leq$ int)	$0 \sim 2^{16}-1$	
	int	標準の整数(符号あり)	$-2^{31} \sim 2^{31}-1$	
	unsigned int	標準の整数(符号なし)	$0 \sim 2^{32}-1$	
	long	整数(符号あり $\geq$ int)	$-2^{63} \sim 2^{63}-1$	
	unsigned long	整数(符号なし $\geq$ int)	$0 \sim 2^{64}-1$	
	long long	整数(符号あり $\geq$ long)	$-2^{63} \sim 2^{63}-1$	
	unsigned long long	整数(符号なし $\geq$ long)	$0 \sim 2^{64}-1$	
実数型	float	単精度浮動小数点数 (有効桁数：6桁)	$\pm 1.2 \times 10^{-38} \sim \pm 3.4 \times 10^{38}$	
	double	倍精度浮動小数点数 (有効桁数：15桁)	$\pm 2.2 \times 10^{-308} \sim \pm 1.8 \times 10^{308}$	
	long double	拡張倍精度浮動小数点数 (有効桁数：18桁)	$\pm 3.3 \times 10^{-4932} \sim \pm 1.8 \times 10^{4932}$	
文字型	char	1バイト文字(符号あり)	$-2^7 \sim 2^7-1$	
	unsigned char	1バイト文字(符号なし)	$0 \sim 2^8-1$	

[表3-1] 変数・定数の型

※ データ範囲、バイト長はC言語の処理系により異なる。

では、現在使用している処理系における変数のバイト長がどのくらいであるか、次のプログラムの実行結果をもとに [表3-1] の空欄を埋めてみましょう。

----- 例(3)-2 -----

```
#include <stdio.h>

main()
{
    printf("char      = %d\n", sizeof(char));
    printf("short     = %d\n", sizeof(short));    // int型よりは長くない
    printf("int       = %d\n", sizeof(int));
    printf("long      = %d\n", sizeof(long));      // int型よりは短くない
    printf("long long = %d\n", sizeof(long long)); // long型よりは短くない
    printf("float     = %d\n", sizeof(float));
    printf("double    = %d\n", sizeof(double));
    printf("long double = %d\n", sizeof(long double)); // double型よりは短くない

    return(0);
}
```

※sizeof()：指定したデータ型や変数のサイズ（バイト長）を返す演算子

数値を扱う型には、整数型と実数型がありますね。この二つの型にはどんな違いがあるのでしょうか？まず、整数同士の四則演算ではどのような結果が得られるか、次のプログラムを実行してみましょう。

----- 例(3)-3 -----

```
#include <stdio.h>

main()
{
    int x,y;    // int型変数の宣言

    x = 5;
    y = 6;

    printf("x + y = %d\n", x + y);    // 加算
    printf("x - y = %d\n", x - y);    // 減算
    printf("x * y = %d\n", x * y);    // 乗算
    printf("x / y = %d\n", x / y);    // 除算

    return(0);
}
```

このプログラムは、x,yという2つの整数型の変数を用意して、その四則演算を行うものです。加減算と乗算は期待した結果が得られますが、除算は期待した結果の0.833...にはなりません。

この理由は、整数どうしの演算では結果も整数となるからです。期待した実数型での結果が得られるようにするには、変数を整数型でなく実数型に変更する必要があります。

例(3)-3を実数型に変更したプログラムが、次の例(3)-4です。ここで登場する「%f」は、実数型の値を表示するためのものです。

----- 例(3)-4 -----

```
#include <stdio.h>

main()
{
    float x,y;

    x = 5.0;
    y = 6.0;

    printf("x + y = %f\n", x + y);    // 加算
    printf("x - y = %f\n", x - y);    // 減算
    printf("x * y = %f\n", x * y);    // 乗算
    printf("x / y = %f\n", x / y);    // 除算

    return(0);
}
```

今度は期待通りに計算されます。

このように、必要に応じて型を決めないと、期待した結果が得られないことがあるので注意してください。

(b) 定数の型

定数には次の4種類がある。

- (ア) 整数定数 : 10進数表現 … 0以外の数字か+, -で始まる数字列
(0~9の数字を使用)
例) 1, 1024, -45など
- 16進数表現 … 0x(ゼロ・エックス) または 0X で始まる英数字列
(0~9の数字, a~f・A~Fの文字を使用)
例) 0x00, 0xF4F5
- 8進数表現 … 0(ゼロ)で始まる数字列
(0~7の数字を使用)
例) 04, 0777, 00
- 倍長整数 … 数字の後ろにl(エル) または L(エル)を付ける
(long) 例) 100001, 12345L
- 倍倍長整数 … 数字の後ろにll(エルエル)またはLL(エルエル)を付ける
(long long) 例) 10000000000011, 1234567890LL
- (イ) 実数定数 : 実数の表現は、通常の数学で使う表現が用いられます。
例えば、0.048, 13.5, -4.9, 2.0, 2. と書きます。
また、1.485e2 (1.485×10^2) のような指数表現も用いられます。
- (ウ) 文字定数 : 文字定数は、文字を一重引用符 (シングルクォーテーション) でくくって表現します。例えば大文字のAは、'A' と書きます。
ASCIIコードでは、20₍₁₆₎ ~ 7E₍₁₆₎ の範囲で文字を表します。
- (エ) 文字列定数 : 二重引用符 (ダブルクォーテーション) で文字綴りをくくって表現します。たとえば、"C program" と書きます。

③ 混合演算と型変換

一般に算術式を記述していくと、整数と実数など異なる型のデータが混在している計算が行われることがあります。例えば、

$$\begin{array}{ccc} 1 + 1.2 & \longrightarrow & \text{結果: } 2.2 \\ \text{(整数値 + 実数値)} & & \text{(実数値)} \end{array}$$

また、i を整数型変数、x を浮動小数点型変数とすると

$$i + x \quad \longrightarrow \quad \text{結果: 浮動小数点型}$$

このように、異なる型どうしの演算を混合演算と呼びます。しかし実際には

$$\begin{array}{ccc} 1 + 1.2 & \longrightarrow & 1.0 + 1.2 \\ \text{(整数値 + 実数値)} & & \text{(実数値 + 実数値)} \end{array}$$

と、整数値を浮動小数点値に内部で変換してから計算されています。

このようにデータ表現を変換することを型変換(type conversion)といいます。

一般に、演算を行う際そのデータ型が異なる場合には、**精度の高い方へ型変換**が行われます。具体的には次のように変換されます。

short → int → long → long long → float → double → long double

----- 例(3)-5 -----

```
#include <stdio.h>

main()
{
    printf("1 + 2 = %d\n", 1 + 2);           // 結果:   int型
    printf("1.0 + 2.0 = %f\n", 1.0 + 2.0);   // 結果: float型
    printf("1.0 + 2 = %f\n", 1.0 + 2);       // 結果: float型
    printf("19 / 10 = %d\n", 19 / 10);        // 結果:   int型
    printf("19.0 / 10.0 = %f\n", 19.0 / 10.0); // 結果: float型

    return(0);
}
```

しかし場合によっては、データの有効桁数が小さくなってしまうことがあります。
次のプログラムを実行するとどうなるでしょうか？予想してみてください。

----- 例(3)-6 -----

```
#include <stdio.h>

main()
{
    long long a;
    float b;

    a = 1234567890LL;
    b = a;

    printf("a = %lld\n", a); // 変数 a の表示
    printf("b = %f\n", b);   // 変数 b の表示

    return(0);
}
```

ところで、i, j をint型、f をdouble型とすると

```
i = 7;
j = 2;
f = i / j;
```

の結果、変数fには 3.0 が代入されます。しかし、3.5 を代入したいというのが本当でしょう。この時、一時的にi, jをdouble型にして計算を行えば、期待した結果が得られます。この一時的に型を変更する方法をキャスト(cast)といい、次のよ

うにします。

f = (double) i / (double) j;
 ↑ ↑
 変数 i, j の値を一時的に double型として使用します。

これを一般形式で表すと、次のような形になります。

(型名) 変数 : (int)a … 変数aを一時的にint型として使用
(型名) 定数 : (float)10 … 定数10をfloat型として使用
(型名) (算術演算) : (double)(i + 1) … 計算結果をdouble型として使用

[表3-2] 書式制御文字一覧

④ 書式制御文字

さて、これまでのプログラム中に「%d」「%f」といった文字が登場してきました。これは、変数や定数の値を表示する時にどのような形で出力するかを指定するもので、書式制御文字と呼ばれます。

書式制御文字には、右の表のようなものがあります。詳細は「資料 02. C言語書式付き入出力の詳細」やmanコマンドでprintf()関数の書式を調べてください。

書式制御文字	用 途
%d , %nd	int型
%ld , %nld	long型
%lld , %nlld	long long型
%f , %n.mf	float型, double型
%Lf , %n.mLf	long double型
%e , %n.me	指数表現
%c , %nc	char型
%s , %ns	char型配列, 文字列定数
%o , %no	int型の8進数表現
%x , %nx	int型の16進数表現
%u , %nu	unsigned型
%g , %n.mg	実数変換

※ n : データの最小表示桁数,
m : %f,%eでは小数点以下の桁数, %gでは有効桁数

----- 例(3)-7 -----

```
#include <stdio.h>

main()
{
    int i;
    float f;

    i = 65;
    f = 12.34;

    printf("10進数          : %d\n", i);      // 10進表示
    printf(" 8進数          : %o\n", i);      //  8進表示
    printf("16進数          : %x\n", i);      // 16進表示
    printf("10進数 (桁指定あり) : %4d\n", i);  // 10進表示 (桁指定あり)
    printf(" 8進数 (桁指定あり) : %4o\n", i);  //  8進表示 (桁指定あり)
    printf("16進数 (桁指定あり) : %4x\n", i);  // 16進表示 (桁指定あり)
    printf("\n");
    printf("float型 (桁指定なし) : %f\n", f);   // float型表示
    printf("float型 (桁指定あり) : %7.1f\n", f); // float型 (桁指定あり)

    return(0);
}
```

ポイント

☆変数とは_____

☆定数とは_____

☆変数名の命名規則

(a) _____

(b) _____

(c) _____

(d) _____

(e) _____

(f) _____

(g) _____

☆変数や定数には3つの型がある。

_____, _____, _____

☆各データ型のバイト長はC言語の処理系により異なる。

☆データ型の異なる値どうしによる演算の場合には_____が行われ、それは精度の_____へ行われる。

☆一時的に変数・定数の型を変更する方法を_____と呼ぶ。

☆書式制御文字とは_____

問題(3)-1

次の設問についてYes/Noで答えなさい。Noのものについては、間違いも指摘しなさい。

- ① C言語で書かれたプログラムは、mainという関数がはじめに実行される。
- ② C言語の変数は必ず型を持つ。
- ③ C言語のソースファイル（ソースプログラム）は直接実行することはできない。
- ④ C言語でfloatは「整数」を表す型である。
- ⑤ printfを使うと、文字列, 文字, 整数, 浮動小数点数が表示できる。
- ⑥ printf("\n"); という文を実行すると、改行の動作をする。
- ⑦ printfで、文字を表示するためには、書式制御文字の「%f」を使う。

問題(3)-2

次の設問に答えなさい。

① 次の変数の中で正しいものと誤っているものとに分類しなさい。

(a) check&check	(e) 007series	(i) weight65.3kg
(b) hello_dial_NTT	(f) Beatタシ	(j) hight170cm
(c) JRnishinippon	(g) _nooryes	
(d) 1_2_3_G0	(h) konnichiwa	

② C言語プログラムで日本文字(漢字、ひらがな、カタカナ)が使えるのはどこか？

③ C言語プログラムで、文の区切り記号とは何か？

④ C言語プログラムの実行時、最初に必ず実行されるものは何か？

問題(3)-3

次のプログラムをコンパイルしてみよう。どんなエラーが表示されるか注意深く観察し、エラーの意味を考えなさい。

また、このプログラムを正しく動作するように修正しなさい。(toi03-03.c)

```
---- toi03-03.c -----  
  
#include <stdio.h>  
  
main()  
{  
    int  lab;  
    int  abc;  
    int  _de;  
    int  fg!  
    int  if;  
  
    lab = 100;  
    abc = 200;  
    _de = 300;  
    fg! = 400;  
    if  = 500;  
  
    printf("123 = %d\n", lab);  
    printf("abc = %d\n", abc);  
    printf("_de = %d\n", _de);  
    printf("fg! = %d\n", fg!);  
    printf("if = %d\n", if);  
  
    return(0);  
}
```


問題(3)-4

次のプログラムの実行の結果、どのように表示されますか。また、どうしてそうなるか考え、常識的に妥当な結果が得られるように修正しなさい。修正方法は少なくとも2つあります。(toi03-04.c)

```
----- toi03-04.c -----  
  
#include <stdio.h>  
  
main()  
{  
    float f;  
  
    f = 7 / 22;  
    printf("結果は %f\n", f);  
  
    return(0);  
}
```

問題(3)-5

次のプログラムの間違いを指摘し、妥当な表示になるよう正しく修正しなさい。

(toi03-05.c)

```
----- toi03-05.c -----  
  
#include <stdio.h>  
  
main()  
{  
    printf("%d\n", 1.23);  
    printf("%f\n", 512);  
    printf("%s\n", 'A');  
  
    return(0);  
}
```

//

(4) 実際に計算をおこなうために（演算子について）

① 算術演算子と優先順位加減乗除算を行う演算子を術演算子といい、下表のものがあ
ります。

二項演算子		使用例	単項演算子		使用例
+	加算	a + b	+	正符号	+a
-	減算	a - b	-	負符号	-a
*	乗算	a * b			
/	除算	a / b			
%	余り	a % b			

[表4-1] 算術演算子

----- 例(4)-1 -----

```
#include <stdio.h>

main()
{
    int a = 25;    // 変数aの宣言と同時に初期値25を代入
    int b = 4;     // 変数bの宣言と同時に初期値 4を代入

    printf("-a + b = %d\n", -a + b);    // 加算
    printf("-a - b = %d\n", -a - b);    // 減算
    printf("a * (-b) = %d\n", a * (-b)); // 乗算
    printf("a / b = %d\n", a / b);      // 除算
    printf("a \% b = %d\n", a % b);     // 余り

    return(0);
}
```

さて「 $3 \times 4 + 4 \div 2$ 」という式があった時、君たちなら、まず「 3×4 」と「 $4 \div 2$ 」を計算してから、最後に加算を行い、結果「14」とするはずですが。このように、加減算よりも乗除算を先に計算するというような順序を優先順位といいます。C言語における主な演算子の優先順位は、次のように規定されています。また、すべての演算子を、付録1「C言語 演算子と優先順位」に示してありますので、確認してください。

優先順位	演 算 子
1	() (関数呼び出し) [] (配列添字)
2	++ -- + (符号) - (符号)
3	(型) (キャスト)
4	* (乗算) / (除算) % (余り)
5	+ (加算) - (減算)
6	<< >> (シフト演算子)
7	< > <= >= (比較演算子)
8	== != (比較演算子)

[表4-2] 演算子の優先順位

```
#include <stdio.h>

main()
{
    printf("3×4+4÷2 = %d\n", 3 * 4 + 4 / 2);
    printf("(3×4+4)÷2 = %d\n", (3 * 4 + 4) / 2);
    return(0);
}
```

② 代入と代入文

i = 1;

という文は、1という整数の定数をiという変数に代入(assign)しなさいという意味です。

この「=」という記号が代入操作を意味し、これを行う文のことを代入文(assignment statement)と呼びます。

代入操作の一般形式は次のように記述します。

変数名 = 算術式 ;

ここで、算術式とは通常の数学の式とほぼ同じものです。

またC言語では、次のような記述も可能です。

a = b = c = 0;

これは、3つの変数の値をすべて0にするという意味で、次の文と同じです。

a = 0;
b = 0;
c = 0;

但し、実際の処理は次のように解釈され実行されます。

a = (b = (c = 0));

(ア) c に 0 を代入

(イ) c = 0 の結果が 0 なので、これを b に代入

(ウ) 同様に a に 0 を代入

つまり代入「=」は、代入した値を返す演算の一種なのです。

また、このように「=」を続けて記述した文は、右から順に計算が行われます（これを右結合と呼びます）。

「=」のように、右辺の計算結果を左辺の変数に代入する式を代入式と呼び、代入に使用する記号（演算子）を代入演算子と呼びます。これには右表のようなものがあります。

このなかで、「=」と組み合わせ、ある変数に対して行った演算結果を同じ変数に代入するものを、特に**複号代入演算子**といいます。

演算子	説 明	意 味
=	代入	$x = \text{value}$
+=	加算代入	$x = x + \text{value}$
-=	減算代入	$x = x - \text{value}$
*=	乗算代入	$x = x * \text{value}$
%=	余剰代入	$x = x \% \text{value}$
<<=	左シフト代入	$x = x \ll \text{value}$
>>=	右シフト代入	$x = x \gg \text{value}$

[表4-1] 代入演算子・複合代入演算子

例えば、「 $a = a + 3;$ 」を代入演算子の「+=」を使って「 $a += 3;$ 」と表現できます。

また、1だけ増やす代入文「 $i = i + 1;$ 」は、インクリメント演算子「++」を使って「 $i++;$ 」や「 $++i;$ 」と表現できます。同様に、1だけ減らす代入文「 $i = i - 1;$ 」は、デクリメント演算子「--」を使って「 $i--;$ 」や「 $--i;$ 」と表現できます。

但し、「++」や「--」を代入文や他の算術式と一緒に使う場合は、注意が必要です。例えば、「 $a = i++;$ 」は「iの値をaに代入してからiを1増やす」ことを意味し、「 $a = ++i;$ 」は「iの値を1増やしてからaに代入する」ことを意味します。このように、「++」や「--」変数の前に書くか後ろに書くかで意味が全く異なってきます。ですから、これらの演算子を代入文や算術式のなかで使用すると、混乱を招くものになるだけであり、あまりメリットはありません。したがって、「++」や「--」は極力単独の文で使うようにしてください。

なお、このインクリメント演算子、デクリメント演算子については、詳しくは次項(5)の⑧で扱います。

ポイント

- ☆「+, -, *, /」等の記号を_____と呼ぶ。
- ☆演算子には_____があり、これにより計算順序が決まる。
- ☆「=」のような記号を_____と呼ぶ。

問題(4)-1

次の式の結果を表示するプログラムを作りなさい。 (toi04-01.c)

$$\frac{7}{2(a+b+c)} \quad \text{float } a = 2, b = 3, c = 5;$$

問題(4)-2

次の式を実行するプログラムを**複合代入演算子**を使って作りなさい。 (toi04-02.c)

```
a = a + 5;          int a = 1, b = 2, c = 3, d = 4, e = 5, f = 6, g = 7;
b = b + c + 2;
d = d - e + 1;
f = f * (-g + 4);
```

問題(4)-3

次の各プログラムを実行するとどのように表示されるか予想し、実際にコンパイル実行して確認しなさい。また、なぜそのように表示されたのか説明しなさい。

---- toi04-03.c -----

```
#include <stdio.h>

main()
{
    int i;
    double x, y;

    x = i = y = 3.65;

    printf("x=%5.3f i=%d y=%5.3f\n", x, i, y);

    return(0);
}
```

問題(4)-4

次の各プログラムを実行するとどのように表示されるか予想し、実際にコンパイル実行して確認しなさい。また、なぜそのように表示されたのか説明しなさい。

---- ① toi04-04-01.c -----

```
#include <stdio.h>

main()
{
    printf("%5.1f\n", 1.27);
    return(0);
}
```

---- ② toi04-04-02.c -----

```
#include <stdio.h>

main()
{
    int x = 1;
    int y = 2;
    x += 1;
    y *= x;
    printf("(x,y)=(%d,%d)\n", x, y);

    return (0);
}
```

問題(4)-5

次のプログラムの中から間違いを指摘し、訂正しなさい。ただし、変数に代入する定数や、式の中で使っている定数の値は変更しないこと。

----- ① toi04-05-01.c -----

```
#include <stdio.h>

main()
{
    int i, j,
    float x, y;

    i = 12345678900;
    j = 0xabcd

    x = 01234,
    y = 123e45;

    prontf("i + j = %f\n", i + j)
    printf("x = %d y = %f\n", x , y);

    retarn(0);
}
```

----- ② toi04-05-02.c -----

```
#include <stdio.h>

main()
{
    print("3+2 = #d\n", 3+2);
    print("5-8 = %d\n", 5-8);
    print("3×4 = %d\n", 3×4);
    print("7÷3 = %c\n", 7÷3);

    return(0)
}
```

問題(4)-6

次の計算を行うプログラムを作成しなさい。

- ① 次のような表を作るプログラムを作りなさい。ただし、次の表の ###の部分には実際の計算結果が入るようにしてください。数値は3桁で右揃えとします。

(toi04-06-01.c)

```
1 × 1 = ###
2 × 2 = ###
  ⋮
10 × 10 = ###
```

- ② 地球の半径を約6,400kmとすれば、地球の表面積及び体積はいくらですか？
また、赤道上空36,000kmの位置にある静止衛星は時速何kmで飛んでいることになりますか？有効数字4桁で表示しなさい。

ただし、地球は完全な球（もちろん実際には違います）であると仮定し、自転にかかる時間は24時間、円周率は3.14159とします。

(toi04-06-02.c)

(5) プログラムの構造 (制御構造、論理・関係演算子)

① プログラムの構造化定理

わかりにくいプログラムが作られる原因は、同じ処理内容であっても作成者の個性により十人十色のプログラムができてしまう所にあります。そのため、他人の作成したプログラムを読むには大変な労力を要します。また、時間が経過するにしたがい、作成した本人でもその内容を思い出すのは大変になっていき、まるで他人のプログラムを読むのと変わらないようになることさえあります。

そこで、プログラミング技法の標準化が唱えられました。オランダにあるアイントホーヘン大学のダイクストラ (Dijkstra) やアメリカ合衆国のミルズ (Mills) は、標準化の技法としてプログラムの構造化定理を提唱し、数学的に証明しました。この定理は構造化プログラミングまたはストラクチャードプログラミング (structured programming) と呼ばれています。

◆ プログラムの構造化定理 ◆

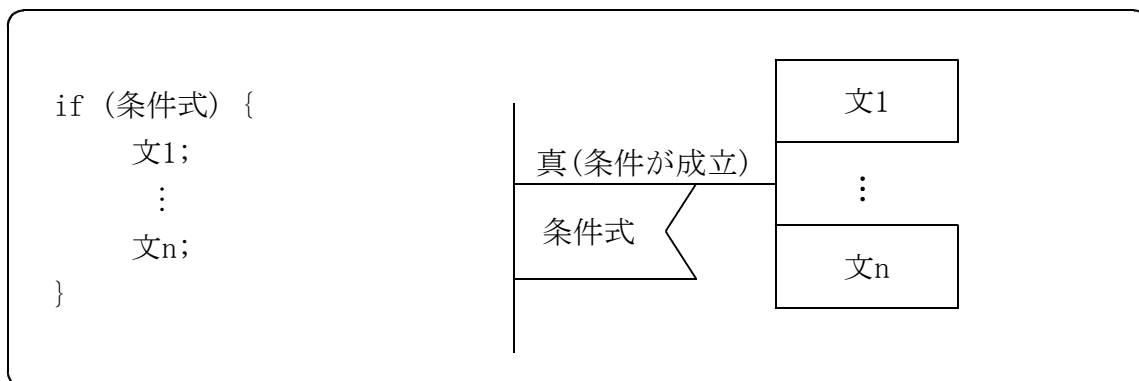
入り口と出口がそれぞれ1つである、「順次」「選択」「繰り返し」の3つの制御構造の組合せのみで、どんなアルゴリズムも表現できる。

- (a) 順次 : プログラムの命令を表れた順に実行する。
- (b) 選択 : 条件により命令を選択し実行する。
- (c) 繰り返し : 条件が満たされる限り、命令 (命令群) を実行する。

C言語には、「選択」「繰り返し」の構造に、次のような命令が用意されています。

選択処理 : if文 switch文	繰り返し処理 : while文 (ループ) for文 (LOOP) do-while文
-----------------------	--

② if文 : もし~ならば~する



最も簡単なif文は、「もし~ならば~する」という制御を行います。

この最も簡単なif文を単純if文と呼びます。単純if文は「条件式が成立すれば文を実行し、成立しなければ文を実行しない」という動作を表現します。

----- 例(5)-1 -----

```
#include <stdio.h>
#include <hamakou.h>

main()
{
    int a;

    a = getint("整数を入力してください。");

    if (a > 0) {
        printf("変数a は 0より大きい。\\n");
    }
    if (a < 0) {
        printf("変数a は 0より小さい。\\n");
    }
    return(0);
}
```

getint() … 本校オリジナル関数

書式：*variable* = getint(メッセージ文字列)

機能：キーボードから整数値を入力する。

入力を促すメッセージを画面に表示し、入力待ちとなる。入力された数値は変数(*variable*)に代入される。

条件式が成立するかしないかの判断を行うことを「条件式を評価する」といいます。条件式の評価結果は、必ず「YES (成立する)」か「NO (成立しない)」のどちらかになります。条件式の評価がYESならば真(true)、NOならば偽(false)であるといえます。例えば「a>0」の場合、変数 a の値が 0 より大きい時に真、それ以外は偽となります（C言語では、真は「非0」、偽は「0」という整数値で表されます）。

ここで登場した「<」「>」の記号を、比較演算子(comparison operator)または関係演算子(relational operator)と呼び、次表のような種類があります。

比較演算子	数学的表現	意味	プログラムでの表現	その意味
==	=	等号	a == b	aとbは等しい
!=	≠	不等号	a != b	aとbは等しくない
>=	≥	以上	a >= b	aはb以上である
<=	≤	以下	a <= b	aはb以下である
>	>	大なり	a > b	aはbより大きい
<	<	小なり	a < b	aはbより小さい

[表5-7] 関係演算子一覧

関係演算子を使用したプログラム例を次に示します。

----- 例(5)-2 -----

```
#include <stdio.h>
main()
{
    int Taro, Hanako, Jiro;
    Taro  = 20;    // 太郎の年齢
    Hanako = 30;    // 花子の年齢
    Jiro   = 10;    // 次郎の年齢

    // 変数と定数の比較
    if (Taro == 20) {
        printf("太郎は20歳です。\\n");
    }
    if (Hanako != 20) {
        printf("花子は20歳ではありません。\\n");
    }
    if (Hanako >= 20) {
        printf("花子は20歳以上です。\\n");
    }
    if (Jiro < 20) {
        printf("次郎は20歳未満です。\\n");
    }
    // 変数と変数の比較
    if (Taro > Jiro) {
        printf("太郎は次郎より年上です。\\n");
    }
    if (Taro < Hanako) {
        printf("太郎は花子より年下です。\\n");
    }
    return(0);
}
```

関係演算子を使うとき、次のような間違いをすることが多いので注意してください。

- (a) 「==」を「=」にしてしまう。
- (b) 「<=」を「<」に、「>=」を「>」にしてしまう。

特に(a)は、コンパイルエラーとならず（「=」は代入演算子です）に処理されてしまうので、ミスが発見できないことがよくあります。

----- 例(5)-3 -----

```
#include <stdio.h>
main()
{
    int a = 10;
    int b = 20;

    if (a =< b) {           // 関係演算子の書き間違い
        printf("変数 a は 変数 b 以下である\\n");
    }
    if (a = b) {           // C言語の文法上は誤りではない
        printf("変数 a は 変数 b と等しい\\n");
    }
    return(0);
}
```

また、複数の条件を指定したい時には、次に示す論理演算子(logical operator)を使います。

条件式	意 味	プログラム表現例
条件 1 && 条件 2	AND条件（論理積）	x > 0 && y == 0
条件 1 条件 2	OR条件（論理和）	x == 1 x == 3
!条件	NOT条件（否定）	!(x < 0 && y > 0)

[表5-8] 論理演算子

----- 例(5)-4 -----

```
#include <stdio.h>
#include <hamakou.h>
main()
{
    int x,y;

    x = getint("整数 x = ");
    y = getint("整数 y = ");

    if (x >= 0 && x <= 100) {
        printf("x は 0以上 100以下の整数\n");
    }
    if (y <= 0 || y >= 100) {
        printf("y は 0以下か 100以上の整数\n");
    }
    if (!(x > y)) {
        printf("x は y より大きくない\n");
    }
    return(0);
}
```

優先順位	演算子
①	()
②	!
③	<, <=, >=, >
④	==, !=
⑤	&&
⑥	

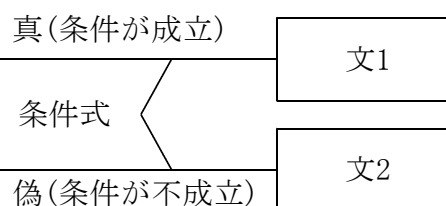
[表5-9] 演算子の優先順位

これら関係演算子・論理演算子にも [表5-9] のような優先順位があります。

③ if-else文：もし～ならば～する、そうでなければ～する

単純if文では、条件が成立した時にのみに文を実行しました。条件が成立した場合と成立しなかった場合で実行する文を変更する場合、つまり「もし～ならば～する、そうでなければ～する。」という処理は、if-else文を使って表現します。

```
if (条件式) {
    文1;
} else {
    文2;
}
```



```

#include <stdio.h>
#include <hamakou.h>

main()
{
    int a;

    a = getint("整数 a = ");

    if (a % 2 == 0) {
        printf("データは偶数です.\n");    // a が偶数
    } else {
        printf("データは奇数です.\n");    // a が奇数
    }
    return(0);
}

```

また、下図のように、条件分岐した後に実行する文が1つしかない場合は、文を囲っている{ } (中カッコ) を省略することができます。

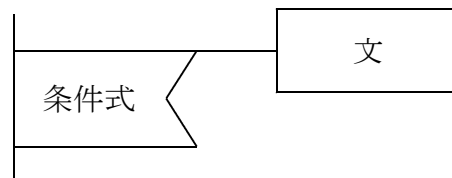
しかしこのような記法は、プログラムの修正で実行すべき文が複数になったときなど、{ } を付け忘れてしまうミスにつながったり、論理構造が把握しにくくなったりするので、あまり好ましくありません。

{ } を省略した好ましくない if 文

```

if (条件式)
    文;

```

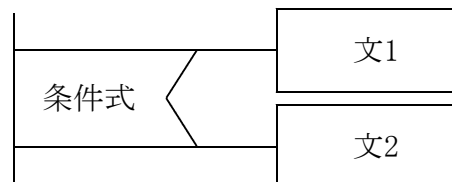


{ } を省略した好ましくない if-else 文

```

if (条件式)
    文1;
else
    文2;

```



----- 例(5)-6 -----

```
#include <stdio.h>
#include <hamakou.h>

main()
{
    int a;

    a = getint("整数 a = ");

    //----- 好ましくないif文
    if (a > 0)
        printf("データは正の数です。\\n");

    return(0);
}
```

----- 例(5)-7 -----

```
#include <stdio.h>
#include <hamakou.h>

main()
{
    int a;

    a = getint("整数 a = ");

    //----- 好ましくないif-else文
    if (a % 2 == 0)
        printf("データは偶数です。\\n");    // a が偶数
    else
        printf("データは奇数です。\\n");    // a が奇数

    return(0);
}
```

ポイント

☆プログラムの構造化定理

☆最も簡単なif文を_____と呼び、_____という動作を表現する。

☆条件が成立しなかった場合に、何か処理をさせるためには_____文を使う。

☆条件分岐後に実行する文が1つしかなくても_____

☆条件式を表現する場合に使用される演算子は_____と_____の2種類で、これらの優先順位は

_____ → _____ → _____ → _____ → _____ である。

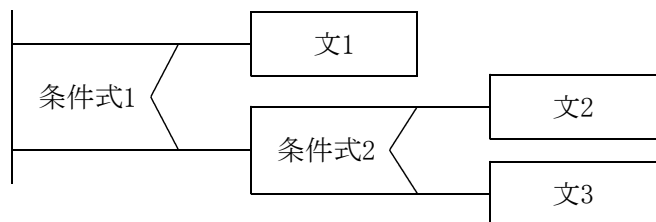
④ switch文：場合分けをする

次のような条件と処理があったとします。

優勝者には海外旅行
2 位には国内旅行
3 位には高級ホテルの食事券
それ以外の参加者にはタオルを配る

このような処理は次のような「else if文」を使って記述します。

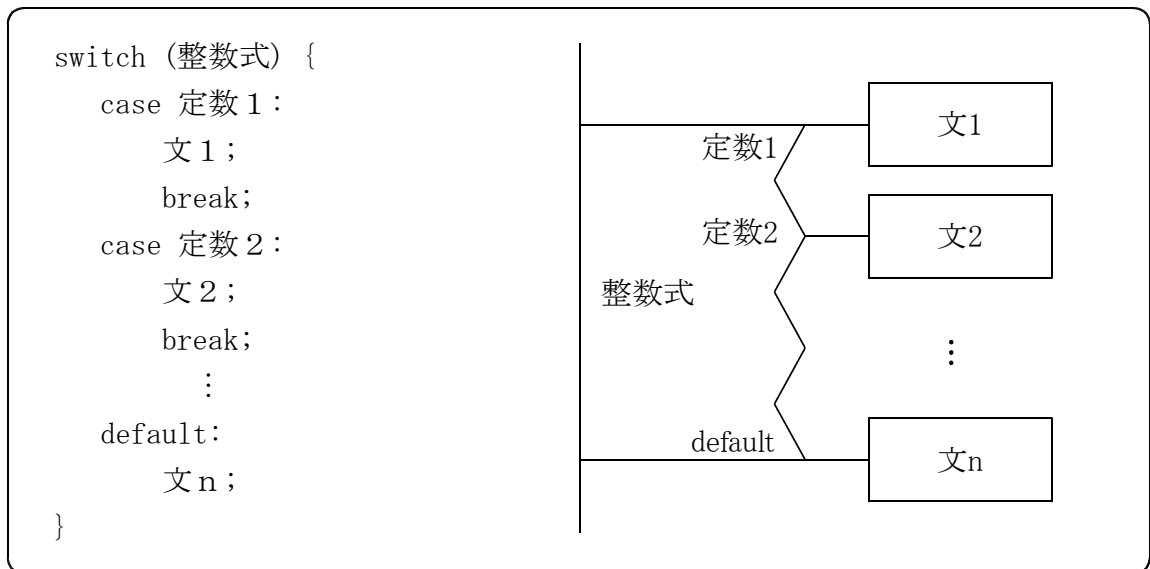
```
if (条件式 1) {  
    文 1;  
} else if (条件式 2) {  
    文 2;  
} else {  
    文 3;  
}
```



----- 例(5)-8 -----

```
#include <stdio.h>  
#include <hamakou.h>  
  
main()  
{  
    int rank;  
    rank = getint("順位を入力してください。");    // 順位を入力する  
  
    if (rank == 1) {  
        printf("優勝したあなたには、海外旅行です。\\n");  
    } else if (rank == 2) {  
        printf("2 位のあなたには、国内旅行です。\\n");  
    } else if (rank == 3) {  
        printf("3 位のあなたには、ホテルの食事券です。\\n");  
    } else {  
        printf("タオルをどうぞ。\\n");  
    }  
    return(0);  
}
```

また、「switch文」を使った次のような記述方法もあります。



但し、「switch文」の「整数式」には結果が整数になるもの、「case文」の「定数」には「整数」しか記述できないことに注意してください。

また、整数式の値がcaseで指定したいいずれの定数とも一致しない場合は、defaultで指定した文を実行します。なお、defaultを記述しない場合、一致する定数がなければ何もしません。

先程の例(5)-8を「switch文」を用いてプログラムにすると、次のようになります。

----- 例(5)-9 -----

```
{  
#include <stdio.h>  
#include <hamakou.h>  
  
main()  
{  
    int rank;  
  
    rank = getint("順位を入力してください。");    // 順位を入力する  
  
    switch(rank) {  
        case 1:  
            printf("優勝したあなたには、海外旅行です。\\n");  
            break;  
        case 2:  
            printf("2位のあなたには、国内旅行です。\\n");  
            break;  
        case 3:  
            printf("3位のあなたには、ホテルの食事券です。\\n");  
            break;  
        default:  
            printf("タオルをどうぞ。\\n");  
            break;  
    }  
    return(0);  
}
```

なお、switch文は次のように動作します。

[ステップ1]

整数式の値を求める。

[ステップ2]

整数式の値が

(1) caseの定数と等しい場合

そのcaseに続く文にプログラムの制御が移り実行される。

(2) どのcaseとも等しくないが、defaultが存在する場合

そのdefaultに続く文にプログラムの制御が移り実行される。

(3) どのcaseとも等しくなく、defaultも存在しない場合

プログラムの制御は、switch文の次の行に移る。

(switch文中の処理はどれも実行されない)

一般の処理では、各caseの処理の最後にはbreak文を付加します。これを忘れると、caseの値と一致した位置から switch文の終わりまでの、すべての文を実行してしまう結果となります。

次のプログラムは、それを実験するものです。

---- 例(5)-10 -----

```
#include <stdio.h>
#include <hamakou.h>

main()
{
    int x;

    x = getint("1~3の値を入力してください。");    // 値を入力する

    printf("\n***** すべての case に break文がある *****\n");
    switch(x) {
        case 1:
            printf("1");
            break;
        case 2:
            printf("2");
            break;
        case 3:
            printf("3");
            break;
    }
```

```

printf("\n***** すべての case に break文がない *****\n");
switch(x) {
    case 1:
        printf("1");
    case 2:
        printf("2");
    case 3:
        printf("3");
}

printf("\n***** case 2 にのみ break文がある *****\n");
switch(x) {
    case 1:
        printf("1");
    case 2:
        printf("2");
        break;
    case 3:
        printf("3");
}

printf("\n");

return(0);
}

```

基本的に、switch文でかかれたプログラムはif文でも記述が可能です。しかし、その逆は必ずしも記述できるとは限りません。if文とswitch文は次のように使い分けるとよいでしょう。

if文とswitch文の使い分け

- if文
複雑な条件を指定する必要がある場合
- switch文
分岐先が非常に多く、条件が単純な定数で表せる場合

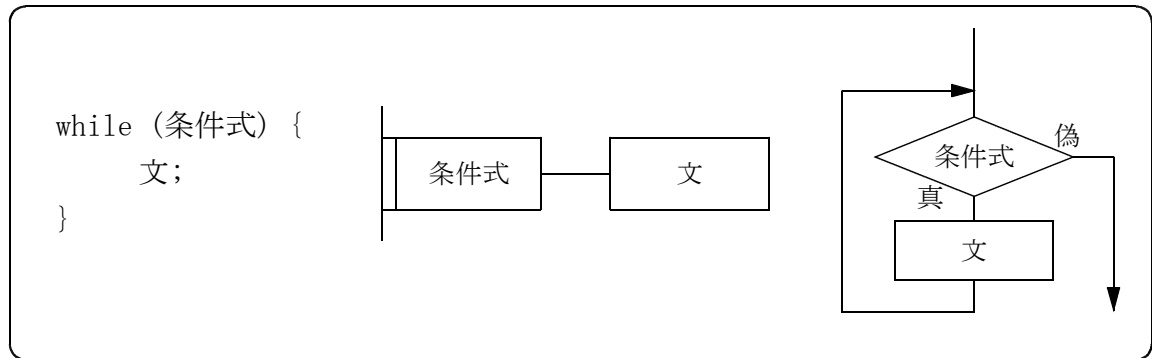
ポイント

- ☆switch文は_____という動作をする。
- ☆各caseの最後には_____を入れるようにする。
- ☆どのcaseにも該当しない場合は_____以下が実行される。
- ☆switch文のプログラムは、if文で_____。
- if文のプログラムは、switch文で_____。

⑤ while文：条件が成り立っている間繰り返す（前判定）

while文は「条件式が成り立っている間、ブロック内の文を実行し続け、条件式が成り立たなくなれば終了する」という繰り返しの動作をします。ブロックとは、{}で囲まれた範囲のことで、while文は次のように記述します。

なお、{}は繰り返し実行する文が1つの時は、if文と同様省略できますがif文と同じ理由で、後で述べる空文を使うとき以外は必ず書くようにしたほうがいいでしょう。



----- 例(5)-11 -----

```
#include <stdio.h>

main()
{
    int i, total; // i : 1, 2, 3, ..., 10 を順に作る
                // total: 1 + 2 + 3 + ... + 10 を格納

    total = 0;
    i = 1;
    while(i <= 10) { // iが1~10の間totalにiを加える
        total = total + i; // totalにはこれまでの合計が入る
        i = i + 1; // 次に加える値を更新
    }
    printf("1から 10までの和は %d です。\\n", total);

    return(0);
}
```

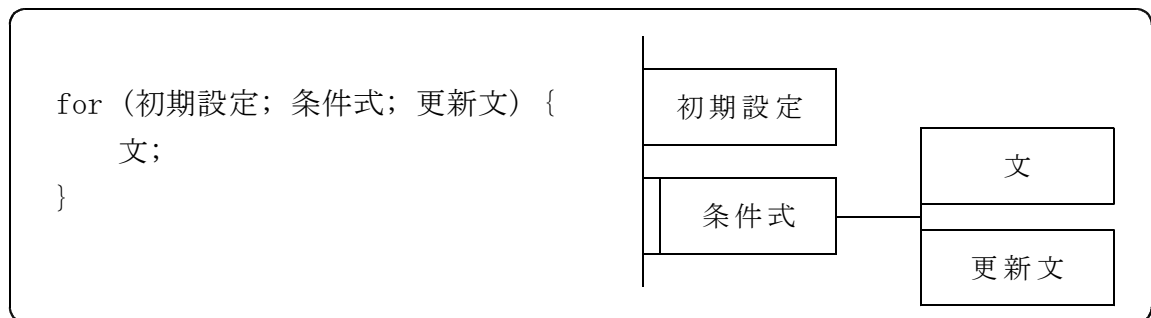
ポイント

☆while文は_____という動作をする。

⑥ for文：条件が成り立っている間繰り返す（前判定）

for文もwhile文と同様、「条件式が成り立っている間、ブロック内の文を実行し続け、条件式が成り立たなくなれば終了する」という繰り返しの動作をします。但し、for文では、初期設定と条件式を更新する文を、まとめて記述することができ、次のように記述します。

なお、{}は繰り返し実行する文が1つの時は、if文と同様省略できますがif文と同じ理由で、後で述べる空文を使うとき以外は必ず書くようにしたほうがいいでしょう。



----- 例(5)-12 -----

```
{  
#include <stdio.h>  
main()  
{  
    int a;  
  
    for( a = 1; a <= 1000000000; a *= 2 ) { // aを倍々に変化させる  
        printf("%10d\n", a);  
    }  
    return(0);  
}
```

初期設定や更新文は、次のようにカンマ(,)で区切り、複数の文を記述することもできます。

----- 例(5)-13 -----

```
{  
#include <stdio.h>  
main()  
{  
    int a, b;  
  
    // bが100以下の間、aをaを1から順に1ずつ増やしながらbに加え、毎回aとbの値を表示する  
    for( a = 1, b = 0; b <= 100; b = b + a, a = a + 1 ) {  
        printf("a:%3d, b:%3d\n", a, b);  
    }  
    return(0);  
}
```

しかし、例(5)-13の動作を実現するプログラムとしては、この書き方はあまり好ましくありません。それは、このプログラムの本質である「bに繰り返しaを加える」という意味が分かりにくくなっているからです。さらに、繰り返し本体に書かれているprintf文と、更新文に書かれているb = b + aという代入文のどちらが先に実行されるかもはっきりしません。したがって、例(5)-13は次のように書くべきです。

----- 例(5)-14 -----

```
#include <stdio.h>
main()
{
    int a, b;

    // bが100以下の間、aを1から順に1ずつ増やしながらbに加え、毎回aとbの値を表示する
    b = 0;
    for( a = 1; b <= 100; a = a + 1 ) {
        printf("a:%3d, b:%3d\n", a, b);
        b = b + a;
    }
    return(0);
}
```

なお、本質的にfor文とwhile文は全く同じものであり、for文はwhile文で次のように書き換えることもできます。

```
初期設定;
while (条件式) {
    文;
    更新文;
}
```

次の例(5)-15は、for文とwhile文の比較をしたプログラムです。

----- 例(5)-15 -----

```
#include <stdio.h>
main()
{
    int i, n;

    // iを1から10まで1ずつ増やしながら、nにiを加えていく

    /***** for 文 *****/
    for ( n = 0, i = 1; i <= 10; i = i + 1 ) {
        n = n + i;
    }
    printf("for文で求めた和は %d\n", n);

    /***** whlie 文 *****/
    n = 0;
    i = 1;
    while(i <= 10) {
        n = n + i;
        i = i + 1;
    }
    printf("while文で求めた和は %d\n", n);

    return(0);
}
```

while文とfor文の使い分け

- ・ while文
繰返す回数が特に決まっていない場合
- ・ for文
繰返す回数がある程度決まっている場合

さて、BASICでは次のような何もしないループを用いて、タイマとして使ったりします。

```
FOR T=1 TO 10000 : NEXT T
```

これをC言語で記述すると次のようになります。

```
for (i = 1; i <= 10000; i = i + 1)  
    ;
```

ここで「;」という、セミコロン以外何も書かれていない行があります。これを空文といい、「何の仕事もしないけれど、そこにひとつの文がある」という役目をします。for文は、その文法上「ひとつ以上の文を従えていなければならない」ということになっているため、このような空文を用いるのです。

なお、これを

```
for (i = 1; i <= 10000; i = i + 1) ;
```

と記述しても正しく動作しますが、ここでは「そこに空文がある」という意味が希薄になるため、あまり望ましくありません。

＝ ポイント ＝

☆for文は_____という動作をする。

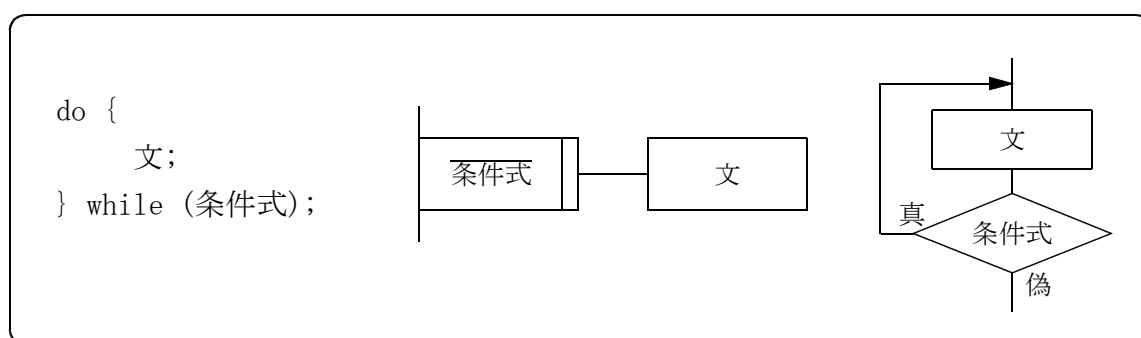
☆「;」だけの文を_____という役目をする。

⑦ do-while文：条件が成り立っている間繰返す（後判定）

do-while文もwhile文と同様、「条件式が成り立っている間、ブロック内の文を実行し続け、条件式が成り立たなくなれば終了する」という繰返しの動作をします。

しかし、while文とdo-while文では大きな違いが1つあります。それは、繰返しの終了をループの「最初で判断する（前判定）」か「最後で判断する（後判定）」かの違いです。結果として、while文では繰返しの処理が1度も実行されないことがあるのに対し、do-while文では必ず1度は実行されます。

do-while文は次のように記述します。なお、while（条件式）の後に付ける最後の「;」は忘れやすいので注意してください。



次のプログラムは、例(5)-11をdo-while文に書き直したものです。実行して同じ結果が得られることを確認してみましょう。

----- 例(5)-16 -----

```
#include <stdio.h>

main()
{
    int i, total; // i : 1, 2, 3, ..., 10 を順に作る
                // total: 1 + 2 + 3 + ... + 10 を格納

    total = 0;
    i = 1;
    do {
        total = total + i; // totalにはこれまでの合計が入る
        i = i + 1;         // 次に加える値
    } while(i <= 10);      // iが1~10の間totalにiを加える

    printf("1から 10までの和は %d です。\\n", total);

    return(0);
}
```

ポイント

☆do-while文は_____という動作をする。

☆while文は_____に判断するので、文が_____。

☆do-while文は_____に判断するので、文が_____。

⑧ インクリメント演算子・デクリメント演算子

これまで、繰り返し処理の中などで、変数をカウントアップする際、

```
i = i + 1;
```

のようにしていましたが、これは次のように記述することもできます。

```
i++;
```

または

```
++i;
```

この「++」をインクリメント演算子と呼びます。逆に、カウントダウンするには

```
i--;
```

または

```
--i;
```

とすることもできます。この「--」をデクリメント演算子と呼びます。

for文とインクリメント演算子を用いて例(5)-11を書き直す次のようになります。

----- 例(5)-17 -----

```
#include <stdio.h>
main()
{
    int i, total; // i : 1, 2, 3, ..., 10 を順に作る
                // total: 1 + 2 + 3 + ... + 10 を格納
    total = 0;
    for ( i = 1; i <= 10; i++ ) { // iのカウントアップにはインクリメント演算子を使う
        total = total + i;      // iが1～10の間totalにiを加える
    }
    printf("1から 10までの和は %d です。\\n",total);

    return(0);
}
```

インクリメント演算子やデクリメント演算子は文を簡潔に記述するのに便利ですが、これらを使うときには、注意が必要です。例えば、「i++」と「++i」を繰り返し処理の更新文として単独で使用する場合には、どちらも「iをカウントアップする」という同じ働きをします。しかし、通常の代入文や計算式の中で使用した場合には「カウントアップする」タイミングが異なるため、同じ結果にならないことがあります。

次の例を見てみましょう。

----- 例(5)-18 -----

```
#include <stdio.h>

main()
{
    int i, j, x, y;

    i = j = 5;

    x = i++; // x = i を実行後、i++ を実行
    y = ++j; // ++j を実行後、y = jを実行

    printf("i = %d\n", i); // i, j はともに 6 になる
    printf("j = %d\n", j);
    printf("x = %d\n", x); // x は 5、y は 6 になる
    printf("y = %d\n", y);

    return(0);
}
```

このプログラムの中で、インクリメントの演算は次の手順で実行されます。

- ・ 「x = i++;」 の場合
最初に「x = i」 → 次に「i++」
- ・ 「y = ++j」 の場合
最初に「++j」 → 次に「y = j」

そのため、iとjはどちらも6になりますが、xは5、yは6という異なる値になります。

このように、「++」や「--」変数の前に書くか後ろに書くかで意味が全く異なってきます。ですから、これらの演算子を代入文や算術式のなかで使用すると、混乱を招くもとなるだけであり、あまりメリットはありません。したがって、「++」や「--」は単独の文で使うことを原則とするようにしてください。

ポイント

☆「++」を_____「--」を_____と呼ぶ。

☆「i++」と「++i」では_____が異なるため、使用時には_____にする。

問題(5)-1

次の設問についてYes/Noで答えなさい。Noのものについてはどうして間違っているのかも説明しなさい。

- ① C言語で「もし～ならば」という処理をする時に使う構文は「for文」である。
- ② 単純if文において、条件式が偽のときは何の処理もしない。
- ③ 「<」「>=」のような記号を関係演算子と呼ぶ。
- ④ 「a=b」は「aとbは等しい」という条件式である。
- ⑤ if文やfor文において複数の文を記述する時は []でくくる。
- ⑥ 繰返す回数が決まっている場合はwhile文を使うとよい。

問題(5)-2

変数aに任意の整数をキーボードから入力し、

- ・ aが0未満なら「aは負数」
- ・ aが0以上100未満なら「aは0以上100未満」
- ・ aが100以上なら「aは100以上」

と表示するプログラムをif文を用いて作りなさい。(toi05-02. c)

問題(5)-3

例(5)-5のプログラムをswitch文を使って書き直しなさい。(toi05-03. c)

問題(5)-4

1から100までの奇数の和を求めるプログラムを、while文を用いて作りなさい。
(toi05-04. c)

問題(5)-5

問題(5)-4をfor文を使って記述しなさい。(toi05-05. c)

問題(5)-6

問題(5)-4を参考に、1から100までの偶数の和を求めるプログラムを、do-while文を用いて作りなさい。(toi05-06. c)

問題(5)-7

次のプログラムを実行すると、どんな結果が得られるか答え、説明を加えなさい。

---- ① toi05-07-01.c -----

```
#include <stdio.h>
main()
{
    int n = 50;
    if (n > 50) {
        printf("Rainy\n");
    } else {
        printf("Fine\n");
    }
    return(0);
}
```

---- ② toi05-07-02.c -----

```
#include <stdio.h>
main()
{
    int i;
    for (i = 0; i < 10; i++) {
        printf("i");
    }
    return(0);
}
```

---- ③ toi05-07-03.c -----

```
#include <stdio.h>
main()
{
    int p = 0;
    if (p = 0) {
        printf("ゼロ\n");
    } else {
        printf("ゼロでない\n");
    }
    return(0);
}
```

---- ④ toi05-07-04.c -----

```
#include <stdio.h>
main()
{
    int i;
    for (i = 0; i < 10; i++) {
        printf("%d\n", i * i);
    }
    return(0);
}
```

問題(5)-8

次のプログラムの文法的な誤りや意味的におかしい点、または好ましくない書き方をしている部分を指摘し、正しく動作するように修正しなさい。

---- ① toi05-08-01.c -----

```
#include <stdio.h>
main()
{
    int i = 0;
    int j = 1;
    if (i = 0 && j < 0) {
        printf('i = 0 かつ j < 0/n');
    }
    if (i <> 0 || j <> 0);
        printf('i, jともゼロ/n');
    return(0);
}
```

---- ② toi05-08-02.c -----

```
#include <stdio.h>
main()
{
    int i == 1;
    switch (i) {
        case '1';
            printf("i = 1\n");
        case 2,
            printf("i = 2\n");
        case3:
            printf("i = 3\n");
        default
            printf("error");
    }
    return(0);
}
```

----- ③ toi05-08-03.c -----

```
// 1から10までの和を求めるプログラム
#include <stdio.h>

main()
{
    int i,s;

    do {
        i = 0;
        s = s + i++ ;
    } while (i < 10)

    printf("1から10までの和は%d"です。\\n", s);

    return(0);
}
```

//

(6) 三角関数などの使用方法（数学関数・数学定数）

まず、これまでに学んだ知識だけで次の問題に取り組んでみてください。

問題(6)-1

整数 x の2乗を表示するプログラムを作りなさい。但し、C言語には累乗演算子は存在しないため、 x^2 のような書き方はできません。

(ヒント) x の2乗は、 x に x 自身を掛ける。 (toi06-01.c)

問題(6)-2

2つの整数 x , y をキーボードから入力し、 x の y 乗を表示するプログラムを作りなさい。但し、 $y \geq 0$ とします。 (toi06-02.c)

(ヒント) 繰り返し処理を利用する。

問題(6)-2では、 x の y 乗を求めるのに繰り返し処理を利用しましたが、標準ライブラリに用意されている数学関数利用する方法もあります。

ここでは、べき乗を計算する関数 `pow()` を使う例を示します。

----- 例(6)-1 -----

```
#include <stdio.h>
#include <math.h>
#include <hamakou.h>

main()
{
    double x, y;           // 数学関数で用いる変数はdouble型で宣言

    x = getfloat("x = ");  // getfloat() はdouble型の値を取得する
    y = getfloat("y = ");

    printf("%gの%g乗 = %g\n", x, y, pow(x, y));

    return(0);
}
```

変数 x, y を `double` 型で宣言しているのは、関数 `pow()` に渡す値（これを^{ひきすう}引数といいます）が `double` 型と決められているためです。

また、「`#include <math.h>`」という記述がありますが、これは今のところ、`pow()` などの数学関数や円周率 π のような数学定数を使用するのに必要なおまじないとして覚えておいてください。

主な数学関数や数学定数には、次のようなものがあります。

関数名	意 味
$\sin(x)$	xの正弦 単位は[rad]
$\operatorname{asin}(x)$	$\sin^{-1}(x)$
$\cos(x)$	xの余弦 単位は[rad]
$\operatorname{acos}(x)$	$\cos^{-1}(x)$
$\tan(x)$	xの正接 単位は[rad]
$\operatorname{atan}(x)$	$\tan^{-1}(x)$
$\exp(x)$	指数関数 e^x
$\log(x)$	xの自然対数 $\log_e(x)$, $\ln(x)$
$\log_{10}(x)$	xの常用対数 $\log_{10}(x)$
$\operatorname{pow}(x, y)$	xのy乗 x^y
$\operatorname{sqrt}(x)$	xの平方根 ($x \geq 0$) $\sqrt{x}$
$\operatorname{fabs}(x)$	xの絶対値
$\operatorname{fmod}(x, y)$	x/yの浮動小数点数の余り。結果の符号はxと同じ

[表6-1] 主な数学関数

※引数はすべてdouble型

定数名	意 味	値
M_PI	円周率 π	3.14159265358979323846
M_1_PI	$1/\pi$	0.31830988618379067154
M_E	自然対数の底 e (ネイピアの数)	2.7182818284590452354
M_LN10	$\log_e(10)$	2.30258509299404568402
M_LOG10E	$\log_{10}(e)$	0.43429448190325182765

[表6-2] 主な数学定数

ここでは、関数 $\operatorname{sqrt}()$ を使った次のプログラムを実行してみましょう。

----- 例(6)-2 -----

```

#include <stdio.h>
#include <math.h>

main()
{
    double x = 3.0;          // 直角三角形の底辺
    double y = 4.0;          // 直角三角形の高さ
    double z;                // 直角三角形の斜辺

    z = sqrt(x * x + y * y); // 三平方の定理
    printf("SQRT(%g^2 + %g^2) = %g\n", x, y, z);

    return(0);
}

```

また、次のようにすると、三角関数 $\sin \theta$ の波形を描かせることもできます。

----- 例(6)-3 -----

```
#include <stdio.h>
#include <math.h>
main()
{
    int t;                // 角度設定用
    int s;                // sin() の計算結果を格納
    double pi = M_PI;     // π の値(3.14159265358979323846)
    int i;                // カウンタ用変数

    for (t = 0; t <= 360; t += 15) {
        printf("%3d ", t);          // 角度の表示
        s = sin((double)t * pi / 180) * 20; // 表示のためにsin(t)の計算結果を20倍に拡大
        for (i = 1; i <= s + 20; i++) {
            printf(" ");            // sin(t)*20個分の「 」(スペース)を表示
        }
        printf("*\n");
    }
    return(0);
}
```

ポイント

☆数学関数を使用する時は「_____」を記述しなければならない。

☆数学関数の引数は「_____」型である。

問題(6)-3

整数0～100について、 $\sqrt{x}$ 、 $\sqrt[3]{x}$ 、 $\log x$ 、 $\log_{10} x$ を一覧表示するプログラムを作りなさい。
(toi06-03.c)

問題(6)-4

- (1) 例(6)-3を修正して $\cos \theta$ のグラフを描くプログラムを作りなさい。(toi06-04-01.c)
- (2) $\tan \theta$ のグラフを描くプログラムを作りなさい。 $\tan \theta$ は θ が $\pi/2$ と $3\pi/2$ のとき、 ∞ となるのに注意すること。(toi06-04-02.c)

問題(6)-5

次のプログラムの誤りを指摘し、期待通りの動作をするよう修正しなさい。

----- toi06-05.c -----

```
#include <stdio.h>
main()
{
    int i, j;

    for (i = 1; i >= 10); {
        for (j = 1; j > 5); {
            printf("%dの %d乗は %dです\n", i, j, pow(i, j));
        }
    }
    return(0);
}
```

(7) まとまった同型データの扱い方 (配列変数)

変数に5人の身長を代入する場合を考えます。5人分の身長ですから、5つの変数が必要になります。プログラムにすると次のようになりますね。

----- 例(7)-1 -----

```
#include <stdio.h>

main()
{
    float t1, t2, t3, t4, t5;    // 5つの変数を用意する

    t1 = 148.5;
    t2 = 163.2;
    t3 = 183.5;
    t4 = 156.7;
    t5 = 172.5;

    printf("1人目の身長 : %gcm\n", t1);
    printf("2人目の身長 : %gcm\n", t2);
    printf("3人目の身長 : %gcm\n", t3);
    printf("4人目の身長 : %gcm\n", t4);
    printf("5人目の身長 : %gcm\n", t5);

    return(0);
}
```

ところが、これが100人, 200人ともなると、このようにはいかないことはすぐ気がつきます。何か他の方法でこれらの変数を用意しなくてはなりません。

このような同じ型のデータを複数扱いたい時には、変数を配列(array)として宣言します。配列を使って例(7)-1を書き直すと次のようになります。

----- 例(7)-2 -----

```
#include <stdio.h>

main()
{
    float t[5];    // 5つの変数を用意する
    int i;         // カウンタ変数として i を用意

    t[0] = 148.5;
    t[1] = 163.2;
    t[2] = 183.5;
    t[3] = 156.7;
    t[4] = 172.5;

    // for文を使ってデータを出力
    for (i = 0; i < 5; i++) {
        printf("%d人目の身長 : %gcm\n", i + 1, t[i]);
    }
    return(0);
}
```

配列変数を宣言する時は、その変数名の後ろに[数値]を付けるだけです。上の例では

```
float t[5];
```

の部分がそうです。[]内の数値のことを添字(subscript)、t[0], t[1]のような各変数を配列の要素(element)と呼びます。また、**添え字の番号は0から始まります**。

ここで注意しなければならないのは、float t[5]によって**5個のfloat型変数が用意される**のであって、t[0]~t[5]の配列変数が用意されるわけではない、という点です。実際に**使用できる配列要素は、t[0]~t[4]の5個**になります。

当然ですが、1つの配列を構成する要素はすべて同じ型になります。1~3個目は整数型、4~7個目は実数型にするというようなことはできません。

なお、配列変数も普通の変数と同様、宣言時に初期化できます。初期化は各要素の初期値を順に「,」(カンマ)で区切り、全体を中括弧{}で囲むことにより行います。この方法を使って例(7)-2を書き直すと次のようになります。

----- 例(7)-3 -----

```
#include <stdio.h>
main()
{
    // 配列tを t[0]=148.5, t[1]=163.2, t[2]=183.5, t[3]=156.7, t[4]=172.5 で初期化
    float t[5] = {148.5, 163.2, 183.5, 156.7, 172.5};
    int i; // ループカウンタ
    for (i = 0; i < 5; i++) {
        printf("%d人目の身長: %gcm\n", i + 1, t[i]);
    }
    return(0);
}
```

また、配列要素を宣言時にすべて0で初期化するときに関り、次のような記述ができます。

```
float t[5] = {};
```

 // 全要素 (t[0]~t[4]) を0で初期化

任意の値で全要素を初期化しようとして次のような記述をしても、先頭要素以外はすべて0になるので注意してください。

```
float t[5] = {2.0};
```

 // 先頭要素t[0]のみ2.0、残りはすべて0になる

配列を使うメリットは、先程も説明したようにまとまったデータを一括して扱える点にあります。例えば次のような場合です。

配列を使うべき処理

- データを蓄えておいて後で再利用する場合
- データの任意の要素を参照する場合

具体例を挙げると、データの並べ替え(sorting)などは、配列を使用すると処理が容易になります。例(7)-4にプログラム例を示しておきます。

----- 例(7)-4 -----

```
#include <stdio.h>

main() // バブルソート
{
    float t[5] = {148.5, 163.2, 183.5, 156.7, 172.5}; // 5つのデータを用意する
    int i, j; // データ配列の添え字として使用
    float temp; // 入れ替えのための一時使用変数

    for (i = 0; i < 4; i++) {
        for (j = 0; j < 4 - i; j++) {
            if (t[j] < t[j + 1]) {
                temp = t[j]; // --+
                t[j] = t[j + 1]; // |- 入れ替え処理
                t[j + 1] = temp; // --+
            }
        }
    }

    printf("*** 身長の高い順 ***\n");
    for (i = 0; i < 5; i++) {
        printf("%d人目の身長: %gcm\n", i + 1, t[i]);
    }

    return(0);
}
```

一方で、注意しなければならない点もいくつかあります。

第1に、配列は宣言した要素数分のメモリ領域を実行時にあらかじめ確保するため、要素数があまりにも多いと宣言した要素数に必要な領域を確保できず、処理不能となることがある点です。次の例は、用意した配列の要素をすべて0で初期化する単純なプログラムです。このプログラムはコンパイルはできますが、あまりにも大きな配列を宣言しているため、実行すると“Segmentation Fault (core dumped)”となり異常終了します。

----- 例(7)-5 -----

```
#include <stdio.h>

main()
{
    int a[1000000000], i;

    for (i = 0; i < 1000000000; i++) {
        a[i] = 0;
    }

    return(0);
}
```


第2に、配列要素を参照する際、その添え字の値が宣言した範囲内にあるかどうかチェックされないという点です。実行中に添え字の値が宣言した範囲を超えた場合、何が起るかわかりません。次の例(7)-6-1, 例(7)-6-2を実行してみてください。

----- 例(7)-6-1 -----

```
{
#include <stdio.h>

main()
{
    int a[10], i;

    i = 0;
    while (1) {
        a[i] = i;
        printf("a[%d] = %d\n", i, a[i]);
        i = i + 1000;
    }

    return(0);
}
```

----- 例(7)-6-2 -----

```
{
#include <stdio.h>

main()
{
    int a[10], i;

    i = 9;
    while (1) {
        a[i] = i;
        printf("a[%d] = %d\n", i, a[i]);
        i = i - 1000;
    }

    return(0);
}
```

第3に、配列の宣言時に要素数を可変にすることはできないという点です。次のようにキーボードから入力した値によって、配列の大きさを自由に変えることはできません。

```
{
#include <stdio.h>
#include <hamakou.h>

main()
{
    int n;
    n = getint("要素数を入力してください：");
    int a[n];
    :
}
```

(参考：C言語標準仕様と可変長配列)

C言語は、1990年にISOで標準仕様として規格化されました。その後、何度か誤りの訂正や `wchar_t` 型の追加といった追補がなされた後、さらに使いやすくするための新しい機能が検討・追加され、1999年に新しいC言語仕様「ISO/IEC 9899:1999 Programming Language C」（通称 C99）として、1999/12/01 付けで規格化されました。C99には数多くの新機能が導入され、そのなかに可変長配列も含まれています。

C99規格に基づいたC言語では、次の例(7)-7のようなプログラムも可能になりました。キーボードから入力した値によって、配列の大きさが変化する様子を `sizeof` 演算子で実際に確かめることができます。ただし、可変長配列が使用できる場所は、変数宣言の最後に1つだけに限られます。

しかし、2011年に規格化されたC言語仕様「ISO/IEC 9899:2011」（通称 C11）では、可変長配列はオプション扱いとなりました。そのため、この規格に基づいて実装されたCコンパイラでは、可変長配列が使えるとは限りません。さらに、2014年に規格化されたC++言語仕様（C言語の規格はこの時点からC++言語に統合されています）「ISO/IEC 14882:2014」では、可変長配列は削除されてしまいました。

本校で使用しているC/C++コンパイラ (`gcc-8.5.0`) は、C++14規格に準拠したものであり、C++コンパイラ (`c++`, `g++` コマンド) では特別なオプションを指定しない限り、可変長配列は使えません。ただし、Cコンパイラ (`cc`, `gcc` コマンド) はC99規格の互換モードで動作するため、例(7)-7のプログラムを問題なくコンパイル、実行することができます。

----- 例(7)-7 -----

```
#include <stdio.h>
#include <hamakou.h>

main()
{
    int n, i;
    n = getint("要素数を入力してください:");
    int a[n];

    printf("a[]の大きさ: %dバイト\n", sizeof(a));
    printf("a[]の要素数: %d個\n", sizeof(a) / sizeof(int));
    for (i = 0; i < n; i++) {
        a[i] = i;
    }
    for (i = 0; i < n; i++) {
        printf("a[%d] = %d\n", i, a[i]);
    }

    return(0);
}
```

ポイント

- ☆同様の複数データを扱うには_____を使うとよい。
- ☆配列の [] 内の数値を_____, 配列内の各変数を_____という。
- ☆「int a[10];」という文は、a[___]～a[___]の____個の変数を用意する。
- ☆配列を使う際の注意点は、次の3つである。
 - _____
 - _____
 - _____
- 但し、C99規格準拠のC言語では、_____を使用できる。

問題(7)-1

次の設問にYes/Noで答え、理由も説明しなさい。

「float b[5];」 とすると、b[1]～b[5]の配列が用意される。

問題(7)-2

例(7)-4のプログラムで使っている「身長の高い順に並び替える」手順を、紙上で手作業にて行い、その様子をレポートに記しなさい。

問題(7)-3

例(7)-4を身長の低い順に並び替えるプログラムに変更しなさい。(toi07-03.c)

問題(7)-4

例(7)-4のプログラムを、データをすべてキーボードから配列変数に入力できるように変更しなさい。またデータ数も可変にできるようにしてみなさい。(toi07-04.c)

問題(7)-5

例(7)-6-1および例(7)-6-2のプログラムの実行結果が、なぜそうなったのか説明しなさい。

問題(7)-6

君たちの使っているシステムでは、最大どのぐらいの要素数の整数配列が宣言できるか、例(7)-7を変更して確かめてみなさい（キーボードから入力した可変長配列の要素数のみを表示するようにする）。

(8) 文字データの扱い (文字・文字列)

① 文字と文字定数

定数に文字定数があるように、変数にも文字変数があります。まずは、文字「A」を表示する次のプログラムを実行してみましょう (例(8)-1)。文字変数は、すでに学んだようにchar型で宣言します。また、文字の表示は書式制御文字に「%c」を用います。

----- 例(8)-1 -----

```
#include <stdio.h>

main()
{
    char moji;           // 文字を格納する変数定義

    moji = 'A';          // 文字 A を変数 moji に代入
    printf("%c\n",moji); // 変数 moji を表示

    return(0);
}
```

では、この文字変数mojiを、整数を10進数表示するための書式制御文字「%d」で表示するとどうなるのでしょうか(例(8)-2)。

----- 例(8)-2 -----

```
#include <stdio.h>

main()
{
    char moji;           // 文字を格納する変数定義

    moji = 'A';          // 文字 A を変数 moji に代入
    printf("%d\n",moji); // 変数 moji を「%d」で表示

    return(0);
}
```

コンパイルも実行もエラーなく正常に終了し、画面には「65」と表示されるはずで
す。この数値は文字「A」の文字コード (本校の環境では UTF-8 エンコーディング)
を10進数で表したものです。

実は文字は定数でも変数でも、このように内部においては文字コードを表す整数の
データでしかありません。したがって、次のような文字に対して整数演算を行うよう
なプログラムも作成可能です (例(8)-3)。

----- 例(8)-3 -----

```
#include <stdio.h>

main()
{
    char moji;          // 文字を格納する変数定義

    moji = 'A';          // 文字 A を変数 moji に代入
    moji++;              // 変数 moji をインクリメント
    printf("%c\n",moji); // 変数 moji を文字型で表示
    moji += 3;           // 変数 moji を +3
    printf("%c\n",moji); // 変数 moji を文字型で表示

    return(0);
}
```

これを利用して文字の一覧を表示する、次のプログラムを実行してみましょう
(例(8)-4)。

----- 例(8)-4 -----

```
#include <stdio.h>

main()
{
    char moji; // 文字を格納する変数定義
    int i,j;   // カウンタ変数定義

    printf("    0 1 2 3 4 5 6 7 8 9 A B C D E F\n");
    printf("    +-----\n");
    for (i = 2; i <= 7; i++) {
        printf("%X |", i * 16);
        for (j = 0; j <= 0xf; j++) {
            moji = i * 16 + j; // 文字コードを生成
            printf(" %c",moji); // 変数 moji を文字型で表示
        }
        printf("\n");        // 行末で改行
    }

    return(0);
}
```

② 文字列と文字配列

定数の中には、文字定数の他に文字列定数があることは既に説明しました。同様に、変数にも文字型と文字列型があるかという点、残念ながらC言語には文字型変数はありませんが、文字列型変数というのは存在しません。その代わり、C言語では文字型の配列変数を利用して文字列を表現します。

次のプログラムは、文字と文字列、文字変数の関係を確認するものです。表示結果を予想してから、コンパイルして実行してみてください。

----- 例(8)-5 -----

```
#include <stdio.h>

main()
{
    int i;                                // カウンタ用変数
    char a[5] = {'a','b','c','d','\0'}; // a[]の宣言と初期化
    char b[4] = {'e','f','g','h'};      // b[]の宣言と初期化
    char c[] = "ijkl";                  // c[]の宣言と初期化

    for (i = 0; i < 5; i++) { // 配列 a を1文字ずつ表示
        printf("変数 a[%d]の内容は %c です。\\n", i, a[i]);
    }
    for (i = 0; i < 4; i++) { // 配列 b を1文字ずつ表示
        printf("変数 b[%d]の内容は %c です。\\n", i, b[i]);
    }
    for (i = 0; i < 5; i++) { // 配列 c を1文字ずつ表示
        printf("変数 c[%d]の内容は %c です。\\n", i, c[i]);
    }
    printf("配列 a の内容は %s です。\\n", a); // 配列 a を文字列として表示
    printf("配列 b の内容は %s です。\\n", b); // 配列 b を文字列として表示
    printf("配列 c の内容は %s です。\\n", c); // 配列 c を文字列として表示

    return(0);
}
```

(予想)

(実行結果)

文字列はchar型で宣言された配列変数に格納されます。{ }でくくられた文字データは、配列へ左側から順番に要素番号の[0],[1], … に対応して代入されます。

文字配列の初期値は、配列a,bのように、配列変数の各要素をそれぞれ一文字ずつ初期化したり、配列cのように文字列定数で初期化したりできます。いずれの場合も、配列要素1つだけを%c(char型)で出力すれば文字で、配列全体を%s(文字列型)で出力すれば文字列として表示されます。ただし、char型の配列変数を文字列として取り扱うには、その最後に「\0」(ヌル文字)を付加して末尾をはっきりさせる必要があります。

○文字配列 a[], b[], c[] 初期化後の様子

	[0]	[1]	[2]	[3]	[4]	
a	'a'	'b'	'c'	'd'	'\0'	← a[]は文字列
b	'e'	'f'	'g'	'h'		← b[]は4文字を格納した配列
c	'i'	'j'	'k'	'l'	'\0'	← c[]は文字列

また、変数は宣言した順スタックの底から(つまり、メモリ番地の大きい方から小さい方に向かって)確保されていきます。そのため、この例題で宣言された配列は、メモリの先頭から見るとc[], b[], a[]の順に並ぶことになります。

○文字配列 a[], b[], c[] がメモリ (スタック) 内に確保された様子

	c[0]	c[1]	c[2]	c[3]	c[4]	b[0]	b[1]	b[2]	b[3]	a[0]	a[1]	a[2]	a[3]	a[4]	
先頭	'i'	'j'	'k'	'l'	'\0'	'e'	'f'	'g'	'h'	'a'	'b'	'c'	'd'	'\0'	底

ここで、もしb[]のように「\0」がない文字配列を文字列として出力しようとする、「\0」があるところまでを文字列として判断してしまいます。そのため、b[]の次に領域が確保されている a[]までを表示してしまいます(但し、C言語の処理系やコンパイルオプションによっては、異なる結果になることがあります)。また、c[]のように直接文字列定数で初期化した場合には、末尾に自動的に「\0」が付けられます。

問題(8)-1

次の設問についてYes/Noで答えなさい。Noのものについては、どうして間違っているのかも説明しなさい。

- ① 'a' と"a"は同じ内容である。
- ② 「char x[5] = {'a','b','c','\0'};」 と 「char x[5] = "abc";」 は同じである。
- ③ 「char x[4] = {'a','b','c','d'};」 と 「char x[] = "abcd";」 は同じである。
- ④ 「int x[4] = {'a','b','c','\0'};」 と 「char x[4] = "abc";」 は同じである。
- ⑤ 「char x[5] = {'a','b','c','d'};」 のとき、x[2]の内容は 'c' である。
- ⑥ 「int x[5] = {'a','b','c','d'};」 のとき、x[2]の内容は 'c' である。
- ⑦ 「int x[5] = "abcd";」 のとき、x[2]の内容は 'c' である。

問題(8)-2

次の変数宣言を行うと、配列x[]がどうなるか正確に述べなさい。

```
char x[] = {'1','2','3','4','5'};
```

問題(8)-3

次のプログラムをできるだけ簡潔に作りなさい。ただし、表示に用いるのは文字または文字列のみ（%cまたは%sで表示する）とし、整数などの数値を%dで直接表示するのは反則とします。

①下図のように画面表示するプログラム
(toi08-03-01.c)

```
0 : 9
1 : 8
2 : 7
3 : 6
4 : 5
5 : 4
6 : 3
7 : 2
8 : 1
9 : 0
```

②下図のように画面表示するプログラム
(toi08-03-02.c)

```
0 :
1 : 1
2 : 12
3 : 123
4 : 1234
5 : 12345
6 : 123456
7 : 1234567
8 : 12345678
9 : 123456789
```


(9) 二次元配列の利用（二次元配列、文字列の配列）

次のような一覧表のデータを処理する場合を考えます。この表は、1年生から3年生までの各クラスの生徒数とその合計を、学年別およびクラス別に集計したものです。

	1	2	3	4	5	6	7	8	合計
1年	41	40	40	40	40	41	41	42	325
2年	39	38	34	40	37	39	35	39	301
3年	30	39	31	40	38	40	33	36	287
合計	110	117	105	120	115	120	109	117	913

[表9-1] 生徒在籍者数

もし、ある特定の学年（例えば2年生）のみのデータを集計すればよいのであれば、

```
int zaiseki[8] = {39, 38, 34, 40, 37, 39, 35, 39}; // 2年生のデータ
```

という具合に配列変数を用意し、次のようなプログラム（例(9)-1）を作ればよいでしょう。

```
----- 例(9)-1 -----
#include <stdio.h>

main()
{
    int i; // クラスの番号
    int s; // 合計を格納
    int zaiseki[8] = {39, 38, 34, 40, 37, 39, 35, 39}; // 2年生のデータ

    s = 0; // 合計をクリア
    for (i = 0; i < 8; i++) {
        s = s + zaiseki[i];
    }
    printf("2年生の生徒数は %d人です。\\n", s);

    return(0);
}
```

しかし、各学年ごとに集計したい時はちょっと工夫が必要になります。
例えば、次のように学年ごとに配列変数を用意したとすると、

```
int zaiseki1[8] = {41, 40, 40, 40, 40, 41, 41, 42}; // 1年生のデータ
int zaiseki2[8] = {39, 38, 34, 40, 37, 39, 35, 39}; // 2年生のデータ
int zaiseki3[8] = {30, 39, 31, 40, 38, 40, 33, 36}; // 3年生のデータ
```

プログラムは次のようになります（例(9)-2）。

----- 例(9)-2 -----

```
#include <stdio.h>

main()
{
    int i;          // クラスの番号
    int s[3] = {};  // 学年別の合計を格納(0で初期化)
    int zaiseki1[8] = {41, 40, 40, 40, 40, 41, 41, 42}; // 1年生のデータ
    int zaiseki2[8] = {39, 38, 34, 40, 37, 39, 35, 39}; // 2年生のデータ
    int zaiseki3[8] = {30, 39, 31, 40, 38, 40, 33, 36}; // 3年生のデータ

    for (i = 0; i < 8; i++) {
        s[0] = s[0] + zaiseki1[i]; // 1年生の集計
        s[1] = s[1] + zaiseki2[i]; // 2年生の集計
        s[2] = s[2] + zaiseki3[i]; // 3年生の集計
    }

    for (i = 0; i < 3; i++) {
        printf("%d年生の生徒数は %d人です。\\n", i + 1, s[i]);
    }

    return(0);
}
```

問題(9)-1

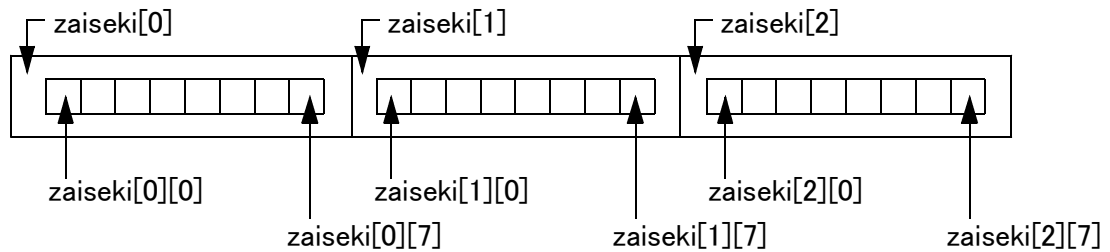
例(9)-2を修正して、クラス別の生徒数も集計するプログラムを作りなさい。

このとき、必要なら新たに変数や配列を宣言しても構いません。但し、あらかじめ用意されている配列には変更を加えないものとします。 (toi09-01.c)

これまでの方法では、小学校のように6学年もある場合や、一般的にもっと大きな表を扱いたいときには、表の行数に合わせて列変数を増やしていかなければならず、現実的ではありません。また、問題(9)-1のようにクラス別の集計をしたい時や、特定のクラスについてのみ集計したいという時も都合が悪いことが分かります。

このように、同種類のデータが二次元の表のように整理されているデータを扱いたい時は、配列の中にさらに配列を格納した、二次元配列を用いるとうまくいきます。この例の場合、具体的には `int zaiseki[3][8]` という配列の宣言をします。これは、[図9-1]のように「要素数3の配列に要素数8の整数型の配列を詰め込んだものを用意する」ということを意味します。

なお、二次元配列に対して、これまで取り上げてきた配列を一次元配列と呼びます。



〔図9-1〕 二次元配列zaiseki[3][8]

この2次元配列の初期化は、次のようにして行います。

```
int zaiseki[3][8] = {
    {41, 40, 40, 40, 40, 41, 41, 42}, // 1年生のデータ
    {39, 38, 34, 40, 37, 39, 35, 39}, // 2年生のデータ
    {30, 39, 31, 40, 38, 40, 33, 36}  // 3年生のデータ
};
```

これにより、各クラスの生徒数は〔表9-2〕のように格納されます。

	1	2	3	4	5	6	7	8	合計
1年	41	40	40	40	40	41	41	42	325
2年	39	38	34	40	37	39	35	39	301
3年	30	39	31	40	38	40	33	36	287
合計	110	117	105	120	115	120	109	117	913

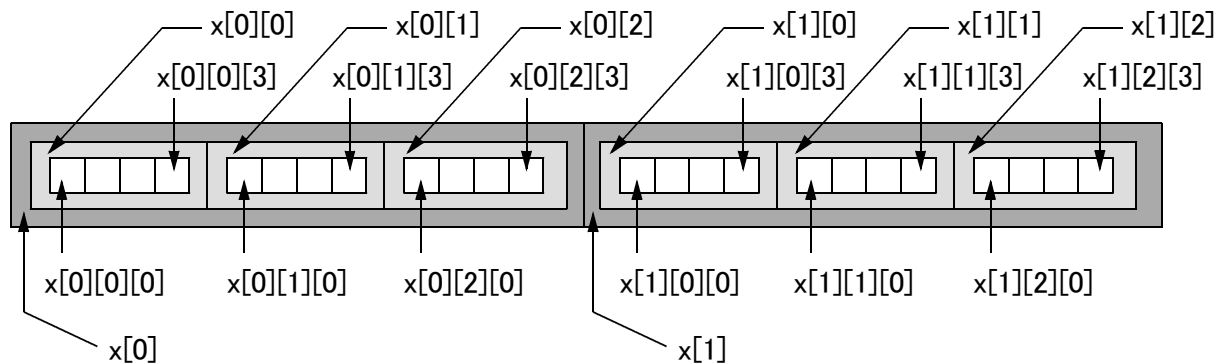
〔表9-2〕 生徒在籍者数

（多次元配列）

二次元配列は、いわば一次元配列の中にさらに一次元配列を入れ子にしたものですが、C言語ではこの入れ子の数に制限はありません。このように、配列を多重に入れ子にしたものを多次元配列といいます。

例えば、`int x[2][3][4]`のような三次元配列を作ることができます。この場合、4つの要素からなる一次元配列を3つ持つ二次元配列が作られ、さらにこの二次元配列を2つ持つような配列を用意したことになります（〔図9-2〕）。

しかし、多次元配列を多用すると、データがどのように格納されているのかイメージできなくなります。そのため、かえってプログラムが読みにくくなったり、プログラム自体が複雑になることもあり、保守の妨げになるので避けた方がいいでしょう。多次元配列を用いる場合でも、三次元までとし、それ以上の次元の配列が必要になるときは、後に学習する構造体を用いるようにしましょう。



[図9-2] 三次元配列 $x[2][3][4]$

問題(9)-1のプログラムを二次元配列を用いると次のように書けます。

----- 例(9)-3 -----

```
#include <stdio.h>

main()
{
    int i, j;          // iはクラスの番号, jは学年の番号
    int s[3] = {};     // 学年別の合計を格納(0で初期化)
    int c[8] = {};     // クラス別の合計を格納(0で初期化)
    int zaiseki[3][8] = {
        {41, 40, 40, 40, 40, 41, 41, 42}, // 1年生のデータ
        {39, 38, 34, 40, 37, 39, 35, 39}, // 2年生のデータ
        {30, 39, 31, 40, 38, 40, 33, 36}  // 3年生のデータ
    };

    for (j = 0; j < 3; j++) {          // jは学年の番号
        for (i = 0; i < 8; i++) {      // iはクラスの番号
            s[j] = s[j] + zaiseki[j][i]; // j+1学年の合計
            c[i] = c[i] + zaiseki[j][i]; // i+1組の合計
        }
    }

    for (i = 0; i < 3; i++) {
        printf("%d年生の生徒数は %d人です.\n", i + 1, s[i]);
    }

    for (i = 0; i < 8; i++) {
        printf("%d組の生徒数は %d人です.\n", i + 1, c[i]);
    }

    return(0);
}
```

ところで、前節(8)で説明したように、文字列を扱うには文字型の配列変数を使用する必要があります。それでは、複数の文字列をまとめて扱うにはどうしたらよいでしょうか？もう分かりますね。文字型の二次元配列を用意してやればいいわけです。

例えば、

"Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"
という7個の文字列を扱うには

```
char weekname[7][10] = {
    "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"
};
```

のような配列を用意します。これを用いたプログラム例を次に示します（例(9)-4）。

----- 例(9)-4 -----

```
#include <stdio.h>
main()
{
    int week;    // 曜日を 0~6 で表す
    int i = 0;   // カウンタ変数宣言
    char weekname[7][10] = {
        "Sunday",
        "Monday",
        "Tuesday",
        "Wednesday",
        "Thursday",
        "Friday",
        "Saturday"
    };

    // 各曜日を 1 文字ずつ表示
    printf("***** 1 文字ずつ表示 *****\n");
    for (week = 0; week < 7; week++) {
        i = 0;
        while(weekname[week][i] != '\0') {
            printf("%c", weekname[week][i]);
            i++;
        }
        printf("\n");
    }
    // 各曜日を文字列として表示
    printf("***** 文字列として表示 *****\n");
    for (week = 0; week < 7; week++) {
        printf("%s\n", weekname[week]);
    }
    return(0);
}
```

この例で、文字配列の添字を一つ省略した場合（例(9)-4太字）、それは1つの文字ではなく、文字列を表していることに注意してください（[図9-3] 参照）。

		weekname[0][0]						weekname[0][6]		
weekname[0]	S	u	n	d	a	y	\0			
weekname[1]	M	o	n	d	a	y	\0			
weekname[2]	T	u	e	s	d	a	y	\0		
weekname[3]	W	e	d	n	e	s	d	a	y	\0
weekname[4]	T	h	u	r	s	d	a	y	\0	
weekname[5]	F	r	i	d	a	y	\0			
weekname[6]	S	a	t	u	r	d	a	y	\0	
		weekname[6][0]						weekname[6][8]		

[図9-3] 複数の文字列を扱う二次元配列

もう一つ二次元配列を利用する例を示しておきます。以前、例(6)-3で *sin* 波形を縦形のグラフで描きましたが、これをよく見かける横形で表示させるには、次のプログラムのように二次元配列を用いることで可能になります。

----- 例(9)-5 -----

```
#include <stdio.h>
#include <math.h>

main()
{
    double rd = M_PI / 180.0;      // ラジアン変換用定数設定
    char graph[21][37];           // マス目が21行×37列のグラフ用紙を用意
    int x, y; // x座標(列), y座標(行) グラフ用紙上の点(x, y)は、graph[y][x]

    /**** グラフ用紙を空白で埋め、x軸を描く *****/
    for (y = 0; y <= 20; y++) {
        for (x = 0; x <= 36; x++) {
            graph[y][x] = ' '; // 空白で埋める
        }
    }
    for (x = 0; x <= 36; x++) {
        graph[10][x] = '-'; // グラフ用紙の10行目にx軸を描く
    }

    /**** sin(x)を計算してグラフ用紙に「*」をプロット *****/
    // 0° ~360° まで10° ごとにsin(x)を計算
    for (x = 0; x <= 36; x++) {
        // y座標を10倍に拡大し、y軸の上方向に10行分平行移動
        // y座標を四捨五入して整数にするために、0.5加えて切り捨てる
        y = (int)(10 - 10 * sin(x * 10 * rd) + 0.5);
        graph[y][x] = '*';
    }
    printf("\n");

    /**** グラフ用紙全体を表示する *****/
    // この部分は、グラフ用紙の初期化を工夫すればもっと簡潔に書ける <<問題(9)-6>>
    for (y = 0; y <= 20; y++) {
        for (x = 0; x <= 36; x++) {
            printf("%c", graph[y][x]);
        }
        printf("\n");
    }
    return(0);
}
```

＝ ポイント ＝

☆配列のうち、添字が1つのものを_____、2つのものを_____という。

☆二次元配列は、_____の中にさらに_____を_____にしたものである。_____さらに多重にした配列を、_____という。

☆文字配列中のデータを「文字列」として表示するには、配列の添字を_____する。

問題(9)-2

次の設問について Yes/No で答え、Noのものについてはどうして間違っているのか説明しなさい。

- ① 「int a[5][7];」 と宣言すると、a[0][0]～a[5][7]の要素が使用できる。
- ② 「int b[2][3][4];」 と宣言したとき最も内側の1次元配列の要素数は2である。

問題(9)-3

次のプログラムは、3行5列の表に格納されている整数値について、各行と列の合計を求め、それぞれ最終行と最終列に表示しようとしています（実行例参照）。

このプログラムの誤りを指摘し、期待通りの動作をするよう修正しなさい。

(toi09-03.c)

```
----- toi09-03.c -----
#include <stdio.h>
main() // 3行5列の表の各行と列の合計を、最終行と最終列に格納後、一覧表示する
{
    int t[5][3] = {{10, 20, 30, 40, 50}, {11, 12, 13, 14, 15}, {21, 22, 23, 24, 25}};

    for (row = 0; row < 3; row++) {
        for (col = 0; col < 5; col++) {
            t[5][col] = t[5][col] + t[row][col]; // 行データの合計
            t[row][3] = t[row][3] + t[row][col]; // 列データの合計
        }
        t[5][3] = t[5][3] + t[5][row]; // 合計の合計を求める
    }
    for (row = 0; row < 3; row++) { // 表中のデータを合計と共に表示する
        for (col = 0; col < 5; col++) {
            printf("%3d ", t[row][col]);
        }
        printf('\n');
    }
    return(0);
}
```

(実行例)

10	20	30	40	50	150
11	12	13	14	15	65
21	22	23	24	25	115
42	54	66	78	90	330

問題(9)-4

[表9-1] において、在籍人数が最も多いクラスと最も少ないクラスの学年・組・人数を表示するプログラムを、二次元配列を用いて作りなさい。

但し、同人数のクラスがあった場合には、最も低学年でクラスの番号が最も小さいクラスを表示するものとします。

(toi09-04.c)

問題(9)-5

キーボードより n 個 ($0 < n \leq 100$) の文字列を入力し、それらを全て逆順にした文字列を表示するプログラムを作りなさい。

但し、入力した文字列は一度 [図9-3] のような二次元配列に格納すること。また、**逆順にした文字列の表示は %s で行いなさい**。なお、1つの文字列の長さは255文字を超えないものとします。

(toi09-05.c)

(実行例) 入力する文字列の数 : 3
文字列1: abcdefghijklmn
文字列2: 0123456789
文字列3: OPQRSTUVWXYZ
文字列1の逆順: nmlkjihgfedcba
文字列2の逆順: 9876543210
文字列3の逆順: ZYXWVUTSRQPO

問題(9)-6

例(9)-5 のプログラムは、グラフ用紙の初期化を工夫することで、グラフ用紙全体を表示する処理をもっと簡潔に書くことができます。このプログラムを改良しなさい。

(toi09-06.c)

(ヒント)

例(9)-5 では、二次元配列 `graph[][]` に格納されている文字 (' ', '*', '-' のいずれか) を、行方向に1文字ずつ表示することでグラフ用紙全体を描画しています。これを文字列単位で表示するにはどうしたらよいでしょうか？

なお、この方法で改良を行う場合、用意するグラフ用紙の大きさにも注意を払う必要があります。

////////////////////////////////////

(10) データの入力方法 (文字単位の入出力、数値を正しく入力する)

① 1 文字単位の入出力

次のプログラムは、標準入力から入力した文字を標準出力へ出力するものです。

----- 例(10)-1 -----

```
#include <stdio.h>

main()
{
    int c; // 入力文字格納用変数

    while ((c = getchar()) != EOF) {    // EOFを入力するまで繰り返す
        putchar(c);                    // 標準出力に 1 文字出力
    }
    return(0);
}
```

ここでの標準入力はキーボード、標準出力はキャラクタ端末（端末エミュレータ）になります。標準入力から 1 文字のデータを読み込むためにgetchar()関数、標準出力へ 1 文字のデータを出力するためにputchar()関数を使用しています（実は、これらは関数ではなく後で学ぶマクロで記述されています）。

また、プログラム中の“EOF”は“End of File”の略で、データの終了を意味します（EOFの値は“stdio.h”の中で定義されています）。なおEOFは、UNIXでは[CTRL]+[D]、Windowsでは[CTRL]+[Z]の打鍵により入力することができます。

関数	戻り値	機 能
getchar()	読み込んだ文字	標準入力から 1 文字読み込む
putchar(c)	書き出した文字	標準出力へ文字cを 1 文字書き出す

[表10-1] getchar() 関数と putchar() 関数

getchar() を使うときには注意しなければならない点があります。getchar() は標準入力から文字を 1 文字だけ入力するための関数ですが、キーボードから文字キーを打鍵しただけでは、実際の入力動作は行われないという点です。[ENTER]キーが押されて初めてgetchar() の実行が開始されます。次のプログラムを実行してみましょう。

----- 例(10)-2 -----

```
#include <stdio.h>

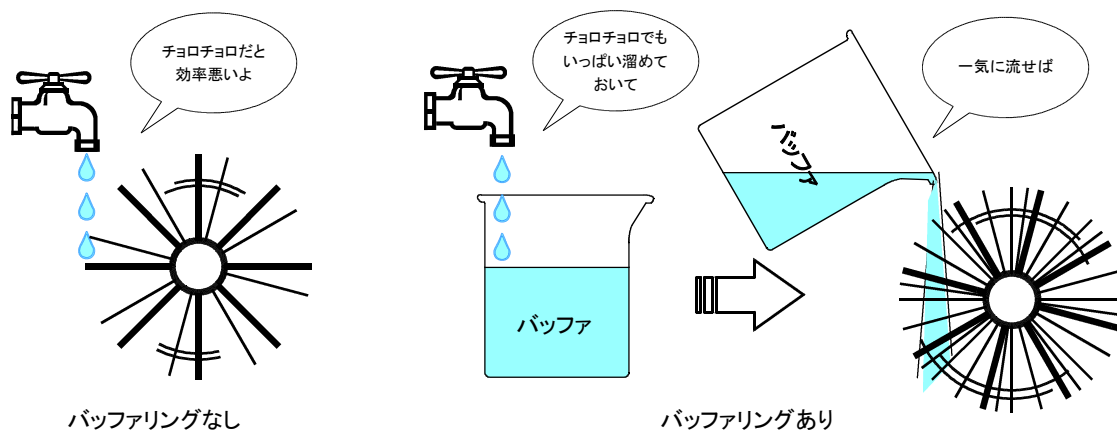
main()
{
    int c[4];

    c[0] = getchar(); printf("%02X\n", c[0]);
    c[1] = getchar(); printf("%02X\n", c[1]);
    c[2] = getchar(); printf("%02X\n", c[2]);
    c[3] = getchar(); printf("%02X\n", c[3]);

    return(0);
}
```

今、[a][b]と続けて2つの文字キーを打鍵したとしましょう。入力したのにこの段階では、実行は最初の `c[0] = getchar()` のところで入力待ちとなってしまいます。本来なら、`c[0]`に'a'、`c[1]`に'b'が格納され、それぞれの文字コードが16進数で表示されてしかるべきです。それでは、ここで[ENTER]キーを押してみましょう。すると、その瞬間、入力動作が開始され、いきなり `c[2] = getchar()` まで実行が進んでしまいます。`c[0]`に'a'、`c[1]`に'b'、`c[2]`に'\n'が格納され、文字コードが16進数で 61, 62, 0A と表示された後、`c[3] = getchar()` の入力待ちで停止します。

どうして、このような一見普通感覚とは異なる動作をするのでしょうか？一般に、入出力装置の速度はCPUの実行速度に比べると極端に遅く、入出力が発生するたびに一々入出力装置に直接制御を移しては、大変効率の悪いことになってしまいます。入出力の効率を高めるためには、バッファリングという処理を行います。バッファリングとは、入出力が発生するたびに入出力装置とデータのやりとりをするのではなく、バッファという、一時的にデータを溜めておくためのメモリを介して入出力処理を行う方法です。入力の場合は、入力装置からのデータをバッファに溜めておき、一定量たまったところで一気にプログラムに読み込みます。逆に出力の場合は、プログラムから出力されたデータをバッファに溜めておき、一定量溜まったところで一気に出力装置に送り出します。こうすることにより、入出力装置と直接やりとりする回数を減らすことができ、CPUとの極端な速度差による効率低下を最小限にとどめることができます。なお、このようなバッファリングによる入出力をストリームといいます。標準入力、標準出力、および標準エラー出力はすべてストリームです。



[バッファリングのイメージ]

つまり、`getchar()` では、[ENTER]キーが押されるまで入力された文字がこのバッファに溜められていき、[ENTER]キーが押されると一気にバッファ内の文字がプログラムによって読み込まれるのです。

ちなみに、“hamakou.h”の中には、このバッファリングを無効にしたり有効にしたりする`keybuff()`関数を用意してあります。次のプログラムを実行してみてください。このプログラムでは入力がバッファリングされず、すぐに読み込まれているはずですが、

----- 例(10)-3 -----

```
#include <stdio.h>
#include <hamakou.h>

main()
{
    int c[4];

    keybuff("off"); // バッファリングを無効にする
    c[0] = getchar(); printf("%02X\n", c[0]);
    c[1] = getchar(); printf("%02X\n", c[1]);
    c[2] = getchar(); printf("%02X\n", c[2]);
    c[3] = getchar(); printf("%02X\n", c[3]);
    keybuff("on"); // バッファリングを有効にする

    return(0);
}
```

ところで、`getchar()`が返す値は文字ですから、それを格納する変数も`char`型であるはずですが、ところが、それが`int`型になっていることに気づいたでしょうか？

実は、`getchar()`は文字を`char`型ではなく`int`型の値として返すのです。ではなぜ、文字を`char`型ではなく`int`型で返すのでしょうか。それは、データである文字とデータの終わりを表す`EOF`とを区別しなければならないからです。もし、`getchar()`が`char`型の値を返したとしたら`EOF`も1バイトで表現されることになり、その値は必ず何らかの文字コードと重複してしまいます。これでは文字なのかデータの終わりなのか区別がつかなくなってしまうですね。また、`putchar()`では、`getchar()`で入力した文字をそのまま出力できるように、文字を`int`型で与えるようになっているわけです。

ポイント

☆`getchar()`関数は_____する関数である。

☆`putchar()`関数は_____する関数である。

☆`getchar()`関数の戻り値は_____型なので、`getchar()`関数で入力した文字を格納する変数も_____型で宣言する。また、`putchar()`関数の引数が_____型なのは、_____のためである。

☆`putchar()`関数の戻り値が_____型になっているのは_____のためである。

② 数字や文字綴りを変換して入力する：`scanf()`関数

指定した形式（これを書式といいます）でデータを入力する関数に`scanf()`関数があります。`scanf()`で指定する書式は、`printf()`関数で指定した書式制御文字と同じものを用いて記述します。また、複数のデータを同時に入力するとき、入力するデータとデータの間に区切り文字を指定することもできます。

scanf()関数

scanf(入力書式文字列, 入力データを格納する変数の並び);

入力書式文字列 : 入力データの型とデータ間の区切り文字を指定する書式
(書式制御文字の並び)

入力データを格納する変数の並び : 入力されたデータを格納する変数の“,”区切り並び
(ポインタを学習するまでは、単純変数には「&」を付け、
文字配列には付けないと考えていてください)

動作 標準入力からの入力を入力書式に従って変換し、これに続く変数に代入します。
入力書式文字列は、入力として許される文字列とそれらの代入時の変換方法とを
指定する書式です。

(書式の詳細は<http://www.bohyoh.com/CandCPP/C/Library/fscanf.html>を参照)

書式に見合うだけの実引数がないときの動作は定義されません。実引数が残って
いるにもかかわらず書式が尽きてしまう場合、余分の実引数は評価するだけで無
視されます。

次のプログラムを実行してみてください。

例(10)-4

```
#include <stdio.h>

main()
{
    int    year, month, day;
    float  x, y;

    printf("今日の年月日をスペースで区切って入力してください:");
    scanf("%d %d %d", &year, &month, &day); // 整数をスペースで区切って入力
    printf("%d年%d月%d日\n", year, month, day);

    printf("今日の年月日を yyyy-mm-dd のように入力してください:");
    scanf("%d-%d-%d", &year, &month, &day); // 整数を“-”で区切って入力
    printf("%d年%d月%d日\n", year, month, day);

    printf("実数x, yをスペースで区切って入力してください:");
    scanf("%g %g", &x, &y); // 実数をスペースで区切って入力
    printf("%g + %g = %g\n", x, y, x + y);

    printf("実数x, yを x, y のように入力してください:");
    scanf("%g,%g", &x, &y); // 実数を“,”で区切って入力
    printf("%g + %g = %g\n", x, y, x + y);

    return(0);
}
```

実行例

```
$ ./a.out
今日の年月日をスペースで区切って入力してください：2004 5 8
2004年5月8日
今日の年月日を yyyy-mm-dd のように入力してください：2004-5-8
2004年5月8日
実数x, yをスペースで区切って入力してください：3.5 6.8
3.5 + 6.8 = 10.3
実数x, yを x, y のように入力してください：3.5, 6.8
3.5 + 6.8 = 10.3
$
```

入力書式の使い方が理解できたでしょうか。ところで、ここで登場する「&year」という表記は、変数yearが使用しているメモリ上のアドレスを示しています（詳細はポインタの章で説明します）。このように、scanf()関数で1データを1変数に入力する場合は、入力データの格納先となる変数の先頭に「&」を付けます。しかし、文字列を文字配列に格納する場合は「&」を付けません。

次の例(10)-5を見てください。文字変数aには1文字を入力しますので「&」が付けられていますが、文字列を入力する文字配列bに「&」は付けられていないことに注意してください。

----- 例(10)-5 -----

```
#include <stdio.h>

main()
{
    char a;        // 文字入力用変数
    char b[10];    // 文字列入力用変数

    printf("文字を入力してください。=> ");
    scanf("%c",&a); // 1文字を入力（1データを1変数に格納）

    printf("文字列を入力してください。=> ");
    scanf("%s",b); // 文字列を文字配列に格納

    printf("入力文字   = %c\n",a);
    printf("入力文字列 = %s\n",b);

    return(0);
}
```

ポイント

☆scanf関数は_____する関数である。

問題(10)-1

時刻を"20:35"というような24時間表記の文字列で表すことにします。

ある開始時刻と終了時刻を"開始時刻 終了時刻"の順で入力し、その経過時間を分と秒で表すプログラムをscanf()関数を用いて作りなさい。ただし、経過時間は24時間を超えないものとします。(toi10-01.c)

(実行例)

時刻 = 10:35 19:23

10:35から19:23までは、528分または31680秒経過しています。

問題(10)-2

n個の整数をキーボードから配列に入力し、それをそのまま表示するプログラムを作りなさい。但し、入力にはscanf()関数を用いるものとします。(toi10-02.c)

③ scanf()関数の問題点

次のプログラム(例(10)-6)は、「1つの整数を入力し、それをそのまま表示する」ということを繰り返すだけの単純なものです。0を入力するとプログラムは終了します。プログラムを実行してみてください。

---- 例(10)-6 -----

```
1: #include <stdio.h>
2:
3: main()
4: {
5:     int dec = 0; // 数値入力用変数
6:
7:     do {
8:         printf("整数を入力してください。\\n");
9:         scanf("%d", &dec); // 文字列を10進整数として入力
10:        printf("dec = %d\\n", dec); // 入力した10進整数を表示
11:    } while (dec != 0); // 0を入力したら終了
12:
13:    return(0);
14: }
```

このプログラムは、正しく整数値（を表す文字列）を入力した場合は、もちろん正常に動作します。それでは、次の実験をしてみてください。

実験

例(10)-6 を再度実行し、わざと整数以外の文字や記号を入力して、どんな結果になるか確認してみなさい。

(プログラムを停止する時は CTRL + C)

間違えて「X+Y」などと文字や記号を入力してしまうと、表示がおかしくなったり、永久ループのような現象になったりと、とにかくおかしい動作をしてしまいます。また、整数入力であるのに「12.34」などのような小数点混じりの数値を入力した場合は、整数部分のみが取り込まれます。小数点以下は入力バッファに溜まったままとなり、取り出されずに残ってしまうのです（残ったものは、次の入力時に参照されるのですが、整数ではないので永久に取り込まれることはありません）。

さらに、scanf()関数には「行」という概念がありません。改行(' \n')も、タブ(' \t')や半角スペース(' ')と同じ扱いであり、入力データの区切り記号として解釈されてしまいます。したがって、これらを区別をしたくてもそれができないという欠点もあります。

このように、scanf()は便利なようで非常に使いにくく、職業プログラマはめったに使用しません。

ポイント

☆scanf関数の問題点

- (a) _____

- (b) _____

- (c) _____

④ scanf()関数を使用しない入力

これまで説明したように、scanf()関数を用いた入力を行うと思ってもよらない結果になってしまうことがあります。

そこでCプログラマの間では、「scanf()関数の代わりにfgets()関数とsscanf()関数を組み合わせて使う」というのが定石になっています。

fgets()関数

fgets(文字配列, 文字配列のサイズ, 入力先);

文字配列：入力した文字列を格納するための文字配列
文字配列のサイズ：文字配列の要素数（int型でバイト数を指定する）
入力先：文字の入力先（入力ストリーム）

動作 第3引数で指定した入力先から文字列を読み取り、第1引数で指定した配列に格納します。読み取る文字数の最大は[文字配列のサイズ-1]文字になります。改行文字を読み取ったとき、またはファイルの終わりを検出したときに、文字の読み取りは終了し、読み取った改行文字も配列に格納します。最後に配列に格納した文字の直後に' \0'を追加します。

sscanf() 関数

sscanf(文字配列, 入力書式文字列, 変換データを格納する変数の並び);

文字配列 : これに続く書式に従って変換される文字列を格納する配列

入力書式文字列 : 変換対象データの型とデータ間の区切り文字を指定する書式
(書式制御文字の並び)

変換データを格納する変数の並び : 変換されたデータを格納する変数の“, ”区切り並び
(ポインタを学習するまでは、単純変数には「&」を付け、
文字配列には付けないと考えてください)

動作 標準入力ではなく、第1引数で指定した文字列から読み込む点を除いて、
scanf関数と等価です。

fgets() 関数と sscanf() 関数を組み合わせにより、

```
scanf("%d %d", &x, &y);
```

という文は、次のように書き直すことができます。

```
char buf[256];  
fgets(buf, sizeof(buf), stdin);  
sscanf(buf, "%d %d", &x, &y);
```

この中の「stdin」は、「標準入力」から読み込むことを意味します。(stdinを標準入力ストリームといいます。また、標準出力ストリームはstdout、標準エラー出力ストリームはstderrで表します。)

例(10)-6 も同様にfgets()関数とsscanf()関数を使って次のように書き直すことができます。このプログラムでは、誤って整数以外の文字列を入力しても例(10)-6のときようなおかしい動作にはなりません。実行して確かめてみてください。

例(10)-7

```
1: #include <stdio.h>  
2:  
3: main()  
4: {  
5:     int dec;          // 数値入力用変数  
6:     char buf[256];    // 一時入力用変数  
7:  
8:     do {  
9:         printf("整数を入力してください。\\n");  
10:        fgets(buf, sizeof(buf), stdin); // キーボードより1行入力  
11:        sscanf(buf, "%d", &dec);        // bufの文字列を10進整数に変換  
12:        printf("dec = %d\\n", dec);      // 入力した10進整数を表示  
13:    } while (dec != 0);                  // 0を入力したら終了  
14:  
15:    return(0);  
16: }
```


ポイント

☆キーボードからのデータ入力はscanf()関数より _____
_____を使う方が良い。

⑤ getint()関数

このgetint()関数は情報基礎やプログラミング技術の授業でも利用してきましたが、実はこれもfgets()関数とsscanf()関数を組み合わせて作られています。

例(10)-7の9行目のように、入力時に入力を促すメッセージを表示したいとき、わざわざprintf()関数などを使って別途メッセージを表示してやる必要がありました。これを、BASICのINPUT命令やpythonのinput()関数のように、入力を促すメッセージ表示とともに整数を入力できるようにしたのが、getint()関数なのです。

----- getint()関数のソースコード (抜粋) -----

```

        :
/*-----
getint -- キーボードから整数を入力する。

パラメータ
    msg -- 入力時に表示するメッセージ
戻り値
    value -- 入力した整数
            数値以外を入力した場合は、再度入力を要求する。
            浮動小数点数を入力した場合は、小数点以下を
            切り捨てて整数化する。
-----*/
int getint(char *msg)
{
    char line[100];
    int value;

    while(1) {
        (void)printf("%s", msg);
        (void)fgets(line, sizeof(line), stdin);
        if(sscanf(line, "%d", &value) == 1) // 正しく整数に変換できたかチェック
            break;                          // 正しく変換できたときはループを抜ける
        line[strlen(line) - 1] = '\0';      // 正しく変換できなければ再度入力を促す
        (void)fprintf(stderr, "%s` is not numerical value.\n", line);
    }
    return(value);
}
```

ポイント

☆getint()関数は_____した関数である。

問題(10)-3

次の設問について Yes/No で答え、Noのものについてはどうして間違っているのか説明しなさい。

- ① 「printf("\n");」と「putchar('\n');」は同じ動作をする。
- ② scanf()関数はどんな場合でも正常にデータを入力できる。
- ③ ストリーム入出力関数は、感覚と異なる動作をするので使用しない方がよい。

問題(10)-4

例(10)-4のプログラムを、fgets()とsscanf()を用いて書き直しなさい。

(toi10-04.c)

問題(10)-5

次のプログラム中の誤りを指摘し、意図通りの動作をするように修正しなさい。

```
----- toi10-05.c -----
{
// [CTRL+D]キーが押されるまでに入力した文字の行数を数える

#include <stdio.h>

main()
{
    char c;
    long lines = 0L;

    while((c == getchar() != EOF) {
        if (c == "\n")
            lines++;
    }
    printf("Lines = %d\n", lines);

    return(0);
}
```

(11) わかりやすいプログラムのスタイル（インデント、マクロ機能）

① 各行の先頭を字下げする

C言語プログラムの書式は、フリーフォーマットといい、カッコ（ ），{ }の対応や、関数名・変数名があらかじめ定められた規則にのっとって正しく書かれていれば、空白のあるなしや改行の位置に関係なく自由にプログラムを記述することができます。

ですから、次の2つのプログラムは、あらかじめ定められた規則にのっとって記述されているため、いずれも問題なくコンパイル，実行ができます。

----- 例(11)-1 -----

```
#include <stdio.h>
main() {printf("Hello world.\n");return(0);}
```

----- 例(11)-2 -----

```
#include <stdio.h>
main()
{
    int i,n;
    n = 0;
    for (i = 1; i <= 10; i++) {
        n += i;
    }
    printf("10までの自然数の和は %d\n",n);
    return(0);
}
```

しかし、このように書かれたプログラムは、プログラムの論理構造が一見ただけでは分かり難く、決して人に優しい（分かりやすく読みやすい）プログラムだとは言えません。人に優しいプログラムとは、次のような点に留意して書かれたプログラムのことをいいます。

- ～ 人に優しいプログラム ～

 - (a) コメントを適宜挿入する
 - (b) 基本的に「1行1命令」を守る
 - (c) 適切なインデントを設ける

(a)は既に説明してありますので、ここでは省略します。

(b)を無視して記述したのが、例(11)-1の例です。非常に分かり難くいと思いませんか？まれに、この「1行1命令」に従わずに記述した方が分かりやすいこともあります。一般的には、このようなことはめったにありません。

(c)は、^{おろそ}かにはされがちですが、上手に利用すればかなり読みやすいプログラムにすることができます。インデント(indent)とは字下げのことで、行頭から何文字分右側にその行の先頭文字をずらすことによって、プログラムの論理的なまとまりを明示することができます。インデントを行う時の文字数をインデ

ト幅といいますが、この幅については特に決められた値というものはありません。一般的には、2, 4, 8といった値が使われることが多いようです（ちなみにこのテキストでは、インデント幅を4としています）。また、空白の他にタブを使用することもよく行われます。通常、emacsのようなプログラミング用のテキストエディタには、適切なインデントを自動的に行ってくれる機能が備わっていますのでこれを積極的に利用するとよいでしょう。

ポイント
☆人に優しいプログラムとは

・ _____
・ _____

☆インデントとインデント幅

② 定数や式を別の文字列に置き換える

次のプログラムは、標準入力から10個の整数を入力し、そのデータを降順に並べ換えるものです。

----- 例(11)-3 -----

```
#include <stdio.h>

main()
{
    int data[10];          // データを格納する配列
    int max_idx;           // 最大値の番号を格納する変数

    char line[100];        // fgets()入力用文字配列
    int i, j, work;

    /*===== データの入力 =====*/
    printf("\n<<10個のデータを入力してください>>\n");
    for (i = 0; i < 10; i++) {
        printf("data[%2d] = ", i);          // 入力要求のメッセージ
        fgets(line, sizeof(line), stdin);   // データを文字で入力
        sscanf(line, "%d", &data[i]);       // データを10進数に変換
    }

    /*===== データの並べ換え =====*/
    for (i = 0; i < 9; i++) {
        max_idx = i;          // 仮の最大値の番号を保存
        for (j = i + 1; j < 10; j++) {
            if (data[max_idx] < data[j]) { // 並換える必要があるか
                max_idx = j;             // 最大値の番号を保存
            }
        }
        if (max_idx != i) {          // max_idx が変化しているか
            work = data[i];          // -+
            data[i] = data[max_idx]; // |- 入れ替え処理
            data[max_idx] = work;     // -+
        }
    }
}
```

```

/*===== 結果の出力 =====*/
printf("\n** 降順に並べ換えた結果 **\n");
for (i = 0; i < 10; i++) {
    printf("%2d番目=%d\n", i + 1, data[i]);    // ソート後のデータ表示
}

return(0);
}

```

このプログラムで、もし入力するデータの個数を変更するとしたら、プログラム内のいくつかの箇所を変更しなくてはなりません（プログラム中の太字部分）。それでも、この程度の大きさのプログラムであれば、数ヶ所の変更をするだけで済みますが、もっと大きなプログラムになると変更ミスや変更漏れなどが起きやすく、そう簡単にはいかないことは容易に想像できると思います。

このようなことを避けるために、C言語にはマクロ機能というものがあります。**マクロとは、プログラム中の文字列をあらかじめ定義した規則に従って、別の文字列に置換する機能のことです。**この置換は、プリプロセッサによりコンパイル前に自動的に行われます。**マクロは #define文で記述します。**このマクロ機能を使い、データの個数とfgets()で使う文字配列の大きさを定義したものが、次の例(11)-4です。

----- 例(11)-4 -----

```

#include <stdio.h>

#define DATA_SIZE 10    // データの個数の定義
#define BUFF_SIZE 100   // fgets()で使う文字配列サイズの定義

main()
{
    int data[DATA_SIZE]; // データを格納する配列
    int max_idx;         // 最大値の番号を格納する変数

    char line[BUFF_SIZE]; // fgets()で使う文字配列
    int i, j, work;

    /*===== データの入力 =====*/
    printf("\n<<%d個のデータを入力してください>>\n", DATA_SIZE);
    for (i = 0; i < DATA_SIZE; i++) {
        printf("data[%2d] = ", i);          // 入力要求のメッセージ
        fgets(line, sizeof(line), stdin);   // データを文字で入力
        sscanf(line, "%d", &data[i]);       // データを10進数に変換
    }

    /*===== データの並べ換え =====*/
    for (i = 0; i < DATA_SIZE - 1; i++) {
        max_idx = i;                        // 仮の最大値の番号を保存
        for (j = i + 1; j < DATA_SIZE; j++) {
            if (data[max_idx] < data[j]) { // 並べ換える必要があるか
                max_idx = j;              // 最大値の番号を保存
            }
        }
        if (max_idx != i) {                // max_idx が変化しているか
            work = data[i];                // +-
            data[i] = data[max_idx];       // |- 入れ替え処理
            data[max_idx] = work;          // +-
        }
    }
}

```

```

/*===== 結果の出力 =====*/
printf("\n** 降順に並べ換えた結果 **\n");
for (i = 0; i < DATA_SIZE; i++) {
    printf("%2d番目=%d\n", i + 1, data[i]);    // ソート後のデータ表示
}

return(0);
}

```

#define文の一般形式

```
#define 置換前文字列 置換後文字列
```

この#defineマクロを使うことによって、プログラムの修正・変更を手作業でおこなう場合の手間と、変更忘れなどのミスの可能性を小さくすることができます。また、例(11)-4のBUFF_SIZEのような定数などの置き換えに用いることで、その値の意味を連想しやすくする効果もあります。

実は、#define文は単に文字の字面を書き換えているだけなので、次のような文字列の置き換えもできます。

```

----- 例(11)-5 -----
{
    :
#define BUFF_SIZE 100    // バッファサイズ設定
#define END return(0)    // 正常終了
main()
{
    :
    END;    // return(0) に置き換わる
}

```

これは「END」という名前に「return(0)」という文字列が定義してあるので、コンパイル時に「END」が「return(0)」に置き換えられます。

先程のDATA_SIZEの設定でも、数値としての「20」を定義したのではなく、「20」という文字列を定義しておいて、コンパイル時に「DATA_SIZE」という文字列を「20」という文字列に書き換えているだけなのです。

この#define文は大変便利ですが、何でもかんでも別の文字列に置き換えてしまうというのには問題があります。例(11)-5でも、「return(0)」を「END」という文字列に置き換えましたが、この「END」を見て、これが正常終了の意味であるかどうかは、プログラムの先頭の#define文（マクロ定義）を見るまでわかりません。

マクロ機能は、プログラムをわかりやすく簡潔に書くための機能ではありますが、多用しすぎるとかえって理解しにくいプログラムとなってしまうたり、単に字面を書き換えているだけだということを忘れてしまうと、問題(11)-3のように思わぬエラーを招く場合もありますので、注意してください。

ポイント

☆ #define文は、プログラム修正・変更時の_____

☆ #define文の一般形式

問題(11)-1

マクロ機能とはどんなものですか。説明しなさい。

問題(11)-2

例(11)-3 と 例(11)-4 を、それぞれ20個のデータを入力できるように変更してみてください。また、それを踏まえて、マクロを利用する利点を説明しなさい。

(toi11-02-01.c, toi11-02-02.c)

問題(11)-3

次の各設問について答えなさい。(プログラムにはコメントを付加すること)

- ① 次のプログラムでは、144 を出力するつもりだったのが 47 を出力してしまいます。それはなぜですか？ また、意図通り 144 を出力できるように修正しなさい。

```
---- toi11-03-01.c -----
#include <stdio.h>
#define PART1 7
#define PART2 5
#define PART3 PART1 + PART2

main()
{
    printf("%d * %d = %d\n", PART3, PART3, PART3 * PART3);
    return(0);
}
```

- ② 次のプログラムでは、8 を出力するつもりであったのが 10 を出力してしまいます。それはなぜですか？ また、意図通り 8 を出力できるように修正しなさい。

```
---- toi11-03-02.c -----
#include <stdio.h>
#define SIZE 10;
#define FUDGE SIZE - 2;

main()
{
    int size; // 実際に用いるサイズ

    size = FUDGE;
    printf("サイズは %d\n", size);

    return(0);
}
```

(12) プログラムの部品を作る (関数の利用)

例(11)-3, 例(11)-4 では、データの入力部分に項目(10)で説明したfgets()関数とsscanf()関数を使用しています。

この2つの関数の組み合わせによる入力を、BASICのINPUT文のような形で使えるようにすれば、この部分の「入力する」という役割がはっきりして、さらにわかりやすくなります。fgets()とsscanf()の組み合わせによるデータ入力を、input()という別の関数に置き換えてみたのが次の例(12)-1です。

----- 例(12)-1 -----

```
#include <stdio.h>

#define DATA_SIZE 20 // データの個数の設定

int input( void ); // 関数input のプロトタイプ宣言

main()
{
    int data[DATA_SIZE]; // データを格納する配列
    int max_idx;         // 最大値の番号を格納する変数

    int i, j, work;

    /*===== データの入力 =====*/
    printf("\n<<%d個のデータを入力してください>>\n", DATA_SIZE);
    for (i = 0; i < DATA_SIZE; i++) {
        printf("data[%2d] = ", i);
        data[i] = input();
    }

    /*===== データの並べ換え =====*/
    for (i = 0; i < DATA_SIZE - 1; i++) {
        max_idx = i; // 仮の最大値の番号を保存
        for (j = i + 1; j < DATA_SIZE; j++) {
            if (data[max_idx] < data[j]) // 並べ換える必要があるか
                max_idx = j; // 最大値の番号を保存
        }
        if (max_idx != i) { // max_idx が変化しているか
            work = data[i]; // +-
            data[i] = data[max_idx]; // |- 入れ替え処理
            data[max_idx] = work; // +-
        }
    }

    /*===== 結果の出力 =====*/
    printf("\n** 降順に並べ換えた結果 **\n");
    for (i = 0; i < DATA_SIZE; i++) {
        printf("%2d番目=%d\n", i + 1, data[i]); // ソート後のデータ表示
    }

    return(0);
}
```



```

/*-----+
|  整数値の入力          |
|  関数名：input         |
|  引数   ：無し        |
|  戻り値：入力した整数 (int型) |
|-----+
#define BUFF_SIZE  100  // バッファサイズ設定

int input( void )
{
    char line[BUFF_SIZE]; // データ入力用ラインバッファ
    int dat;               // 入力された数値を格納する変数

    fgets(line, sizeof(line), stdin); // データ入力
    sscanf(line, "%d", &dat);         // 数値に変換

    return(dat);
}

```

ここで、関数input()は戻り値としてint型の値を返すため、関数自身の型をint型で宣言しています。

関数は、それを使用するに当たり、どのような引数を取りどのような値を返すのかを、事前に宣言をしておかなければなりません。このための宣言文を「関数プロトタイプ宣言」といいます。例(12)-1 では、main()関数の直前にある、

```
int input( void );
```

の部分がこれにあたります。なお、行の最後に";"が必要なことに注意して下さい。

この例ように、関数を利用するプログラムのソースファイルは、一般に次のようなスタイルで記述します。

```

#include < ... >
#include < ... >
    :
戻り値の型 関数1(引数宣言1, 引数宣言2, ... ); // 関数1のプロトタイプ宣言
戻り値の型 関数2(引数宣言1, 引数宣言2, ... ); // 関数2のプロトタイプ宣言

main()
{
    :
}

戻り値の型 関数1(引数宣言1, 引数宣言2, ... )
{
    :
    return(...);
}

戻り値の型 関数名2(引数宣言1, 引数宣言2, ... )
{
    :
    return(...);
}
    :

```

関数プロトタイプ宣言は、関数定義の先頭部分と同じであるため、一見必要ないように感じます。しかし、これがあることにより、関数の戻り値の型、関数に渡される引数の型およびその個数を、コンパイラに対して明確に伝えることができます。その結果、関数呼び出しにおける引数の記述誤りや、戻り値とそれを代入する変数との型の不一致などの誤りが、コンパイル時にチェックできるようになるのです。

また、関数の戻り値は、return文で指定します。例(12)-1で使用したinput()関数では、return(dat);という文により変数datを戻り値として返しています。datの値は、標準入力より入力した文字列を10進整数に変換したものです。さらに、input()関数の戻り値はint型ですから、当然この変数datもint型になっています。

なお、引数部分に記述されているvoidは、この関数が引数をとらないことを明示的に示すために用いられるものです。また、関数が何も値を返さない（戻り値がない）ということを示すためにもvoidを使います。このような関数のreturn文には、当然返す値を何も指定しません。

次の例(12)-2の関数yoko(), tate()は、引数も戻り値もない関数の例です。

----- 例(12)-2 -----

```
#include <stdio.h>

void yoko( void );    // 横線を表示する関数
void tate( void );    // 縦線を表示する関数

main()
{
    int i; // カウンタ変数の宣言

    yoko(); // 横線の表示

    for (i = 1; i <= 4; ++i) { // 4回繰り返し
        tate();                // 縦線の表示
    }
    yoko(); // 横線の表示

    return(0);
}

void yoko( void )
{
    printf("-----+\n");
    return;
}

void tate( void )
{
    printf("|          |\n");
    return;
}
```

また、関数はその内部でのみ有効な変数を持つことができます。次の例を見てみましょう。

----- 例(12)-3 -----

```
#include <stdio.h>

void ast7( void );

main()
{
    int i; // カウンタ変数の宣言

    for ( i = 1; i <= 5; i++) { // 繰返し処理
        ast7();                // 関数ast7()の呼出し
    }
    return(0);
}

void ast7( void )
{
    int i; // ループ変数の宣言

    for ( i = 1; i <= 7; i++) { // 繰返し処理
        putchar('*');          // 「*」の表示
    }
    putchar('\n');             // 改行

    return;
}
```

これは、画面上に「*」を7文字×5行の四角形に並べるプログラムです。ここでは、まず「*」を7文字表示して改行する ast7() という関数を作り、それを5回呼出しています。

このプログラムでは、ast7() 関数の中で「i」がループ変数として使われ、それを呼び出す側のmain() 関数の中でも「i」がループ変数として使用されています。どちらも同じ「i」という名前の変数ですが、互いに影響を受けない、まったく独立した別の変数として扱われます。このように、ある関数の中でだけ有効であり、その外部にある変数とは独立している変数をローカル変数、または局所変数と呼びます。

C言語をはじめとする近代的な構造化プログラミング言語では、このようなローカル変数を用いることにより、関数の中でどんな名前の変数が使われているかといったことを、それを呼び出す側では一切気にする必要はなくなります。つまり、関数は、その「働き」と「どんな引数を取りどんな値を返すのか」(関数の外部仕様)さえ知っていれば、利用できるのです。ast7()の例でいえば、「「*」を7個表示する」という「働き」と「引数も戻り値も必要ない」ということさえ知っていればいいわけです。

今度は、引数を取る関数を利用する例を考えてみましょう。

例(12)-4は、3つの三角形の面積を求めるプログラムですが、「面積を求める」部分はtriangle()という関数で実現しており、これに底辺の長さと高さを与えることで面積を求めます。

----- 例(12)-4 -----

```
#include <stdio.h>

float triangle( float width, float height );    // 三角形の面積を求める関数

main()
{
    printf("三角形 1 : %.2fcm2\n", triangle(1.3, 8.3));
    printf("三角形 2 : %.2fcm2\n", triangle(4.8, 9.8));
    printf("三角形 3 : %.2fcm2\n", triangle(1.2, 2.0));

    return(0);
}

float triangle( float width, float height )
{
    float area; // 三角形の面積

    area = width * height / 2.0;    // (底辺 × 高さ) ÷ 2

    return(area);
}
```

関数triangle()の定義部分で、関数名triangleに続く()内にある“float width”と“float height”という記述に注目して下さい。これは、外部からtriangle()関数に渡されてきた値を、関数内部に取り込む役割をする変数を宣言しているのです。このような役割をする変数を仮引数といい、この仮引数もその関数内部でのみ有効なローカル変数になります。

これに対し、関数を呼び出す際に、関数へ渡す値を実引数といいます。例(12)-4でいえば、main()関数内で“triangle(1.3, 8.3)”という関数呼び出しがありますが、このときの“1.3”や“8.3”が実引数になります。

なお、プロトタイプ宣言においては、仮引数の型を示しさえすればよいので、仮引数に用いる変数そのものは省略しても構いません。つまり、次のように記述できます。

```
float triangle( float, float );
```

↑
2つの float型の引数

しかし、各引数の役割を明確にする意味では、引数にはその役割を表す適切な変数名を使用し、プロトタイプ宣言でもできるだけ省略しない方がよいでしょう。

~~~~~ 関数化するメリット ~~~~~

- ・ よく使う 1まとまりのコードを、簡単に何度でも使える。
- ・ 関数はそれを利用する側からは、その「働き」と「どんな引数を取りどんな値を返すのか」さえ知っていれば、利用できる。この結果、見通しの良い分かりやすいプログラムが記述できる。

### ~~~~~ コラム 3 ~~~~~

#### ◆◆◆ 引数のあるマクロ ◆◆◆

マクロには関数のように引数を与えることもできます。例えば、次のマクロは与えられた値の2乗を求めるものです。

```
#define SQR(x) ((x) * (x)) // 値を2乗する
```

このマクロは、SQR(5) のように書くと ((5) \* (5)) と展開されます。式全体や変数を( )で囲んであるのは、引数として複雑な式が与えられたとき、演算順序が変わってしまうようなことを避けるためです。**式全体が( )で囲まれていることにも注意**して下さい。

もし( )で囲まず、

```
#define SQR(x) x * x // 値を2乗する
```

とした場合、例えば、8 / SQR(4 - 2) は単純に 8 / 4 - 2 \* 4 - 2 と展開されてしまいます。その結果、本来 8 / (2 \* 2) = 2 となるべきところが、2 - 8 - 2 = -8 になり意図したものとは異なった結果になってしまいます（マクロは、あくまでも字面を書き換えるだけだということを忘れないでください）。( )で囲ってあれば、8 / ((4 - 2) \* (4 - 2)) と展開され、意図通りの結果を得ることができます（式全体を( )で囲む必要がある理由も分かりますね）。

#### 問題(12)-1

プログラムを処理単位ごとに関数化する理由を説明しなさい。

#### 問題(12)-2

$n$  個の '\*' を横一行に表示（表示後は改行）する関数 void ast(int n) を用意し、その動作を確認するプログラムを作りなさい（文字数  $n$  は標準入力より入力する）。

(toi12-02.c)

#### 問題(12)-3

問題(12)-2の ast() を利用して、例(12)-3のように '\*' を  $n$  文字  $m$  行の四角形となるように並べるプログラムを作りなさい。

なお、文字数  $n$  と行数  $m$  は標準入力より入力し、 $n$  と  $m$  は空白文字で区切るものとします。

(toi12-03.c)

#### 問題(12)-4

二次方程式  $ax^2 + bx + c = 0$  の係数を引数とし、判別式  $D = b^2 - 4ac$  を戻り値とする関数を用意し、これが正常に動くことを確認するプログラムを作りなさい。

なお、確認プログラムの雛形は toi12-04.c として用意してありますので、これを利用しなさい。ここでは、係数  $a, b, c$  は標準入力より入力し、 $a, b, c$  はそれぞれ空白文字で区切ります。

(toi12-04.c)

問題(12)-5

- ① 正数  $n_0$  が与えられたとき、 $i = 0, 1, 2, \dots$  に対して次の数列を定義します。

$$n_{i+1} = \begin{cases} \frac{n_i}{2} & n_i \text{が偶数のとき} \\ 3n_i + 1 & n_i \text{が奇数のとき} \end{cases}$$

なお、この数列は  $n_i$  の値が 1 になると終わります。このようにして生成される数は、「<sup>あられ</sup>霰」と呼ばれます。霰を生成するプログラムを作りなさい。このプログラムの main() 関数には、toi12-05-01.c を用意してあるので、これをコピーして利用すること。

但し、関数

```
int hailstones(int n);
```

が、正数  $n$  (上記定義の  $n_0$ ) より生成される数列を表示し、生成した霰の数を戻り値として返すものとします。また、 $n$  は標準入力より入力し、表示は次のようにしなさい。

n を入力してください: 77

77 から生成される「霰」

```
77 232 116 58 29 88 44 22 11 34 17 52
26 13 40 20 10 5 16 8 4 2 1
```

合計 23 個

(toi12-05-01.c)

- ② 標準入力より西暦年 (4 桁の整数) を入力し、その年がうるう年か平年かを判定する関数 isLeapYear() を用意し、これが正常に動くことを確認できるプログラムを作りなさい。isLeapYear() は、うるう年の時 1, 平年の時は 0 を返すものとします。うるう年か平年かは次の条件により判断です。

1. 西暦が400で割り切れる年はうるう年である。
2. 400で割り切れない場合、西暦が100で割り切れる年はうるう年ではない。
3. 100で割り切れない場合、西暦が4で割り切れる年はうるう年である。
4. 4で割り切れない場合、うるう年ではない。

なお、このプログラムの main() 関数には toi12-05-02.c を用意してあるので、これをコピーして利用すること。

(toi12-05-02.c)

### (13) 変数の有効範囲 (変数のスコープ)

項目(12)「関数の利用」の中で、特定の関数内でのみ使用できるローカル変数について学びました。変数にはこの他に、プログラム内のどこからでも参照できるグローバル変数(global variable)というものもあります。ここでは、このような変数の有効範囲について学習します。なお、この変数の有効範囲のことをスコープ(scope)とといいます。

#### ○変数の種類

変数には、大きく分けて次の2つのものがあります。

- ・ローカル変数 (local variable・局所変数)

任意のブロックの先頭で宣言します。宣言したブロック内でのみ有効で、そのブロックの外側からは参照できません。

ここで、ブロックとは中カッコ“{}”で囲まれた範囲のことをいいます。

- ・グローバル変数 (global variable・大域変数)

関数ブロックの外側で宣言します。宣言された位置以降ならどこでも有効で、それ以降であれば、プログラム内のどこからでも参照できます。

ここで、関数ブロックとは関数定義の最も外側のブロックのことをいいます。

ローカル変数は、これまで関数内部でのみ有効な変数として登場してきましたが、実は、任意のブロックの先頭部分で宣言された変数は、すべてローカル変数になります。また、関数ブロックの外側で宣言された変数は、すべてグローバル変数になります。次のプログラム例(13)-1を見て下さい。

---- 例(13)-1 -----

```
1: #include <stdio.h>
2:
3: int global = 1; // グローバル変数
4:
5: main()
6: {
7:     int local = 2; // ローカル変数
8:
9:     printf("global = %d\n", global); // グローバル変数 global が使用できる
10:    printf("local = %d\n", local);    // ローカル変数 local が使用できる
11:    // ここから新しいブロック
12:    {
13:        int very_local; // ブロック内のローカル変数
14:
15:        very_local = global + local; // 外側のブロックの変数は参照可
16:        printf("very_local = %d\n", very_local); // このブロック内で使用可
17:        global *= 10; // 外側のブロックの変数は参照可
18:        local *= 10;  // 外側のブロックの変数は参照可
19:    }
20:    printf("very_local = %d\n", very_local); // 使用不可 (エラーとなる)
21:    printf("local = %d\n", local);
22:    printf("global = %d\n", global);
23:
24:    return(0);
25: }
```

① 変数 global のスコープ、② 変数 local のスコープ、③ 変数 very\_local のスコープ

このプログラムでは、global、local、very\_local という3の変数を使用していますが、これらはそれぞれスコープが違います。globalのスコープは3行目～25行目まで、localのスコープは7行目～25行目まで、very\_localのスコープは13行目～19行目までです。そのため、20行目のvery\_localはスコープ外からの参照となり、コンパイル時にエラーが報告されます。

なお、ローカル変数には、グローバル変数と同名のものが宣言可能です。例えば、グローバル変数としてcountという名前の変数を宣言すると、通常そのスコープは宣言以降のプログラム全体となります。しかし、ローカル変数に同じcountという名前があると、そのブロック内では、ローカル変数がグローバル変数よりも優先されます。つまり、ローカル変数のcountが宣言されたブロック内では、グローバル変数のcountは隠されてしまうことになるのです。同様に、複数のローカル変数宣言を多重化（ネスティング）して、より外側のローカル変数を隠すこともできます。

次のプログラム例(13)-2を見て下さい。

----- 例(13)-2 -----

```
1: #include <stdio.h>
2:
3: int total;    // 合計を格納する変数
4: int count;    // カウンタ変数（グローバル）の宣言
5:
6: main()
7: {
8:     total = 0;
9:     count = 0; // カウンタ変数（グローバル）の初期化
10:
11:     printf("count(global) = %d\n", count);
12:     {
13:         int count; // カウンタ変数（ローカル）の宣言
14:
15:         count = 1; // カウンタ変数（ローカル）の初期化
16:         printf("count(local) = %d\n", count);
17:         while (count <= 10) {
18:             total += count;
19:             count++; // カウンタ変数（ローカル）の更新
20:         }
21:         printf("count(local) = %d\n", count);
22:     }
23:     count++; // カウンタ変数（グローバル）の更新
24:     printf("count(global) = %d\n", count);
25:     printf("total(global) = %d\n", total);
26:     return(0);
27: }
```

① グローバル変数 count のスコープ  
② ローカル変数 count がこの領域ではグローバル変数 count を隠す

この例で変数countは、4行目のグローバル変数と、13行目のローカル変数の2つが宣言されています。グローバル変数countのスコープは4行目～27行目までですが、



13行目にある同名のローカル変数の宣言により、このブロック内(13行目～22行目)ではこちらが有効となります。

しかし、このよな変数が隠されてしまうようなプログラムは、あまり上手なプログラムとはいえません。なぜならば、ある部分で参照している変数が、ローカルのもなのかグローバルなものなのかが、一見しただけではわかりにくいからです。例えば、上のプログラムでは、21行目と25行目の `count++`; がグローバル変数を更新しているのかローカル変数を更新しているのか、じっくり見極めないとわかりませんね。

したがって、こういったことを避けるために、グローバル変数とローカル変数とは、それぞれ `total_count`, `item_count` などのように、異なる名前を付けるべきなのです。また、グローバル変数はプログラム中でどこからでも参照できてしまうので、便利な反面、意図しないところで値が書き換えられてしまったりするなど、デバッグを困難にする原因にもなります。ですから、グローバル変数は極力使わないようにすることがベストです。どうしても必要な場合は、他の場所で使われないような、特徴的な変数名にするのが賢明です。

ポイント

☆スコープとは\_\_\_\_\_のことである。

☆変数のには\_\_\_\_\_と\_\_\_\_\_の2種類がある。

\_\_\_\_\_ : \_\_\_\_\_  
\_\_\_\_\_  
\_\_\_\_\_ : \_\_\_\_\_  
\_\_\_\_\_

---

### 問題(13)-1

変数のスコープについて説明しなさい。また、局所変数と大域変数のスコープはどうなっていますか？

### 問題(13)-2

次のプログラムの実行結果を示し、なぜそうなるか説明しなさい。

```
----- toi13-02.c -----
#include <stdio.h>

int s = 0;

main()
{
    int a = 1, b = 2, c = 3;

    s = a + b + c;
    printf("%d %d %d %d\n", a, b, c, s);
    {
        int b = 4;
        float c = 5.0;

        printf("%d %d %g\n", a, b, c);
        a = b;
        {
            int c;

            c = b;
            s = a + b + c;
            printf("%d %d %d\n", a, b, c);
        }
        printf("%d %d %g\n", a, b, c);
    }
    printf("%d %d %d %d\n", a, b, c, s);
}
```

---

## (14) 変数の生存期間（記憶クラス）

変数のスコープはその変数が有効な範囲のことでした。これを変数の空間的な寿命とすれば、変数には時間的な寿命（生存期間）というものも存在します。変数の生存期間とは、その変数が宣言されてから、「時間的にどのくらいの間、有効であるか」ということです。これを変数の**記憶クラス**といいます。C言語では、変数の記憶クラスは「恒久的」か「一時的」かのどちらかしありません。これらはそれぞれ次のような性質のものです。

### ・ 恒久的記憶クラス

プログラムの始動時に最初からメモリ上に固定的に確保されていて、プログラムが終了するまで存在します。初期化は実行開始時に一度だけ行われます。

### ・ 一時的記憶クラス

ブロックに入るたびに（スタックという領域に）その都度確保され、毎回初期化が行われます。そのブロックの実行が終了すると自動的に解放されます。

大域変数（グローバル変数）は常に恒久的です。局所変数（ローカル変数）は、恒久的なものも一時的なものも利用できます。ある変数がどの記憶クラスのものになるかは、型宣言の前に付ける宣言修飾子によって決まります。auto修飾子を付けると一時的記憶クラスになり、static修飾子を付けると恒久的記憶クラスになります。

~~~~~ 宣言修飾子を指定した変数宣言の一般形式 ~~~~~

宣言修飾子 データ型 変数名, … ;

実は、宣言修飾子には次の3つがあり、これらには記憶クラスを決定する以外の役割を持っているものもあります。

・ auto（自動変数）

一時的な記憶クラスの変数を宣言するときに用います。auto修飾された変数を、auto変数または自動変数といいます。ブロック内で宣言修飾子を省略すると、auto修飾子があるものと見なされます。このため、通常auto修飾子を付けることはあまりありません。

・ static（静的変数）

恒久的な記憶クラスの変数を宣言するときに用います。局所変数を恒久的記憶クラスとする時は、明示的にstatic修飾子を記述しなければなりません。static修飾された変数をstatic変数または静的変数といいます。

ただし、大域変数をstatic修飾すると、その変数が「現在のソースファイル内で局所的である」という、まったく違った意味になります。つまり、「その大域変数のスコープは、宣言されているソースファイル内になる」ということです。ですか

ら、たとえ大域変数であっても、static修飾されたものは、別のソースファイルからは参照できないことになるわけです。これについては、後で詳しく説明します。

・extern（外部変数）

extern修飾子は、他のソースファイルで宣言されている、同一名の大域変数を参照するために用います。extern修飾された変数をextern変数または外部変数（外部参照変数）といいます。これによって、大規模なプログラムを複数のソースファイルに分けて分割コンパイルする時、分割したソースファイル間でデータを受け渡すことができるようになります。これについても、後で詳しく説明します。

宣言修飾子を省略したとき、宣言された変数が上記3つの変数のどれになるかは、宣言した位置によってコンパイラが自動判別します。ブロック外部での宣言は静的変数、ブロック内部では自動変数とみなします。

ここからは、各宣言修飾子により宣言された変数の使い方について説明します。

① auto変数（自動変数）

関数の引数やローカル変数はauto変数で、その関数が呼ばれると生成され、その関数が終了すると消滅します。

次のような関数があるとします。

```
void proc( int a )
{
    auto int b;    // ローカル変数宣言（autoは通常省略・初期化していないので値は不明）
    printf("a = %d, b = %d\n", a, b);
    b = 10;
    return;
}
```

ここで、この関数が次のような異なる状況で、2回呼び出される場合を考えます。

(a) 連続して2回呼び出す

```
⎧
  ⋮
  proc(1);
  proc(2);
  ⋮
⎫
```

(b) 2回目の呼び出し前に、何らかの処理をおこなう

```
⎧
  ⋮
  proc(1);
  ⋮ ← 何らかの処理
  proc(2);
  ⋮
⎫
```

それぞれの場合について、次の例(14)-1と例(14)-2を使って実行結果を見てみましょう。

(a) 連続して2回呼び出す場合

----- 例(14)-1 -----

```
#include <stdio.h>

void proc( int a ); // 関数プロトタイプ宣言

main()
{
    proc(1); // 1回目の呼び出し
    proc(2); // 2回目の呼び出し

    return(0);
}

void proc( int a )
{
    int b; // ローカル変数宣言 (初期化していないので値は不明)
    printf("a = %d, b = %d\n", a, b);
    b = 10;
    return;
}
```

例(14)-1 の実行結果はどうなったでしょうか。

| | |
|-----------|-----------|
| 実行結果 | |
| a = _____ | b = _____ |
| a = _____ | b = _____ |

この場合、運がよければ2回目の実行時に「b = 10」と表示されます。この運がよければといったのは、次の例を試してみれば、その理由がわかるでしょう。

(b) 2回目の呼び出し前に別の処理を行う場合

----- 例(14)-2 -----

```
#include <stdio.h>

void proc( int a ); // 関数プロトタイプ宣言
void dummy( void ); // 関数プロトタイプ宣言

main()
{
    proc(1); // 1回目の呼び出し
    dummy(); // 何もしないダミー関数
    proc(2); // 2回目の呼び出し

    return(0);
}
```

```

void proc( int a )
{
    ...   (略)   ...
}

void dummy( void )
{
    int x = -50;
    return;
}

```

例(14)-2 の実行結果はどうなったでしょうか？

| 実行結果 | |
|-----------|-------------|
| a = _____ | , b = _____ |
| a = _____ | , b = _____ |

実際には何もしない、ただ変数を宣言し値を代入しているだけのdummy関数を呼び出すことで、1回目に設定した値が別の値に置き換えられてしまうのです。

つまり自動変数は、その関数が呼び出された時のみ有効で、関数を抜け出すと同時に、その変数は消滅します。実は、値そのものはスタックの中から消去される訳ではないのですが、次の関数呼び出し時に必ずそれが復活するとは限らないことが、この例を通して分かります。ですから、auto変数は必ず初期化してから使うようにしなければなりません。

| ポイント |
|---------|
| ☆自動変数とは |
| _____ |
| _____ |
| _____ |

② static変数（静的変数）

自分自身が呼ばれた回数をカウントアップする関数up()と、呼ばれるたびにカウントダウンする関数down()を次のように定義します。

```

int up( void )
{
    int x = 0;
    x++;
    return(x);
}

int down( void )
{
    int y = 0;
    y--;
    return(y);
}

```

これを使ったプログラム、例(14)-3を考えてみましょう。

----- 例(14)-3 -----

```
#include <stdio.h>

int up( void );    // 関数プロトタイプ宣言
int down( void );  // 関数プロトタイプ宣言

main()
{
    int up_dat, down_dat;

    up_dat  = up();
    up_dat  = up();
    down_dat = down();
    down_dat = down();

    printf("up_dat   = %2d回\n", up_dat);
    printf("down_dat = %2d回\n", down_dat);

    return(0);
}
```

まずは、プログラムを実行する前に結果を予測してください。

| | |
|----------|-----------|
| 実行結果（予測） | |
| up_dat | = _____ 回 |
| down_dat | = _____ 回 |

次に、実際にプログラムを実行してみましょう。結果はどうなったでしょうか。

| | |
|----------|-----------|
| 実行結果（実際） | |
| up_dat | = _____ 回 |
| down_dat | = _____ 回 |

実行してみると、期待通りの結果にはなりません。それは、変数x, yはともに自動変数なので、この関数の実行が終了するとその値は消えてしまうからです。

up() 関数およびdown() 関数を正しく動作させるには、変数xをグローバル変数として定義する方法が考えられます。この考えでプログラムに修正を施したのが次の例(14)-4です。

----- 例(14)-4 -----

```
#include <stdio.h>

int up( void );    // 関数プロトタイプ宣言
int down( void );  // 関数プロトタイプ宣言

int x, y; // グローバル変数として宣言

main()
{
    int up_dat, down_dat;

    x = 0; // 変数 x を初期化
    y = 0; // 変数 y を初期化
    ...   (中略)
}

int up( void )
{
    x++;
    return(x);
}

int down( void )
{
    y--;
    return(y);
}
```

| | |
|----------|-----------|
| 実行結果 | |
| up_dat | = _____ 回 |
| down_dat | = _____ 回 |

このプログラムでは、main() 関数の最初でxおよびyの値を0に初期化しています。その後、up() 関数およびdown() 関数が自分自身が呼び出された回数を計数していきます。結果は、もちろん期待した通りになります。

このプログラムでは、関数up(), down() とともに目的の動作を正しく実現しています。しかし、プログラムの作り方としてはあまり勧められるものではありません。なぜならば、グローバル変数x, yは、プログラムのどこでもその値を変更できるからです。意図しない場所で、変数の値を書き換えてしまうと、up() 関数とdown() 関数は正しく動作しなくなります。これらの関数にとって、変数x, yの値は、いわば「命よりも大切な変数」なのです。この変数は、関数とその内部で自分だけが管理したい変数であり、プログラムの他の部分からは触られたくない大切な変数なので、このようにどこからでも自由に変更できる状態では非常に困るのです。

こういうときに、static変数が役に立ちます。static変数を使い、次のように変更してみます(例(14)-5)。

----- 例(14)-5 -----

```
#include <stdio.h>

int up( void );    // 関数プロトタイプ宣言
int down( void );  // 関数プロトタイプ宣言

main()
{
    int up_dat, down_dat;

    up_dat  = up();
    up_dat  = up();
    down_dat = down();
    down_dat = down();

    printf("up_dat  = %d回\n", up_dat);
    printf("down_dat = %d回\n", down_dat);

    return(0);
}

int up( void )
{
    static int x = 0;  // static変数
    x++;
    return(x);
}

int down( void )
{
    static int y = 0;  // static変数
    y--;
    return(y);
}
```

ローカル変数に宣言修飾子staticを付けて定義すると、プログラムの実行が始まってから終了するまで、ずっとその変数は存在し続けます。

ここで、static変数x, yは、宣言と同時に初期化していることに注意してください。これは、一番最初にこの関数が呼び出された時に、一回だけ初期化を行うためです。2回目以降の呼び出しでは、前回の値が保存されています。これをもし、

```
static int x;
x = 0;
```

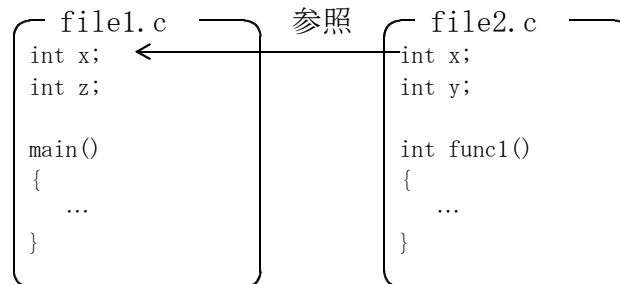
としたら、この関数が呼び出されるたびにxには0が代入されてしまうので、本来の目的通りには動いてくれません（必ず 1 を返す関数になってしまいます）。

ポイント

☆静的変数とは

③ extern変数（外部変数）

今度は、次のように、2つの異なるソースファイル間で同じ変数を参照したい場合を考えます。



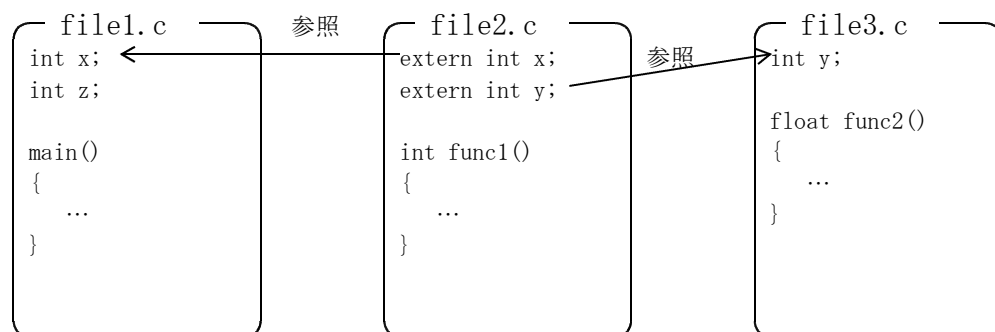
file2.c中の変数xは、file1.cで使用している変数xと同じものを使用したい。

file1.cとfile2.cは、それぞれ別ファイルとして作ります。コンパイルも別々に行い、最後に結合して1つの実行可能プログラムにします（これを分割コンパイルといいます）。このとき、変数xは、実体と同じ共通の変数（すなわち同じアドレス）にならなければなりません。

1つのプログラムを複数のソースファイルによって構成することは、規模の大きなプログラムを作りたいが、1つのソースファイルにするとサイズが大きくなりすぎる、あるいは、複数のプログラマがそれぞれ別々のソースファイルを作り、最終的に1つにしたい、などという場合によくあります。このようなとき、各ソースファイル間で、データを受け渡すための共通の変数というものがどうしても必要になります。

まず、共通変数はグローバル変数でなければなりません。次に、あるソースファイルで宣言されたグローバル変数を他のソースファイルから参照するためには、それを参照する側でちょっとしたおまじないが必要になります。そのおまじないとは、参照したい側のファイルで、その変数をextern修飾子を使って宣言することです。

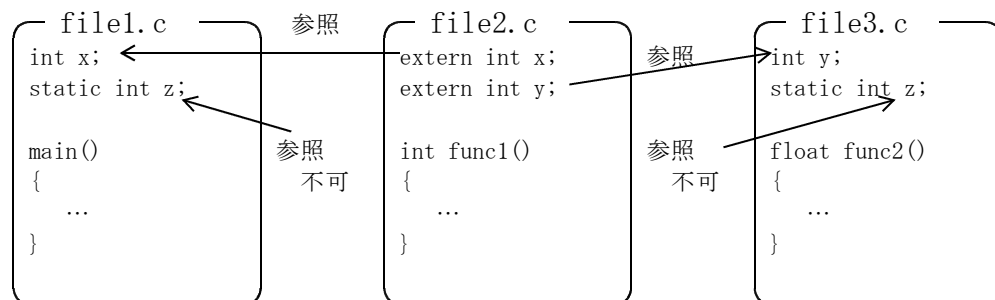
次の図を見てください。



file1.cで使用されている変数xをfile2.cで使用するために、file2.c側でextern

修飾子を使って宣言をしています。同様に、file3.cでグローバル変数として宣言されている変数yをfile2.cから参照するために、file2.cでは変数yもextern修飾子を使って宣言してます。

では、file1.cの変数zを、他のファイルから見えないようにする（参照できないようにする、つまり、変数zの範囲をfile1.c内に限定する）にはどうしたらよいのでしょうか？ このようなことは、各ソースファイルの独立性を高めるために、絶対必要です。これには、グローバル変数をstatic修飾子を使って宣言します。このように、グローバル変数へのstatic修飾は、ブロック内でのstatic修飾とはまったく違う意味になります。混同しないようにくれぐれも注意してください。



file1.cとfile3.c中の変数zは、まったく別の変数で、どちらも外部から参照できない。

なお、グローバル変数は、宣言修飾子を省略した場合、extern修飾子が付けられたものと見なされます。同一名のグローバル変数が複数のソースファイルで宣言されているとき、変数の実体はそれが初めて出現した宣言文によって確保され、それ以降で出現した宣言文では、extern修飾されたものとしてその変数を参照できるようになります。ですから、いちいちextern修飾をしていなくても変数の参照は可能です。しかし、これでは参照関係がわかりにくいプログラムになってしまいますので、明示的にextern修飾を使って宣言するようにしましょう。

ポイント

☆外部変数とは

~~~~ コラム 4 ~~~~~

◆◆◆ 静的変数の初期化 ◆◆◆

静的変数の宣言時に、本テキストでは明示的に「`static int x = 0;`」と初期化していますが、実は、特に初期化されていないときは、暗黙のうちに0で初期化されることになっています。

しかし、0で初期化するという意図を明確にすることは、プログラムの保守性を高めるのに重要ですから、明示的に「`static int x = 0;`」としておくべきなのです。

~~~~ コラム 5 ~~~~~

◆◆◆ 配列変数の記憶クラス ◆◆◆

一時的な記憶クラスの変数（すなわち自動変数）は、その宣言時にスタックと呼ばれる領域に確保されることは、既に説明しました。このとき、確保しようとする変数の数が多すぎると、実行時に「スタックオーバーフロー（スタック領域が足りなくなる）」というエラーが生じることがあります（コンパイルエラーではありません）。

スタックのサイズは、使っているシステムやコンパイラによって違います。UNIXシステムの多くは、実行プログラム自身が、自動的にできるだけ最大のスタックを確保しようとします。その他のシステムでは、コンパイラが適当に決めたスタックサイズが確保され、そのサイズはコンパイラのオプションで変更できるのが一般的です。

いずれにしても、スタックのサイズは、ユーザが利用できる全メモリのサイズよりもずっと小さいのが普通です。ですから、巨大な配列などを一時的な記憶クラスとすると、たちまちスタックは食い尽くされてしまいます。したがって、巨大な配列が必要なときは、できるだけ恒久的な記憶クラスの変数にすることを検討すべきです。

問題(14)-1

記憶クラスとは何ですか。また、どんな種類があり、それぞれどういうものか説明しなさい。

問題(14)-2

次のプログラムを実行すると何が出来されるか予想しなさい。また、実際に実行し、なぜのような結果になったのか説明しなさい。

---- toi14-02-01.c -----

```
#include <stdio.h>
#include <stdlib.h>

#define FOREVER    1
#define STOP      17

void f( void );

main()
{
    while (FOREVER) {
        f();
    }
}

void f( void )
{
    static int cnt = 0;

    printf("cnt = %d\n", ++cnt);
    if (cnt == STOP) {
        exit(0);
    }
}
```

※exit()はプログラムを終了させる関数

--- toi14-02-02.c -----

```
#include <stdio.h>

int z;
void f( int x );

main()
{
    z = 5;
    f(z);
    printf("z = %d\n", z);

    return(0);
}

void f( int x )
{
    x = 2;
    z += x;

    return;
}
```

問題(14)-3

本文中で登場した up() 関数(rei14-05.c)を参考にして、呼び出されるたびに前の呼び出しで返した値の3倍の値を返す関数 int threeTimes(void) を作りなさい。ただし、初めて呼び出されたときに返す値は、1とします。

この threeTimes() 関数を 1~10回呼び出し、その都度返す値を表示するプログラムを作りなさい。(toi14-03.c)

```
int threeTimes( void )
{
    :
}
```

問題(14)-4

n個($n \leq 1000$)の整数データを配列 data[0], data[1], ..., data[n-1] に入力し、その合計と平均を求めるプログラムを作りなさい。

但し、データ数 n と n 個の整数データは標準入力から入力するものとします。なお、配列 `data[]` は大域変数とし、次の (a), (b), (c) の各プログラムを、それぞれ別々のソースファイルで作成すること。また、使用する関数の外部仕様は適切に考えること。

- (a) 合計、平均を表示するプログラム (メインルーチン) $\cdots$ `main()` 関数
- (b) `n`個の値を配列 `data` に入力するプログラム $\cdots$ `input()` 関数
- (c) `n`個の値の合計を求めるプログラム $\cdots$ `sum()` 関数

(toil4-04-main.c, toil4-04-input.c, toil4-04-sum.c)

《分割コンパイルの方法》

各関数を、それぞれ次のようなファイル名のソースファイルに記述した場合、

(a) `... main.c` (b) `... input.c` (c) `... sum.c`

コンパイルの方法は以下のようになります。

(注意): main.c の中で input(), sum() 両関数のプロトタイプ宣言を忘れないこと。

① `cc -c main.c input.c sum.c` 

↑ オブジェクトファイルを作成するオプション

↑ コンパイルしたいファイルの並び

これによって、次の3つのオブジェクトファイルが生成されます。

main.o, input.o, sum.o ... オブジェクトファイル

② cc -o sumavg main.o input.o sum.o

↑ ↑
実行プログラムの名前 オブジェクトファイルの並び

これによって3つのオブジェクトファイルが結合され、実行プログラムsumavgが完成します。

③ プログラムを修正した場合は、修正したファイルだけを

cc -c 修正したファイル.c ↵

でコンパイルし、②の方法で結合します。

参考

①, ②を1度実行する方法

```
cc -o sumavg main.c input.c sum.c
```

但し、この場合は毎回必ずすべてのファイルがコンパイルされます。

(15) 高度なデータ型を使う [1] (構造体)

ここまで、いろいろな種類の変数宣言の方法を学習してきました。しかし、これまで登場した変数は、どれも宣言した型の値を一度に1つしか格納できないものばかりでした（例えば、int型やfloat型の変数）。項目(6)で学習した配列は、複数の値を格納できますが、やはり、各要素にはどれも同じ型の値を1つずつしか格納できません（int a[10]; という記述があったら、それは10個のint型の値を入れることしかできません）。

それでは、次の質問に答えて下さい。

(質問) 次のような情報を格納する変数を用意したいとします。

- ・ 学年 (int型)
- ・ 組 (int型)
- ・ 名前 (char型, アルファベットで20文字)

現在、君達が持っている知識の範囲で、この情報を格納するための変数宣言を解答欄に記述しなさい。

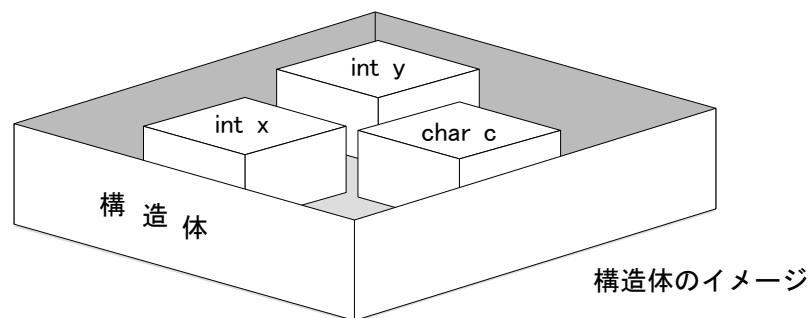
解答欄

現在持っているの知識を使って表現すれば、おそらく次のようになるのではないのでしょうか。

```
int gakunen;    // 学年
int kumi;       // 組
char namae[20]; // 名前
```

このように3つの変数（配列）を一ヶ所でまとめて宣言すれば、これらの変数が互いに関連した情報であることは、ある程度推測はできます。しかしプログラム中では、これらはあちらこちらでバラバラに参照されます。この場合、各変数が出現した部分だけを見ていたのでは、これらが互いに関連していることを読み取るのは、かなり困難ではないでしょうか。

そこで、C言語ではこのような互いに関連のあるデータをまとめて、1つの型として宣言する機能があります。これが構造体(structure)と呼ばれるものです。構造体は、複数の変数（もちろん、異なる型の変数が混在してもかまいません）をひとくくりにして箱詰めしたような、構造を持つデータだと考えればいいでしょう（下図）。



一般に、構造体を用いる変数は、構造体を定義した後で、次のように宣言します。

~~~~~ 構造体による変数宣言 ~~~~~

```

struct 構造体タグ名 {
    データ型 メンバ変数;
    データ型 メンバ変数;
    ⋮
}; ←「;」を忘れやすいので注意

```

} 構造体の定義

```

struct 構造体タグ名 変数1, 変数2, ..., 変数n; ←構造体変数の宣言
    ↑
    これ全体が型名になる

```

~~~~~

メンバ変数とは、構造体の内部で宣言された変数のことです（つまり構造体という箱の中にある1つ1つの変数）。なお、構造体定義の最後の「;」を忘れやすいので注意してください（ここまでが構造体定義文という、1つの文になります）。

先程の質問の答えは、構造体を使うと次のように表現できます。

```

struct student {
    int gakunen;    // 学年
    int kumi;       // 組
    char namae[20]; // 名前
};
struct student my_data;

```


ここで、student が構造体タグ名、gakunen, kumi, namae[] がこの構造体のメンバです。そして、my_data が、これら3つの変数を内部に持つ「struct student」という型の構造体変数ということになるわけです。

では、構造体の内部にあるメンバ変数を参照するにはどうするのでしょうか。構造体の各メンバは、構造体変数とそのメンバ変数を「.」（ピリオド）でつなぐことによって参照できます。例えば、「struct student」型の変数 my_data 中のメンバ変数 gakunen に値を代入したい時は、次のようにします。

```
my_data.gakunen = 2;
```

これは、「.」（ピリオド）を「の」と読んで、構造体変数 my_data 「の」 gakunen と考えると分かりやすいと思います。

```
my_data.gakunen = 2;
```

my_data の gakunen に2を代入する

~~~~~ 構造体のメンバの参照 ~~~~~

「構造体変数. メンバ変数」… 構造体変数「の」中にあるメンバ変数

次の例題を実行して確かめてみて下さい。

----- 例(15)-1 -----

```
#include <stdio.h>
#include <hamakou.h>

struct student {
    int gakunen;    // 学年
    int kumi;       // 組
    char namae[20]; // 名前
};

main()
{
    struct student my_data;

    my_data.gakunen = getint("学年: ");
    my_data.kumi = getint("組: ");
    getstring("名前: ", my_data.namae);

    printf("学年 = %d\n", my_data.gakunen);
    printf("組 = %d\n", my_data.kumi);
    printf("名前 = %s\n", my_data.namae);

    return(0);
}
```

構造体変数を宣言と同時に初期化するには、配列の初期化と同じように、メンバ変

数の初期値を「,」（カンマ）で区切ったものを「{ }」で囲みます。このとき、もちろんメンバ変数を宣言した順序と型、「{ }」内の初期値の並び順と型は一致していなければなりません（例(15)-2）。

----- 例(15)-2 -----

```
#include <stdio.h>

struct student {
    int gakunen;    // 学年
    int kumi;       // 組
    char namae[20]; // 名前
};

main()
{
    struct student my_data = { 2, 1, "浜工太郎" };

    printf("学年 = %d\n", my_data.gakunen);
    printf("組   = %d\n", my_data.kumi);
    printf("名前 = %s\n", my_data.namae);

    return(0);
}
```

また、構造体変数の配列を作ることができます。例えば、例(15)-1や例(15)-2で、40人分のデータを扱えるようにするには、次のように構造体変数を配列にしてやればよいわけです。

```
struct student {
    int gakunen;    // 学年
    int kumi;       // 組
    char namae[20]; // 名前
};

main()
{
    struct student our_data[40];
    ...
}
```

構造体変数が配列であっても、そのメンバの参照の仕方は先程と同様です。この構造体変数の配列 `our_data[]` の0番目の要素（すなわち `our_data[0]`）のメンバ変数 `gakunen` に 2 を代入するには、次のようにします。

```
our_data[0].gakunen = 2;
```

ポイント  
☆構造体とは

---

---

---

問題(15)-1 構造体とは何か、具体的な定義の例を挙げて説明しなさい。

問題(15)-2

次のプログラムは、3教科の試験の合計点を学生ごとに表示するものですが、まだ完成していません。これを完成させて、実行結果のように表示されるようにしなさい。

(toi15-02.c)

---- toi15-02.c -----

```
#include <stdio.h>

struct student {
    int id;           // 出席番号
    char name[20];    // 氏名
    int kokugo;       // 国語の点数
    int suugaku;      // 数学の点数
    int eigo;         // 英語の点数
};

main()
{
    struct student data[4] = {
        {1, "浜工太郎", 65, 90, 100},
        {2, "藤枝花子", 82, 73, 63},
        {3, "青島一郎", 74, 31, 41},
        {4, "島田和馬", 100, 95, 98},
    };
    int i;
    for (i = 0; i < 4; i++) {
        printf("%d %s の合計点は", _____, _____);
        printf("_____\n", _____);
    }

    return(0);
}
```

実行結果

- 1 浜工太郎 の合計点は 255 点です。
- 2 藤枝花子 の合計点は 218 点です。
- 3 青島一郎 の合計点は 146 点です。
- 4 島田和馬 の合計点は 293 点です。

## (16) 高度なデータ型 [2] : 自分だけの新しい型を作る (typedef)

一度次のような構造体を定義すれば、

```
{ struct student {  
    int gakunen;    // 学年  
    int kumi;       // 組  
    char namae[20]; // 名前  
};
```

これ以降はこの構造体型の変数を、例えば、

```
struct student s1, s2, s3;
```

などのようにして、宣言することができました（この例では s1, s2, s3 の3変数が、いわば struct student型 の変数ということでしたね）。

ところで、構造体変数を宣言するとき、いちいち struct をつけなければならないのは面倒ですし、第一、美しくないと思いませんか？ 実は、typedef文 を使うことにより、構造体変数も通常の単純変数を宣言するような形で、宣言することができるようになります。例えば、int x のように、struct を付け加えることなく宣言できるようになるのです。

この例の場合、次のようにします。

```
{ struct student {  
    int gakunen;    // 学年  
    int kumi;       // 組  
    char namae[20]; // 名前  
};  
  
main()  
{  
    typedef struct student Student;  
    ...  
}
```

こうすると、この構造体変数を宣言するのに、

```
Student s1, s2, s3;
```

のようにすることができるようです。これは、まさしく Student という型があるのと同じですね。

つまり、typedef とは、文字通り新しく型(type)を定義(define)するという働きをします。typedef文の書式は、次のようになります。

~~~~~ typedefの書式 ~~~~~

```
typedef   データ型   左のデータ型につける新しい型名 ;  
          ↑         ↑  
        空白文字   空白文字
```

~~~~~

ここで、データ型には、構造体の型名だけでなく任意の型名を記述することができます。ですから、もともとの型名を別の新しい型名にすることもできます。

以下に、typedef文の例をいくつかあげておきます。

- (1) `typedef char Byte;` // これ以後、型名 `Byte` は `char` を表す
- (2) `typedef unsigned int Word;` // これ以後、型名 `Word` は `unsigned int` を表す
- (3) `struct point {`  
    `float x, y;`  
    `};`  
    `typedef struct point Point;` // これ以後、`struct point` は単に`Point` でよい
- (4) `typedef struct point {`  
    `float x, y;`  
    `} Point;` // (3)で構造体の定義とtypedefを一度に行う
- (5) `typedef struct {`  
    `float x, y;`  
    `} Point;` // (4)の場合、構造体タグは不要

(1)は、`char` を `Byte` という別の名前に置き換える例です。変数の値を文字としてではなく、1byteという8bitのデータとして扱うことを明確にしたいときに使います。

(2)は、長い型名を短く扱いやすい型名に定義しなおす例です。ただし、長い型名の方が使えなくなったわけではありません。

(3)は、構造体 `sturct point` を新たな型名 `Point` で表しています。これ以後は、

`struct point p1, p2;`   ⇒   `Point p1, p2;`

のように、構造体変数の宣言を簡単にすることができます。

(4)は、構造体の定義と `typedef`文をひとつの文で記述した例です。このように記述する場合は、自己参照するような構造体（ポインタを使用する構造体でよく利用します）でなければ、構造体タグを省略することもできます。

(5)は、(4)の構造体タグを省略した例です。例(15)-2を(5)の形で書き直したのが次の例(16)-1です。

----- 例(16)-1 -----

```
#include <stdio.h>

typedef struct {
    int gakunen;    // 学年
    int kumi;       // 組
    char namae[20]; // 名前
} Student;

main()
{
    Student my_data = { 2, 1, "浜工太郎" };

    printf("学年 = %d\n", my_data.gakunen);
    printf("組   = %d\n", my_data.kumi);
    printf("名前 = %s\n", my_data.namae);

    return(0);
}
```

ポイント

☆typedef文

---

---

---

---

問題(16)-1 typedef文の働きを説明しなさい。

問題(16)-2 (toi16-02.c)

- (1) 日付を表すデータ型 Date を構造体を用いて定義しなさい。なお、Date型には少なくとも年, 月, 日の情報が格納できること。
- (2) (1)で定義したDate型の値を2つ引数とし、そのうち、新しい方の日付を戻り値とする関数 laterDate() を実装しなさい。
- (3) 次の実行例のように、laterDate() 関数の動作を確認するプログラムを作りなさい。ただし、日付は年, 月, 日を"/"(スラッシュ)で区切る、"yyyy/mm/dd"の形式で入力するようにすること。

実行例

```
$ a.out 
日付 1 (yyyy/mm/dd) : 2013/06/07 
日付 2 (yyyy/mm/dd) : 1995/12/10 
2013/06/07
$
```

問題(16)-3 次のプログラムを作りなさい。(toi16-03.c)

- (1) 「時間の長さ」を表わす次のようなデータ型 Time を定義します。

メンバー	内容	値の範囲 (すべてint型とする)
day	日数	0 以上
hour	時	0 から 23 まで
minutes	分	0 から 59 まで

- ① 日数, 時, 分の値をキーボードから入力し、Time型の値を返す関数 inputTime() を実装しなさい。ただし、入力した値のどれかひとつでも上記の範囲をはずれている場合は、“Input again.” と表示して再び入力待ちになるようにしてください。
- ② ①の inputTime() 関数を用いて Time型の変数 time に値を入力し、time のすべてのメンバーの値を画面に表示するプログラムを作りなさい。

- (2) (1)で定義した Time型の変数2個を引数として、両者の和を Time型の値として返す関数 timeAdd() を実装しなさい。

- (3) 「時間の長さ」(日数, 時, 分の組)を繰り返しキーボードから読み込み、読み込むたびにそれまでの「時間の長さ」の総和を出力するプログラムを、timeAdd() 関数を用いて作りなさい。

ただし、日数, 時, 分の組はこの順で“,”(カンマ)で区切って入力し、最後に3つの0を入力するとプログラムを終了するようにすること。

プログラムの動作は、例えば次のようになります。

実行例

```
$ a.out 
day, hour, minutes: 3, 12, 44 
sum = 3 12 44
day, hour, minutes: 1, 22, 54 
sum = 5 11 38
day, hour, minutes: 8, 21, 0 
sum = 14 8 38
day, hour, minutes: 0, 0, 0  (← 3つの0を入力すると終了)
$
```

## (17) ポインタという考え方 (ポインタ[1])

一般に、2つのint型変数 x, yの値を入れ換えるには、作業用に変数を1つ用意して、次のようにしますね。

```
work = x;    // -+
x = y;       // |- 入れ換え処理
y = work;    // -+
```

では、この部分の処理をswap()という関数にした、次のプログラムを実行してみましょう。

```
----- 例(17)-1 -----
{
#include <stdio.h>
#include <hamakou.h>

void swap(int x, int y);

main()
{
    int x, y;

    x = getint("x: ");
    y = getint("y: ");
    printf("交換前: x = %d, y = %d\n", x, y);

    swap(x, y);

    printf("交換後: x = %d, y = %d\n", x, y);
    return(0);
}

void swap(int x, int y)
{
    int work;    // 作業用変数

    work = x;    // -+
    x = y;       // |- 入れ換え処理
    y = work;    // -+

    return;
}
```

意図した結果は得られませんでしたね。この理由は、関数の仮引数は、その関数内部でのみ有効な局所変数であり、仮引数に渡されてくるのは、実引数のコピーであるという「値渡し」であったことを思い出せば、容易に理解できますね。

では、このように呼び出し元の値を変更するような関数を作ることは、できないのでしょうか？もちろん、そんなことはありません。関数へ変数の値を渡す代わりに、その変数のアドレスを渡すことにします。そして、関数内部からはそのアドレスを通して、間接的に呼び出し元の値を変更することができるのです。

具体的には次のようにします。



----- 例(17)-2 -----

```
1: #include <stdio.h>
2: #include <hamakou.h>
3:
4: void swap(int *p, int *q);
5:
6: main()
7: {
8:     int x, y;
9:
10:    x = getint("x: ");
11:    y = getint("y: ");
12:    printf("交換前: x = %d, y = %d\n", x, y);
13:
14:    swap(&x, &y);
15:
16:    printf("交換後: x = %d, y = %d\n", x, y);
17:    return(0);
18: }
19:
20: void swap(int *p, int *q)
21: {
22:     int work;    // 作業用変数
23:
24:     work = *p;    // ←
25:     *p = *q;    // 入れ換え処理
26:     *q = work;    // ←
27:
28:     return;
29: }
```

まず、14行目のswap()関数の呼び出しで、実引数に「&」がついています。

14:      swap(&x, &y);

この「&」は**アドレス演算子**といい、文字通り、変数のアドレスを求める演算子です。つまり「&x」は変数 x のアドレスであり、「&y」は変数 y のアドレスです。

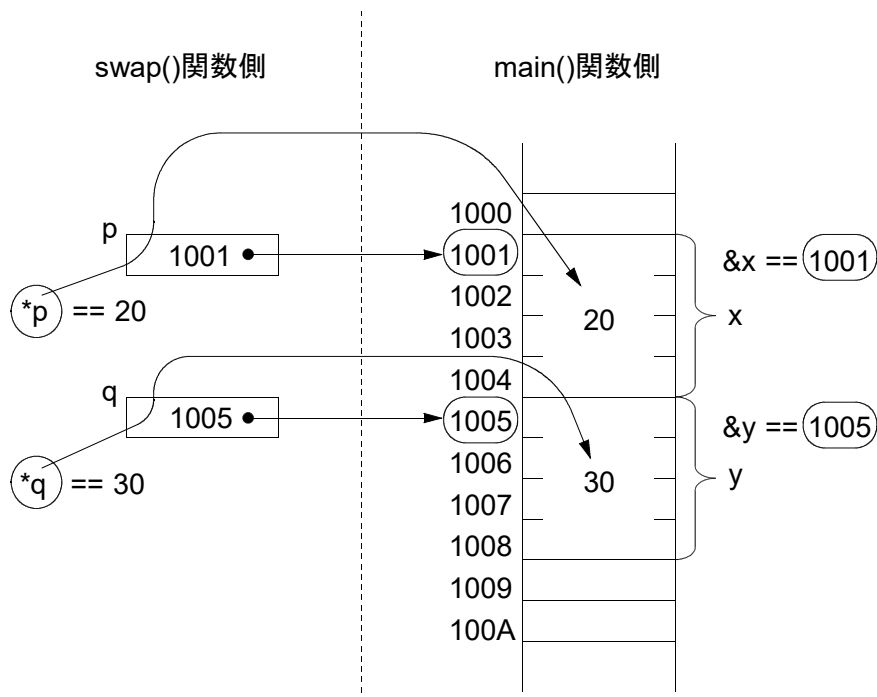
また、swap()関数の定義では、仮引数の宣言に「\*」がついています。

20: void swap(int \*p, int \*q)

これは、その引数がアドレスを受け取ることを表しています。呼び出し元では実引数に「&」がつけられ、swap(&x, &y)のように呼び出されていますから、当然、受け取る側の変数も、アドレスを格納できるものでなければならないのです。このpやqに格納されているアドレスのように、**変数が配置されているメモリ上のアドレスをポインタ**といい、**ポインタを格納する変数をポインタ変数**といいます。

そして、実際に交換を行っている部分でも、変数の前に「\*」がついています。この「\*」は**間接参照演算子**といい、その変数に格納されているアドレスを通して、間接的に実際の値を参照する働きをします（次図参照）。

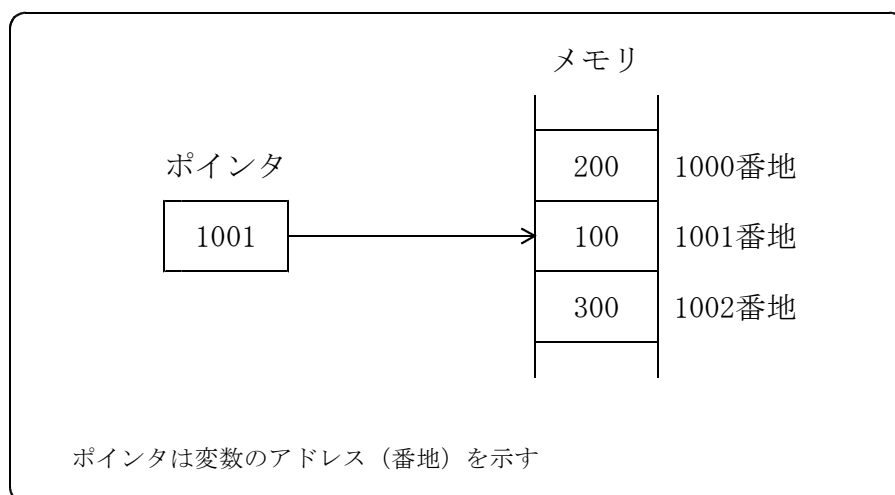
```
24:     work = *p;    // ←
25:     *p = *q;    // 入れ換え処理
26:     *q = work;    // ←
```



- swap(&x, &y) によって、main()側の x, y のアドレスが swap()関数に渡される
- \*p, \*q によって、main()側の x, y の値が参照できる

それでは、ポインタについてももう少し詳しく見ていきましょう。

ポインタとは、変数の配置されているメモリ上のアドレスを示す値のことでした。つまり、変数がある値を入れておく箱のようなものであるのに対し、ポインタはその箱がどこにあるかを示す情報であるといえます。



仮に「int x;」と変数を宣言した時、変数xが配置されるメモリはコンパイラが勝手に決めてしまいます。このアドレスは、アドレス演算子(address operator)である「&」（アンパサンド）を用いることで知ることができました。

では、実際に変数xのアドレスを求めてみましょう。

----- 例(17)-3 -----

```
#include <stdio.h>
main()
{
    int x;    // 変数 x

    printf("変数xのアドレス = %p\n", &x);

    return(0);
}
```

実行結果

変数xのアドレス = \_\_\_\_\_

このプログラムの中で登場した「%p」は、ANSI規格で採用されたポインタ表示用の書式制御文字です。

表示された値は、人それぞれ違うかもしれません。また、同じ人が実行してもそのたびに異なるかもしれません。しかし、このように、アドレス演算子を使う事によって、いつでもその変数の実際のアドレスを知ることができるのです。

今度は、&x によって得られたアドレスを保持できる変数を定義してみましょう。例えば、次のようにします

```
int *p;    // int型へのポインタ
```

このように宣言された変数 p を、ポインタ変数といいます。

この宣言は、int型へのポインタを宣言しています。これは、「変数p に格納されるのは、整数型の値が格納される場所のアドレスである」ことを表しています。また同時に、プログラムの実行文中で「\*p」と表記したとき、（変数p に有効なアドレスが格納されていれば）それが整数値になるということも表しています。「int x;」と宣言された変数 x を、実行文中でxと表記すればxに格納されている整数値を表しているのとまったく同じです。「\*p」が整数型変数になると考えてもいいでしょう。

同様に、double型の変数を指すポインタ変数を宣言するのであれば、次のように記述します。

```
double *p;    // double型へのポインタ
```

なお、特別なポインタとして何も指していないことを表す「**NULLポインタ**」というものが stdio.h で定義されています。特定の変数へのポインタを代入するまでは、

```
int *p = NULL;
```

などのように、ポインタ変数をNULLポインタで初期化しておくのがよいでしょう。

ポインタの意味を理解するために、次のプログラムを実行してみましょう。

----- 例(17)-4 -----

```
#include <stdio.h>
main()
{
    int *p = NULL; // ポインタ変数の宣言(NULLポインタで初期化)
    int x = 1;     // pがこのxを指すようにする

    p = &x;        // ポインタ変数 p へ変数 x のアドレスを代入
    printf("変数xのアドレス = %p\n", p); // ポインタ p を表示
    printf("x = %d\n", x);              // 変数 x を表示
    printf("*p = %d\n", *p);             // ポインタ p の内容を表示

    return(0);
}
```

このプログラムでは、

p = &x;

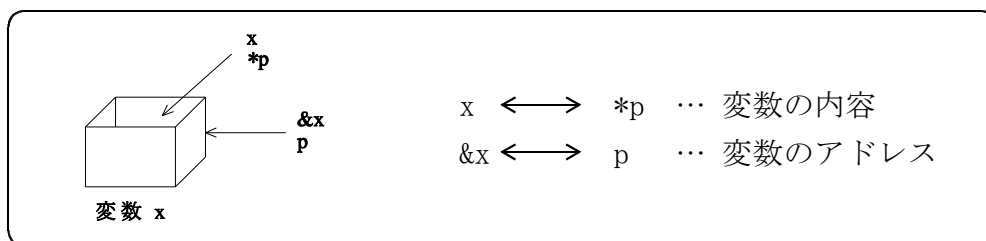
実行結果

変数xのアドレス = \_\_\_\_\_

x = \_\_\_\_\_

\*p = \_\_\_\_\_

とすることで、int型変数 x のアドレスをポインタ変数 p に格納しています。また、プログラムを実行して「\*p」を表示させると、変数 x と同じ値を表示します。実行文中の「\*p」という表記で、pが指す内容、つまり、変数 x の値を参照できることがわかりますね。この「\*」を間接参照演算子(indirection operator)といいました。この場合、これらの変数のはの関係を図示すると次のようになります。



ポイント

☆ポインタとは

---



---



---

☆アドレス演算子

---



---

☆間接参照演算子

---



---

## ○値渡しとアドレス渡し

ポインタについて理解できたところで、もう一度関数の引数の受け渡し方について考えてみましょう。次の例(17)-5では、main()関数から、proc1()関数に変数 a の値を渡しています。

----- 例(17)-5 -----

```
main()
{
    int a;
    ⋮
    proc1(a); // aの値(のコピー)を渡す
    ⋮
}

void proc1(int a)
{
    ⋮
}
```

----- 例(17)-6 -----

```
main()
{
    int a;
    ⋮
    proc2(&a); // aのアドレス(のコピー)を渡す
    ⋮
}

void proc2(int *a)
{
    ⋮
}
```

proc1()関数では、渡された値の格納用に新たに変数 a を用意しています。ここで注意しなければならないことは、proc()関数では、main()関数の変数 a そのものを使用しているのではなく、新しく用意された変数 a (main()関数のものとは別物)にその内容(値)だけをコピーし、proc()関数内ではそのコピーを使っているということです。

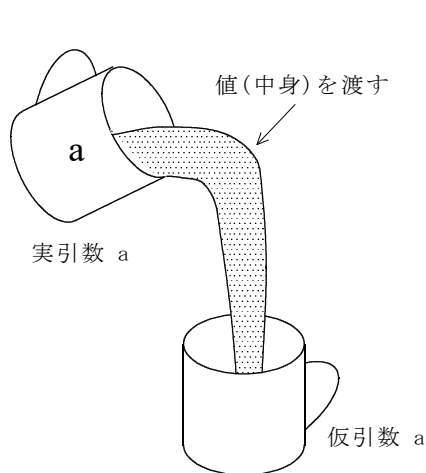
このように、関数呼び出しの時に実引数の値を仮引数に渡す方法を**値渡し(call by value)**と呼び、C言語ではこれが関数に実引数を渡す唯一の方法です。この方法を採用することにより関数の独立性を高め、関数外部からの影響を受けにくくしています。

しかし、先のswap()関数の例のように、場合によっては関数の呼び出し元の変数の内容(中身)を、それを受け取る側の関数で変更したいことがあります。この時には、値渡しの方法では関数内の値を変更しても、呼び出し元の変数の内容を変更することはできません。

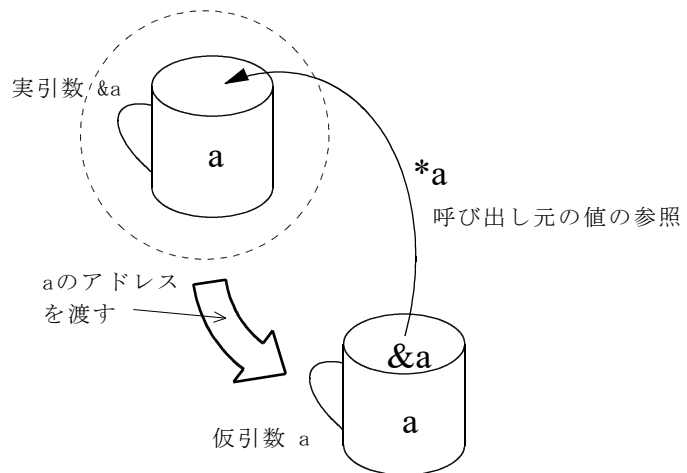
そこで、例(17)-6のようにポインタを用いて、呼び出す側では変数の値ではなく、その変数に割り当てられているメモリ上のアドレスを渡してやるのです。もちろん、**受け取る側ではこのアドレスのコピーを値渡しで受け取る**(あくまでもC言語には値渡ししかありません)のですが、渡されてくるのはアドレスなのですから、受け取る側ではそのアドレスを通して、呼び出し元にある変数を(この場合は \*a として)参照できるのです。このように実引数にはアドレスを与え、そのコピーを仮引数で受け取るやり方を、特に**アドレス渡し(call by address)**といいます。

なお、C言語では利用できませんが、C++やJava, VisualBASICなどには、関数内部から直接呼び出し元の変数を参照できる、**参照渡し(call by reference)**という関数呼び出しの方法もあります(ただし、Javaにはポインタという概念が言語構文上は存在しないため、アドレス渡しはありません)。

## 値渡し (call by value)



## アドレス渡し (call by address)



例(17)-7 は、main()関数内の変数countを、increment()関数によって+1 (インクリメント) するものです。

----- 例(17)-7 -----

```
#include <stdio.h>

void increment(int *p);

main()
{
    int count = 0;

    while (count < 10) {
        increment(&count);
        printf("count = %d\n", count);
    }

    return(0);
}

void increment(int *p)
{
    (*p)++;
    return;
}
```

実引数にcountを指定してしまうと、実際に渡されるのはcountのコピーであるので、main()関数側のcountはいつまでたってもインクリメントされません。そこで実引数にはcountの内容ではなく、countが割り当てられているアドレス (つまりcountを指すポインタ) を渡します。ここで、increment()関数の仮引数のデータ型が、int \*となっています。この関数に渡される引数はint型の値ではなく、int型を指すポインタだからです。

---

#### 問題(17)-1

ポインタとは何か説明しなさい。

#### 問題(17)-2

何が出来されるか予想しなさい。また、出力を確かめるプログラムを作って実行結果を確認し、予想と実際の結果について説明しなさい。

```
unsigned long long i = 3, *p = &i;
printf("%llu %llu %llu %llu %llu\n", p, p + 1, *p + 7, **&p, p - (p - 2));
```

#### 問題(17)-3

i と j がintで、p と q がintへのポインタであるとする、次の代入式のうち、正しくないのはどれですか？また、その理由についても説明しなさい。

```
p = &i;      p = &*i;      i = (int)p;      q = &p;
*p = &j;      i = (&)j;      i = *&*j;      i = *p++ + *q;
```

#### 問題(17)-4

(1) 次のように変数 x, y に scanf()関数を用いて、キーボードから値を入力するとき、scanf()関数の引数 x, y に「&」をつける理由を説明しなさい。

```
int x, y;
scanf("%d %d", &x, &y);
```

(2) 次のように、変数 x, y にキーボードから値を入力する input()という関数を作りました。正しく動作するように空欄を埋めなさい。

```
void input(int *p, int *q);
main()
{
    int x, y;
    input([ (a) ], [ (b) ]);
    :
}
void input(int *p, int *q)
{
    scanf("%d %d", [ (c) ], [ (d) ]);
    return;
}
```

#### 問題(17)-5

項目(12)で取り上げたP78の例(12)-1のプログラムを、例(17)-2で使ったswap()関数を用いて書き直しなさい。(toi17-05.c)

---

## (18) ポインタの高度な使い方 (ポインタ[2])

### ポインタとアドレス

ポインタは、変数が配置されているメモリ上のアドレスを表すものでした。では、アドレスという言葉があるのに、なぜわざわざポインタなどと呼ぶのでしょうか？ポインタと単なるアドレスとは何か違いがあるのでしょうか？このことをポインタが配列を指している場合の例を通して考えてみます。

まず、次のプログラムを実行してみましょう。このプログラムでは、データ型の異なる3つの配列に対して、それぞれの先頭アドレスを格納するポインタを用意しています。そして、その各ポインタを1ずつ増やししながら、ポインタ（のアドレス）とそれが指す場所に格納されている値を表示します。

----- 例(18)-1 -----

```
#include <stdio.h>
main()
{
    int    i;                // カウンタ変数
    char   chr_d[3], *chr_p; // char型変数・ポインタの宣言
    int    int_d[3], *int_p; // int型変数・ポインタの宣言
    double dbl_d[3], *dbl_p; // double型変数・ポインタの宣言

    chr_d[0] = 'a';
    chr_d[1] = 'b';
    chr_d[2] = 'c';

    int_d[0] = 100;
    int_d[1] = 200;
    int_d[2] = 300;

    dbl_d[0] = 1.111;
    dbl_d[1] = 2.222;
    dbl_d[2] = 3.333;

    chr_p = &chr_d[0]; // 先頭要素のアドレスを格納
    for (i = 0; i < 3; i++) {
        printf("chr_p[%d]: chr_p = %p, *chr_p = %c\n", i, chr_p, *chr_p);
        chr_p++;        // ポインタをインクリメント
    }

    int_p = &int_d[0]; // 先頭要素のアドレスを格納
    for (i = 0; i < 3; i++) {
        printf("int_p[%d]: int_p = %p, *int_p = %d\n", i, int_p, *int_p);
        int_p++;        // ポインタをインクリメント
    }

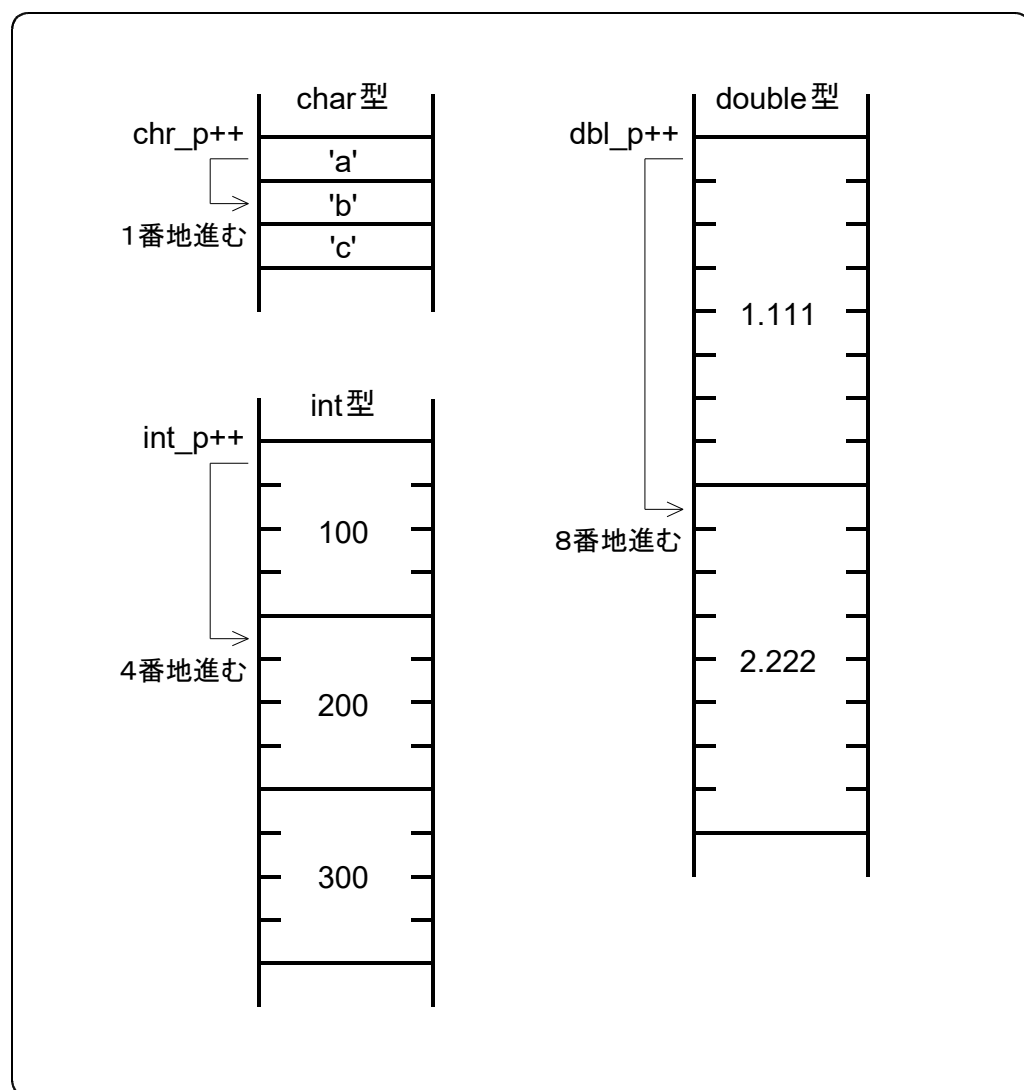
    dbl_p = &dbl_d[0]; // 先頭要素のアドレスを格納
    for (i = 0; i < 3; i++) {
        printf("dbl_p[%d]: dbl_p = %p, *dbl_p = %g\n", i, dbl_p, *dbl_p);
        dbl_p++;        // ポインタをインクリメント
    }

    return(0);
}
```



実行結果から、ポインタ自身の値を+1すると、アドレスは+1進むのではなく、対象となるデータ型の1要素分だけ進んでいるのがわかります。例えば、int型変数は4バイト（本実習での場合。処理系により異なります。詳細は項目(3)-②参照。）で構成されているため、int型を示すポインタを+1すると4番地進んでいます。このように、ポインタは単なるアドレスではなく、「データが占めるメモリ上の領域を指す概念」であり、参照先のデータ型の情報も保持しているということなのです。

ですから、配列のように同じサイズのデータが連続しているような場合、そのデータ型にかかわらず、ある要素を指しているポインタを+1するだけで、正確に次の要素を指すことができるのです。



## ポインタと配列

ところで、例(18)-1 の中で、

```
chr_p = &chr_d[0];
```

と先頭要素のアドレスを求めている部分がありますが、これは次のように記述しても同じ意味になります。

```
chr_p = chr_d;
```

実は配列名は、その配列の先頭要素を指す**ポインタ定数**<sup>※注意</sup>となっているのです。そのため、配列の先頭アドレスが必要なときは、配列名を記述するだけでよいのです。

※注意： 配列名は定数ですので、ポインタ変数のように代入することはできません。

例えば、左の例は正しい使い方ですが、右の例ではコンパイルエラーになります。

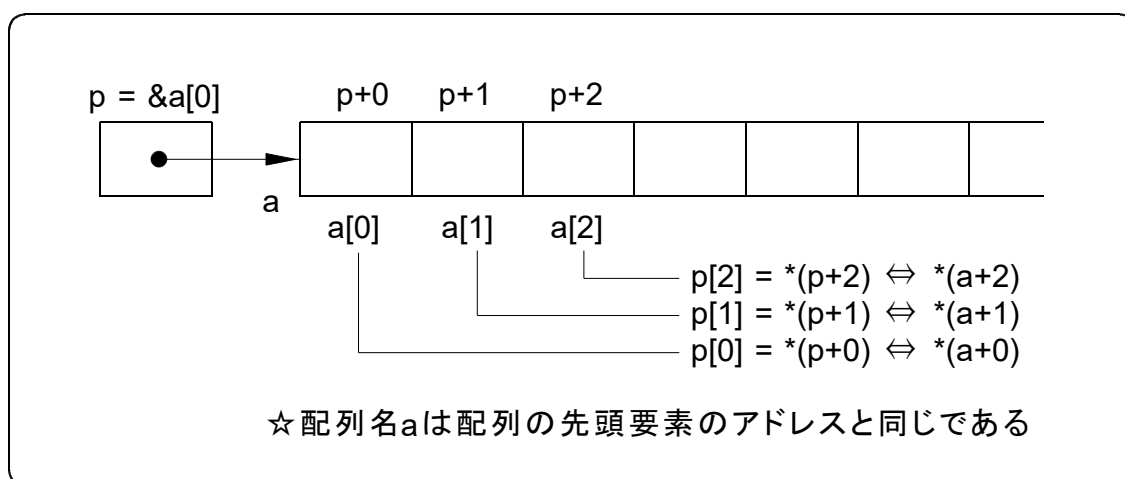
(正しい使い方)

```
int a[10], *p;  
p = a;  
p = p + 1; // pは変数なので代入可
```

(誤った使い方)

```
int a[10], *p;  
p = a;  
a = p + 1; // aは定数なので代入不可
```

また、ポインタに+1したときと同じ理由で、配列へのポインタに整数値を加えるという操作は、次図のように、配列の添字を操作するのと同じ意味になります。



ここで、配列名はその配列の先頭要素のアドレスを表していますので、配列 a の i 番目の要素を参照の仕方は、a[i]という書き方と、\*(a+i)という書き方があり、どちらも同じ意味になります。この図のように配列へのポインタが p に格納されていれば、もちろん a[i] と p[i] も同じ意味になります。つまり、配列とポインタは、その内容を操作するという点ではまったく同一なのです。

ただし、プログラムを書く際は、配列を扱う部分をわざわざポインタを用いて記述したり、また、その逆を行うことは、プログラムをいたずらに読みにくくするだけです。その処理にあった適切な記述法を選択すべきです。

## ポインタの配列

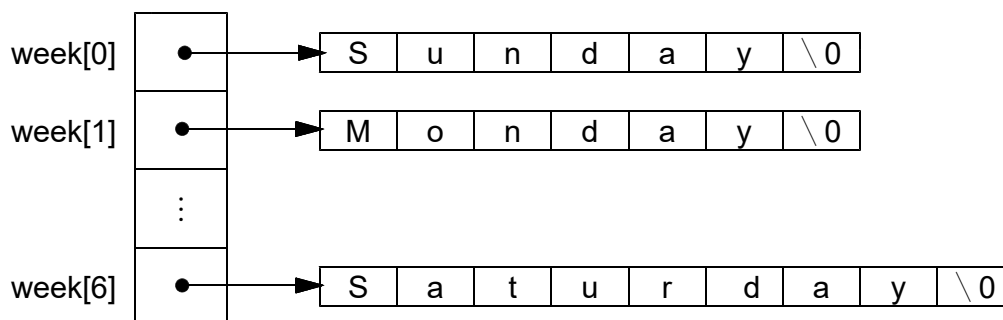
さて、次のような配列`week[]`の要素に、連続した文字列データを格納するにはどうしたらよいのでしょうか？

<code>week[0]</code>	"Sunday"
<code>week[1]</code>	"Monday"
<code>⋮</code>	<code>⋮</code>
<code>week[6]</code>	"Saturday"

このためには、次のように配列を宣言します。

```
char *week[7];
```

そして、配列`week[]`の各要素は、下図のように各文字列へのポインタ、すなわち各文字列を格納している文字配列の先頭アドレスで初期化します。これのように、要素がポインタであるような配列を、**ポインタ配列**といいます。



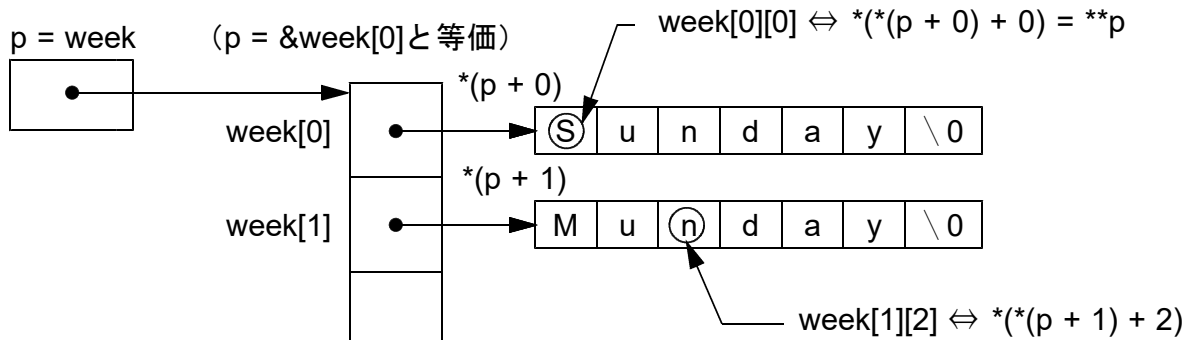
では、この配列`week[]`の先頭アドレスを格納するポインタ変数 (`p = week;` とするための変数 `p`) は、どのように宣言したらよいのでしょうか？結論から言えば、

```
char **p;
```

となります。これは次のように考えます。まず、`week[]` の型は `char *` ですね。ですから、`week[]` を指すポインタ変数 (`&week[0]` を格納する変数) は、この `char *` 型を格納できるものでなければなりません。従って、**`char **` 型**になるわけです。これは、ポインタを2つたどると、`char`型のデータに行き着くと考えてもいいでしょう。このように、ポインタ変数を指すポインタのことを**ポインタのポインタ**といいます。

次の例題は、配列`week[]`に格納されている、各ポインタが指す文字列の先頭文字のみを表示するものです。プログラムが理解できたら、実行してみましょう。

----- 例(18)-2 -----



```
#include <stdio.h>
main()
{
    char *week[7] = {"Sunday", "Monday", "Tuesday",
                    "Wednesday", "Thursday", "Friday", "Saturday"};
    char **p; // 注目: ポインタのポインタ
    int i;
    p = week;
    for (i = 0; i < 7; i++) {
        printf("%c\n", ***(p + i) + 0);
    }

    return(0);
}
```

### 問題(18)-1

次のプログラムは、配列を先頭から調べて0が初めて見つかるまでの要素数を求めるものです。これをポインタのみを用いて書き直しなさい。(toi18-01.c)

```
----- toi18-01.c -----
// 配列中の0が見つかるまでの個数をカウント

#include <stdio.h>

main()
{
    int array[10] = {4, 5, 8, 9, 8, 1, 0, 1, 9, 3};
    int index = 0;

    while( array[index] != 0) {
        index++;
    }
    printf("0が見つかるまでのデータ数は %d個です。\\n", index);

    return(0);
}
```



```
// 配列中の0が見つかるまでの個数をカウント (ポインタを使用)

#include <stdio.h>

main()
{
    int array[10] = {4, 5, 8, 9, 8, 1, 0, 1, 9, 3};
    int *array_p;

    array_p = _____; // 配列arrayの先頭アドレスを格納
    while( _____ != 0) {
        _____;
    }
    printf("0が見つかるまでのデータ数は %d個です。\\n", _____);

    return(0);
}
```

### 問題(18)-2

例(18)-2を参考に、配列 week[] に格納されている各ポインタが指す文字列の、最初の3文字だけを表示するプログラムを作りなさい。(toi18-02.c)

### 問題(18)-3

問題(18)-2 を修正して、配列 week[] に格納されている各ポインタが指す、すべての文字列を表示するプログラムを作りなさい。(toi18-03.c)

#### 問題(18)-4

整数配列 `source[]` の各要素の値を2倍にしたものを、整数配列 `twice[]` の各要素に格納するプログラムを、ポインタを用いて作りなさい。(toi18-04.c)

```

---- toi18-04.c -----
#include <stdio.h>

main()
{
    int source[10] = {1, 5, 8, 12, 2, 6, -6, 0, 4, 10};
    int twice[10];
    int i;

    for (i = 0; i < 10; i++) {
        _____ = _____ * 2;
    }
    for (i = 0; i < 10; i++) {
        printf("source[%d] = %d, twice[%d] = %d\n", i, _____, i, _____);
    }
    return(0);
}

```

#### 問題(18)-5

「名/姓」という文字列を入力し、これを「名」と「姓」という2つの文字列に分割したものを出力したいと思います。これを次の手順で行うプログラムを作りなさい。

(toi18-05.c)

<手順>

- ① 「名/姓」という文字列を `fgets()` 関数で入力する（名と姓を「/」で区切る）。  
ここで、入力する文字列は最大254文字とします。
- ② 読み込んだ文字列の最後の改行文字を取り除く。
- ③ 「/」（スラッシュ）のある位置を見つける。
- ④ 「/」を「\0」に書き換える。
- ⑤ 分割した2つの文字列「名」と「姓」を表示する

/home/pub/practice/c-masterディレクトリに、動作を確認するための実行プログラムは「name\_split」、ベースとなるプログラムは、「name\_split.c」というファイル名で置いてありますので、各自コピーしてから作業を進めなさい。

また、`strchr()` という関数を用いると、文字列に含まれる文字の位置を見つけることができます。この関数の使い方は各自調べ、必要に応じて利用しなさい。

name (入力した文字列)

T	a	r	o	u	/	H	a	m	a	k	o	u	\n	\0
---	---	---	---	---	---	---	---	---	---	---	---	---	----	----



first\_name (名)

T	a	r	o	u	\0
---	---	---	---	---	----

last\_name (姓)

H	a	m	a	k	o	u	\0
---	---	---	---	---	---	---	----

## (19) プログラムにコマンドラインを取り込む (main関数への引数)

UNIXやWindowsのコマンドの多くは、実行時にいろいろなオプションが付加できます。例えば、次の例の (下線部) がオプションです。また、このようなオプションを含めたコマンド入力文字列全体のことを、**コマンドライン**といいます。

UNIX

```
ls -l  
lpr -Pprnd
```

Windows

```
dir b: /w /a  
copy c:XXXXXX.XXX d:XXXXXX.XXX
```

質問

このようなオプションを、C言語プログラムで受取るにはどうしたらよいと思いますか？

(ヒント) Cプログラムは、常にmain関数から実行される。

C言語では、プログラムの実行は次のようなmain()関数から開始されます。

```
main()  
{  
    ...  
}
```

これまで、main()関数には戻り値の型も引数も記述してきませんでした。しかし、実はmain()関数にも戻り値と引数を記述することができます。

main()関数の引数には、第1引数に **int argc** を、第2引数に **char \*argv[]** (または **char \*\*argv**) が記述できます (argcとargvという仮引数の変数名は、慣例的に用いられているだけで、もちろん他の名前を使うこともできます)。これらの引数は、このプログラムをコンパイルし、実行したときのコマンドラインの情報 (コマンドラインで入力したすべての文字列とその個数) を受け取る役割を持ちます。

```
int main(int argc, char *argv[])  
{  
    ...  
}
```

すなわち、**argc**には、**コマンドラインに入力された文字列の個数が格納され** (この中には実行プログラム名も含まれます。従って、必ず**argc>=1**となります。)、**argv[]**には、**各文字列へのポインタが格納されます** (文字列へのポインタを格納する配列と考えればよいでしょう)。

このことは、例えば以下のプログラムで確認できます。

----- 例(19)-1 -----

```
int main(int argc, char *argv[])
{
    int i;

    printf("argc = %d\n", argc);
    for(i = 0; i < argc; i++) {
        printf("argv[%d]: %s\n", i, argv[i]);
    }
    return(0);
}
```

このプログラムをコンパイルし、次のように実行すると、

```
$ ./a.out Hamamatsu Technical High School Electrical Information
```

以下のような出力が得られます。

```
argc = 7
argv[0]: ./a.out
argv[1]: Hamamatsu
argv[2]: Technical
argv[3]: High
argv[4]: School
argv[5]: Electrical
argv[6]: Information
$ _
```

また、main()関数の戻り値は、OSにプログラムの終了状態を知らせるために利用します。UNIXでは、プログラムが正常終了した場合には0を、エラーなどで異常終了した場合には非0を返すことになっているので、main()関数の戻り値はintとします（戻り値を使用しない特別な場合は、voidとすることもあります）。

ポイント  
main()関数の引数

- argc ... \_\_\_\_\_  
\_\_\_\_\_  
\_\_\_\_\_
- argv ... \_\_\_\_\_  
\_\_\_\_\_  
\_\_\_\_\_



次の例は、オプションで指定した数によって、右図のような三角形を表示するプログラムです。コンパイル・実行してみなさい（なお、右図は、オプションに「5」を指定した場合です）。

```
$ a.out 5
+
+++
+++++
+++++++
+++++++
```

----- 例(19)-2 -----

```
#include <stdio.h>

void ch_line(char ch, int n);

int main( int argc, char *argv[] )
{
    int i;      // カウンタ変数
    int size;   // 三角形の大きさ
    int spc;    // ' ' の数
    int len;    // '+' の数

    if ( argc <= 1 ) { // パラメータが未指定の場合
        printf("表示する三角形の大きさを引数で指定して下さい。\\n");
    } else {
        sscanf(argv[1], "%d", &size); // 1つ目のオプションを文字(数字)から数値に変換

        len = 1;
        spc = size;
        for ( i = 1; i <= size; i++) {
            spc--;
            ch_line(' ', spc);
            ch_line('+', len);
            putchar('\\n');
            len += 2;
        }
    }
    return(0);
}

void ch_line(char ch, int n)
{
    int i;

    for (i = 0; i < n; i++) {
        putchar(ch);
    }
    return;
}
```

また、UNIXのように、オプションの形式が「-」と「アルファベット1文字」という形である場合のプログラムは、例えば次のようにします。

----- 例(19)-3 -----

オプションにより、表示する単語を次のように変えるプログラム。

```
-a ... apple (りんご)
-b ... banana (バナナ)
-c ... cherry (さくらんぼ)
```

例えば実行プログラム名を alphabet として、

```
alphabet -a 
```

のように実行すると、画面に次のように表示されます。

```
$ alphabet -a 
apple
```

また、複数のオプションを同時に指定した場合は、そのすべてを表示します。

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int i; // カウンタ変数

    // オプションの有無・正誤のチェック
    if (argc <= 1 || *(argv[1] + 0) != '-') {
        printf("オプションがないかまたは不正です。\\n");
        printf("正しく入力して下さい。\\n");
    } else {
        /*
         * オプションをチェックするループ
         * コマンドラインの引数をすべて調べつくすか、
         * またはハイフンのないオプションにであったら
         * ループから抜ける
         */
        i = 1; // カウンタ変数の初期化
        while (argc > i && *argv[i] == '-') {
            switch (*(argv[i] + 1)) {
                case 'a':
                    printf("apple\\n");
                    break;
                case 'b':
                    printf("banana\\n");
                    break;
                case 'c':
                    printf("cherry\\n");
                    break;
                default:
                    printf("該当する単語がありません：%s\\n", argv[i]);
            }
            i++;
        }
        if (i < argc) {
            printf("不正なオプションです：%s\\n", argv[i]);
        }
    }
    return(0);
}
```

```
$ alphabet -a -b -c 
apple
banana
cherry
```

問題(19)-1

例(19)-3を参考にして、次のプログラムを作成しなさい。(toi19-01.c)

プログラム名 : shopping (← cc -o shopping toi19-01.c でコンパイル)

りんご1個、バナナ3本というように買い物をし、購入した商品の名称・数量、消費税額・合計金額を印字します。購入する商品とその数量、消費税の税率(整数)は、プログラム実行時にオプションで次のように指定します。

ただし、商品の名称はアルファベット1文字の記号で表し、消費税の小数点以下は切り捨てます。

(例)    りんごを10個    ...    -a10  
         バナナを30本    ...    -b30  
         消費税 10%    ...    -10  
  
\$ shopping -a10 -b30 -10 

表示形式は右のようになります。

りんご	@100 × 10 =	1000
バナナ	@ 30 × 30 =	900
-----		
小 計		1900
消費税		190
=====		
合 計		2090

なお、商品は各自で追加し、単価も適当な値段を付けること(ただし、法外な金額は避ける)。

## (20) 必要時に変数領域を確保する（動的なメモリ確保）

プログラム内で配列のような連続したデータ領域を確保する場合、通常次のようにしますね。

```
int int_num[10];    // 整数10個分の領域を確保
```

ところが、このように確保した領域は、プログラムの実行中にその大きさを自由に増やしたり減らしたりすることはできません。そのため、最大限必要な領域を確保できるように、あらかじめ大きさを決めておかなければならず、通常その全部を使い切るようなことはまれです。例えば、平均的には5つのデータしか発生しないような処理でも、最大10個のデータが発生する可能性があるのなら、上の例のように10個分をあらかじめ確保しておく必要があります。このような処理では、ほとんどの場合で利用されていない領域も消費してしまうことになり、メモリの無駄使いになってしまいます（最近ではメモリは安価なのでこの程度の数ならまったく気にする必要はないのですが、巨大な配列を使用するような科学技術計算などでは無視できません）。

以前、項目(7)で、このような場合に利用できる可変長配列について触れましたが、可変長配列は、変数宣言の最後に、それも1つしか宣言できないなどいくつか制限があり、C言語の規格によっては使えないこともあります。また、そもそも配列では、リスト構造や木構造などの動的なデータ構造を実現するのには不向きです（もちろん、できないことはありませんが…）。

ここでは、そのような可変長配列などとは異なる、実行途中でも必要になればいつでも必要なだけの領域を確保できる「動的メモリ確保」の方法について学びます。

### malloc()関数とfree()関数

C言語の標準ライブラリ関数には、動的メモリ確保に便利な `malloc()` 関数というものを用意されています。また、動的に確保したメモリが不要になったとき、その領域を解放するための `free()` 関数も用意されています。

`malloc()` および `free()` 関数の外部仕様は、次のようになっています。なお、これらを使うには、必ずヘッダファイル `stdlib.h` をインクルードしておく必要があります。

#### malloc() 関数

形 式	<code>void *malloc(size_t size);</code> // <code>size_t</code> は符号なし整数型
機 能	<code>size</code> バイトのメモリ領域を確保。確保した領域は初期化されない。
戻り値	確保に成功したとき： 確保した領域へのポインタ 確保に失敗したとき： NULLポインタ

## free() 関数

**形 式** void free(void \*ptr);

**機 能** ptrが指す領域を解放する。ptrは、必ず以前に動的に確保した領域へのポインタでなければならない。そうでないときはエラーになり、動作は不定である。prtがNULLポインタのときは何もしない。

**戻り値** なし

malloc() および free() 関数は、一般的には次のように使います。

```
#include <stdio.h>
#include <stdlib.h>

main()
{
    :
    p = (ポインタ変数pと同じ型へキャスト)malloc(必要なバイト数); // ここで、メモリが必要になった
    if (p == NULL) { // メモリ確保に失敗した
        --- メモリ確保に失敗したときのエラー処理 ---
        retrun(1); // main()の戻り値は、異常終了を表す非0
    }
    :
    free(p); // ここで、pへ確保したメモリが不要になった
    :
}
```

具体的な使用例を次のプログラムで示します。

----- 例(20)-1 -----

```
#include <stdio.h>
#include <stdlib.h>

main()
{
    int *array_p; // 確保する領域へのポインタ
    int i; // カウンタ変数

    array_p = (int *)malloc(sizeof(int) * 5); // 5個分の領域確保
    if (array_p == NULL) {
        printf("メモリを確保できませんでした。\\n");
        return(1);
    }

    for (i = 0; i < 5; i++) {
        *(array_p + i) = i * i; // 5個のデータを格納
    }
    printf("int size = %d\\n", sizeof(int));
    // 各データの格納アドレスと内容を表示
    for (i = 0; i < 5; i++) {
        printf("%d:address = %p, value = %d\\n", i, (array_p + i), *(array_p + i));
    }
    free(array_p);

    return(0);
}
```

例(20)-1 は、int型のデータを5個、連続した領域に確保しています。malloc()で確保した領域へのポインタは void \*型になるので、これをint型の領域として利用するために、(int \*)と型変換（キャスト）していることに注意してください。

また、メモリ確保に失敗したとき、そのままそれ以降の処理を進めてはいけません。存在しないメモリ領域を参照してしまうことになり、セグメント例外などを引き起こしてしまいます。malloc()関数は、メモリの確保に失敗したときNULLポインタを返すので、これを利用して必ずエラー処理を行うようにしましょう。その際、main()関数の戻り値は、異常終了を表す非0値を返すようにします (main()関数の戻り値は0Sに返され、シェルスクリプトなどで利用されます)。

なお、項目(18)で学んだように、配列とポインタは相互に書き換え可能ですから、確保した領域を次のように配列として参照することもできます（例(20)-2）。

----- 例(20)-2 -----

```
{
#include <stdio.h>
#include <stdlib.h>

main()
{
    int *array_p; // 確保する領域へのポインタ
    int i;        // カウンタ変数

    array_p = (int *)malloc(sizeof(int) * 5); // 5個分の領域確保
    if (array_p == NULL) {
        printf("メモリを確保できませんでした。\\n");
        return(1);
    }

    for (i = 0; i < 5; i++) {
        array_p[i] = i * i; // 5個のデータを格納
    }

    printf("int size = %d\\n", sizeof(int));
    // 各データの格納アドレスと内容を表示
    for (i = 0; i < 5; i++) {
        printf("%d:address = %p, value = %d\\n", i, &array_p[i], array_p[i]);
    }

    free(array_p);

    return(0);
}
```

もちろん基本データ型だけでなく、構造体などでも同様に動的に確保することができます。次の構造体を動的に確保するプログラムを、例(20)-3に示しておきます。

```
{
typedef struct {
    int year; // 年
    int month; // 月
    int day; // 日
} Date;
```

----- 例(20)-3 -----

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int year;    // 年
    int month;   // 月
    int day;     // 日
} Date;

main()
{
    Date *array_p; // 確保する領域へのポインタ
    int i;         // カウンタ変数

    array_p = (Date *)malloc(sizeof(Date) * 5); // 5個分の領域確保
    if (array_p == NULL) {
        printf("メモリを確保できませんでした。\\n");
        return(1);
    }

    for (i = 0; i < 5; i++) {
        (array_p + i)->year = i + 2000; // メンバyearへiの値を代入(アロー演算子)
    }
    // 各データの格納アドレスと内容を表示
    printf("Date size = %d\\n", sizeof(Date));
    for (i = 0; i < 5; i++) {
        printf("%d:address = %p, year = %d\\n", i, (array_p + i), (array_p + i)->year);
    }
    free(array_p);

    return(0);
}
```

この例の中で、

`(array + i)->year`

という記述が使われていますが、これは、**ポインタ変数が指す構造体のメンバを参照**するときの表記法です。この「->」を間接メンバ演算子、またはその形からアロー演算子とか矢印演算子などと呼ぶこともあります。

間接メンバ演算子を使った表現は、

`(*(array + i)).year`

と記述するのと同じです。ただし、間接参照演算子「\*」よりも、メンバ参照演算子「.」の方が優先順位が高いため、\*(array + i).year とすると違う意味になってしまいます。これはよくあるミスなので注意してください。また、もちろん配列表現で、

`array[i].year`

のように記述することもできます（例(20)-4）。

----- 例(20)-4 -----

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int year;    // 年
    int month;   // 月
    int day;     // 日
} Date;

main()
{
    Date *array_p; // 確保する領域へのポインタ
    int i;         // カウンタ変数

    array_p = (Date *)malloc(sizeof(Date) * 5); // 5個分の領域確保
    if (array_p == NULL) {
        printf("メモリを確保できませんでした。\\n");
        return(1);
    }

    for (i = 0; i < 5; i++) {
        (*(array_p + i)).year = i + 2000; // メンバ参照演算子の利用
    }
    printf("Date size = %d\\n", sizeof(Date));
    // 各データの格納アドレスと内容を表示(配列表現の利用)
    for (i = 0; i < 5; i++) {
        printf("%d:address = %p, year = %d\\n", i, (array_p + i), array_p[i].year);
    }
    free(array_p);

    return(0);
}
```

ポイント

☆必要な時に必要なだけメモリを確保する方法

---

---

---

☆malloc関数

---

---

☆free関数

---

---



---

問題(20)-1

動的メモリ確保の利点を説明しなさい。

問題(20)-2

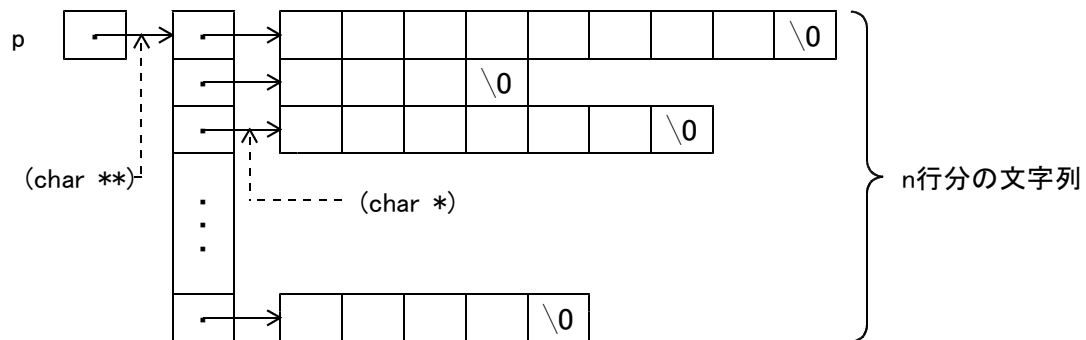
`calloc()` 関数、および `realloc()` 関数について調べなさい。

問題(20)-3

以下の①～④の手順にしたがって  $n$  行分の文字列を扱うプログラムを、`malloc()` 関数を用いて作りなさい。

- ① キーボードから行数  $n$  を入力します。
- ② 続いて、①で入力した  $n$  行分の文字列を連続して入力します。このとき、入力した文字列は、下図のような構造になるようにすべて動的に確保した領域に格納します。ただし、格納する文字列に改行文字は含めないものとし、各文字列を格納する領域は文字列の長さに合わせた大きさにします。なお、入力できる 1 行の最大文字数は、ヌル文字を含めて 256 文字とします。
- ③ 入力した  $n$  行の文字列を、入力した順にすべて表示します。
- ④ プログラム終了時に、動的に確保したすべての領域を `free()` 関数で解放します。

(toi20-03.c)



## (21) 自分で作った関数・マクロをいつでも使えるようにする

### (ライブラリとヘッダファイル)

これまで、C言語のプログラムの先頭部分には、`#include <…>` という文をおまじないとして書いてきました。例えば次の文です。

```
#include <stdio.h>
```

これは、「`stdio.h`というファイルの中身をプログラム中に挿入しなさい」という、プリプロセッサに対する指示だったのです（プリプロセッサとは、コンパイルの前にマクロの展開など様々な前処理を行うプログラムです）。

このように、プログラム中に他のファイルを挿入することを「**インクルードする**」といい、挿入されるファイルを「**インクルードファイル**」と呼びます。また、プログラムの先頭部分でインクルードされることから、「**ヘッダファイル**」とも呼ばれます。このことから、インクルードファイル（ヘッダファイル）のファイル名の拡張子には、「`.h`」を付けるのが習慣になっています。

例に挙げた`stdio.h`ファイルでは、`printf()`関数や`scanf()`関数などの標準的な（standard）入出力（I/O）関数を定義しています。

なお、ファイル名を指定する時の記述方法には、次の2通りがあります。

#### (a) `#include <ファイル名>`

指定のヘッダファイルを、使用しているシステム固有の場所のみから探します（本校処理系では`/usr/include`ディレクトリ）。

#### (b) `#include "ファイル名またはパス名"`

指定したヘッダファイルを、まず、カレントディレクトリまたは指定されたパスから探し、なければ、使用しているシステム固有の場所から探します。例えば、次のような書き方ができます。

```
#include "myheader.h"           // カレントディレクトリを探す
#include "/home/ei19/ei1942/myheader.h" // 指定した絶対パスを探す
#include "../myheader.h"        // 親ディレクトリを探す
#include "include/myheader.h"   // includeの下を探す
```

このようなファイルのうち、C言語処理系が標準で用意し、様々な関数やマクロ、定数、構造体型などを定義しているファイルのことを、標準インクルードファイルまたは標準ヘッダファイルといい、次のようなものがあります（これまでにいくつかは使用したことがありますね）。

## ○標準ヘッダファイルの例

stdio.h … 入出力やファイル操作に関するもの  
stdlib.h … メモリ操作や乱数など一般的なユーティリティに関するもの  
math.h … 数学関数に関するもの  
string.h … 文字列操作に関するもの  
ctype.h … 文字の種類の判定や文字の変換に関するもの

※これ以外にもさまざまなものが用意されています。どんなものがあるか調べてみなさい。

なお、このような標準ヘッダファイルの内で定義されている、printf()などの関数を標準ライブラリ関数と呼びます。

## 標準のヘッダファイルをのぞいてみよう

stdio.hやstdlib.hなどの標準ヘッダファイルは、テキスト形式で保存されているため、自由に参照することができます。これらのファイルにはこういったものが定義されているかのぞいてみましょう。

本校情報技術科のUNIX上のC言語処理系では、/usr/includeというディレクトリに標準ヘッダファイルが格納されています。

## システムが用意するライブラリファイルを確認してみよう

システムが用意するライブラリファイルは、/usr/lib や /usr/local/lib というディレクトリにあります。UNIXのライブラリファイルは、“libライブラリ名.a” または “libライブラリ名.so” というファイル名になります。サフィックス（拡張子）が \*.a のものはスタティックリンク・ライブラリ（静的結合ライブラリ）、\*.so のものはダイナミックリンク・ライブラリ（動的結合ライブラリ）です。

＝ ポイント ＝

☆インクルード（ヘッダ）ファイル

\_\_\_\_\_

\_\_\_\_\_

☆標準ライブラリ関数

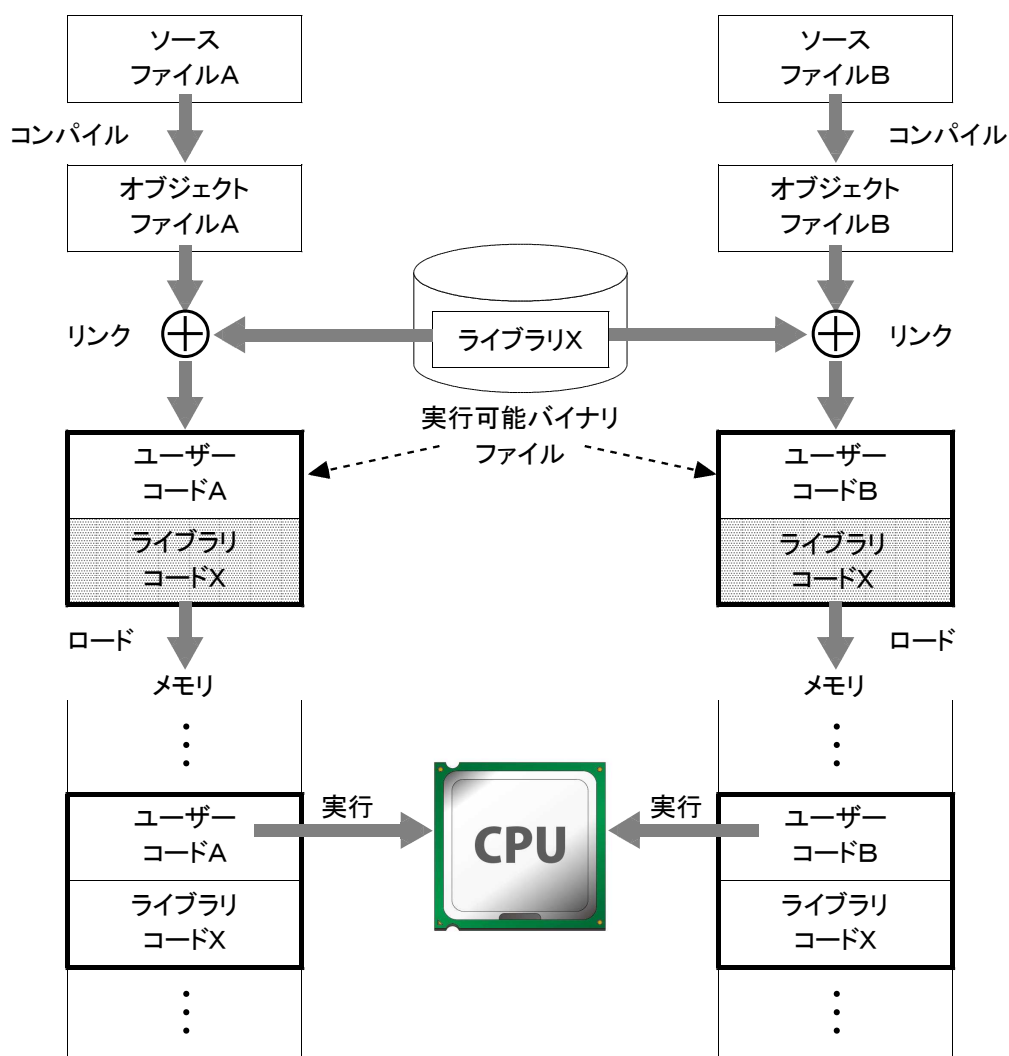
\_\_\_\_\_

\_\_\_\_\_

## スタティックリンク・ライブラリとダイナミックリンク・ライブラリ

### ○スタティック・リンクライブラリ（静的結合ライブラリ）

ライブラリファイルは、コンパイル（とリンク時）にユーザープログラムから生成されたオブジェクトファイルと静的に結合され、実行可能ファイルに組み込まれます（下図[スタティックリンク・ライブラリ]を参照）。

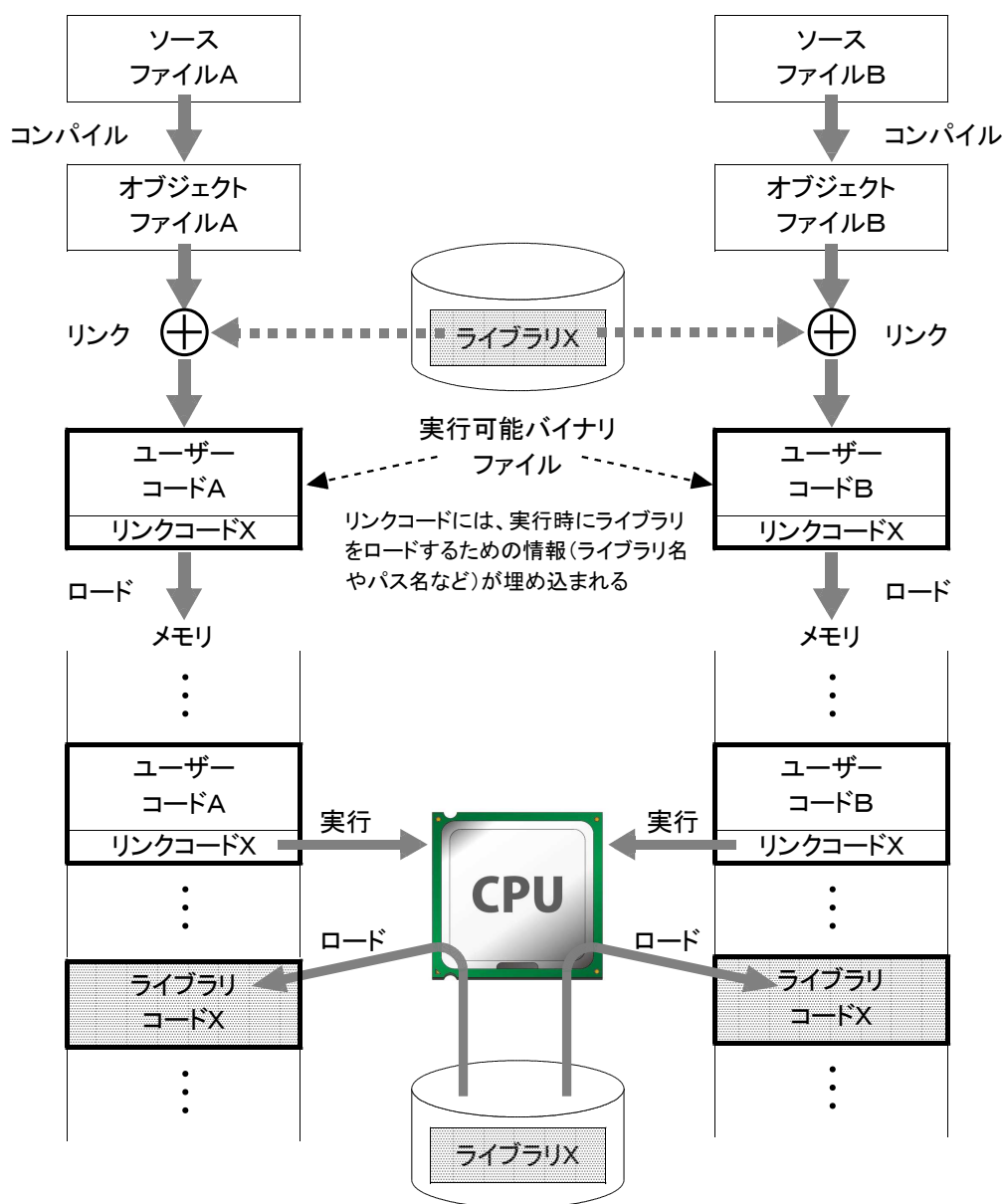


図[スタティックリンク・ライブラリ]

スタティックリンクライブラリを用いた実行可能ファイルのサイズは比較的大きくなり、ライブラリを保存するためのディスク消費量は多くなりますが、同一プラットフォーム（OSとCPUが同じ）であれば、別システム上でも実行可能ファイルの動作は保証されます。しかし、ライブラリを修正したり改良したりした場合、その恩恵を受けるためにはユーザープログラムの再コンパイルが必要になります。

## ○ダイナミックリンクライブラリ（動的結合ライブラリ）

ライブラリ自身は、コンパイルによって生成された実行可能ファイルには組み込まれません。実行可能ファイルには、ライブラリを呼び出すためのリンクコードが埋め込まれており、それによって、ライブラリは実行時に動的に結合されます（下図[ダイナミックリンク・ライブラリ]を参照）。



図[ダイナミックリンク・ライブラリ]

ダイナミックリンクライブラリを用いた実行可能ファイルのサイズは比較的小さくなり、ライブラリを保存するためのディスク領域の消費量は少なくてすみます。ライブラリを修正したり改良したりした場合も、実行可能ファイルは特に何もしなくてもその恩恵を受けることができます（ユーザープログラムの再コンパイルは不要）。逆

にライブラリに不具合があった場合、そのライブラリを利用しているすべての実行可能ファイルにその影響が及んでしまいます。

また、同一プラットフォーム（OSとCPUが同じ）であっても、別システム上での実行可能ファイルの動作は保証されません。ライブラリが存在しなかったり、格納場所のパスが違ったりした場合、実行可能ファイルは動作しません。ライブラリのバージョンの違いにより挙動が違ったり、複数バージョンが併存している場合の動作が特定できなかったりすることもあります。

## 自分用のライブラリとヘッダファイルを作ろう

よく利用するユーザー定義関数は、自分用のライブラリにしておくとう便利です。その際、これらのライブラリに含まれる関数のプロトタイプ宣言や、使用しているマクロ、構造体の定義などは、通常ヘッダファイルに記述して使用時にインクルードするようにします。

ここでは、自分用のライブラリとヘッダファイルの作り方、およびその利用方法を紹介します。ライブラリの一般的な作成手順は次のようになります。

### ○ ライブラリの一般的な作成手順

#### (1) ライブラリ用のヘッダファイルとソースファイル群を用意する

ヘッダファイル                   :   ライブラリ名.h  
ライブラリソースファイル群   :   ファイル<sub>1</sub>.c, ファイル<sub>2</sub>.c, …, ファイル<sub>n</sub>.c

#### (2) ライブラリのソースファイル群をコンパイルしてライブラリファイルを作る

このとき、ライブラリのファイル名は次のようにする必要があります。

スタティックリンク・ライブラリ:   libライブラリ名.a  
ダイナミックリンク・ライブラリ:   libライブラリ名.so

#### (a) スタティックリンク・ライブラリの場合 … libライブラリ名.a を作る

##### ① ライブラリソースファイル群をコンパイルしてオブジェクトファイルを作る

```
$ cc -c ファイル1.c ファイル2.c … ファイルn.c  
→ ファイル1.o ファイル2.o … ファイルn.o ができる
```

##### ② ①で作ったオブジェクトファイル群をライブラリファイルにする

```
$ ar crv libライブラリ名.a ファイル1.o ファイル2.o … ファイルn.o
```

#### (b) ダイナミックリンク・ライブラリの場合 … libライブラリ名.so を作る

次のようにコンパイルオプションを指定し、直接ライブラリを作る

```
$ cc -shared -fPIC -o libライブラリ名.so ファイル1.c ファイル2.c … ファイルn.c
```

## ○ ライブラリを使った実行可能ファイルの作り方

次のプログラムソースファイル群から、実行可能ファイルを作るものとします。

**プログラムソースファイル群** : ファイル<sub>1.c</sub>, ファイル<sub>2.c</sub>, ..., ファイル<sub>n.c</sub>

このとき、コンパイル時には次のオプションを指定します。

**ライブラリパスの指定** : -L ライブラリパス

**リンクライブラリの指定** : -l ライブラリ名

次のコマンドによりコンパイルし、a.out を作ります。

\$ cc プログラムソース<sub>1.c</sub> ... プログラムソース<sub>n.c</sub> -L ライブラリのパス -l ライブラリ名

※スタティックリンク・ライブラリとダイナミックリンク・ライブラリの両方が存在する場合は、ダイナミックリンク・ライブラリが優先的にリンクされます。

## ○ ライブラリ作成の実際

例として、日付を扱うデータ型と関数群を提供するライブラリを作ってみましょう。  
日付を扱うデータ型は年、月、日を格納する例(20)-3で用いたDate型とします。

```
{ typedef struct {  
    int  year;    // 年  
    int  month;   // 月  
    int  day;     // 日  
} Date;
```

また、日付を扱う関数には、以下の3つを用意することになります。

・ **Date getDate(void);**

キーボードから yyyy/mm/dd の形で入力した日付文字列をDate型にして返す。

・ **void printDate(Date date);**

引数dateに格納されている日付を、yyyy/mm/dd の形で表示する。

・ **Date laterDate(Date date1, Date date2);**

2つの引数date1とdate2のうち、新しい方の日付を返す。

(1) 次のライブラリ用のヘッダファイルとソースファイルを用意します

**ヘッダファイル** : date.h

**ライブラリソースファイル** : date.c

Date型の定義とこれらの関数のプロトタイプ宣言を、date.h というヘッダファイルに記述にします。date.h は次のようになります。

```

----- date.h -----
// 日付データを格納する構造体の定義
typedef struct {
    int year;
    int month;
    int day;
} Date;

Date getDate(void);
void printDate(Date date);
Date laterDate(Date date1, Date date2);

```

Date型を扱う関数は、date.c に実装します。date.c は次のようになります。ファイルの冒頭で、date.h をインクルードしていることに注意してください。

```

----- date.c -----
#include <stdio.h>
#include "date.h" // date.hはこのファイル(date.c)と同一ディレクトリに配置する

// キーボードから yyyy/mm/dd の形で入力した日付文字列をDate型にして返す
// 日付文字列の形式が正しくないときの戻り値は不定
Date getDate(void)
{
    char buff[15];
    Date date;

    fgets(buff, sizeof(buff), stdin);
    sscanf(buff, "%d/%d/%d", &date.year, &date.month, &date.day);
    return(date);
}

// 引数dateに格納されている日付を、yyyy/mm/dd の形で表示する
// 表示後、改行はしない
void printDate(Date date)
{
    printf("%4d/%02d/%02d", date.year, date.month, date.day);
    return;
}

// 2つの引数date1とdate2のうち、新しい方の日付を返す
Date laterDate(Date date1, Date date2)
{
    {
        if (date1.year > date2.year) {
            return(date1);
        } else if (date1.year < date2.year) {
            return(date2);
        } else if (date1.month > date2.month) {
            return(date1);
        } else if (date1.month < date2.month) {
            return(date2);
        } else if (date1.day > date2.day) {
            return(date1);
        } else {
            return(date2);
        }
    }
}

```



- (2) ライブラリのソースファイル(date.c)からライブラリファイルを作ります  
ライブラリファイル名は以下のようになります。

**スタティックリンク・ライブラリ** : libdate.a

**ダイナミックリンク・ライブラリ** : libdate.so

- (a) スタティックリンク・ライブラリ libdate.a を作る

```
$ cc -c date.c          ... date.o というオブジェクトファイルができる
```

```
$ ar crv libdate.a date.o ... libdate.a というスタティックリンク・ライブラリができる
```

- (b) ダイナミックリンク・ライブラリ libdate.so を作る

```
$ cc -shared -fPIC -o libdate.so date.c
```

```
... libdate.so というダイナミックリンク・ライブラリができる
```

- (3) (2)で作ったdateライブラリを使って実行可能ファイルを作ります

以前課題で取り組んだ、問題(16)-2 (toi16-02.c) のプログラムをライブラリを用いて作ってみます。

問題(16)-2 は、2つの日付文字列(年,月,日が"/"で区切られた、"yyyy/mm/dd"の形式)を入力キーボードから入力し、新しい方の日付を表示するものでした(実行例参照)。

#### 実行例

```
$ a.out
日付1 (yyyy/mm/dd) : 2013/06/07
日付2 (yyyy/mm/dd) : 1995/12/10
2013/06/07
```

date.hは、main()関数のソースファイルと同一ディレクトリに配置するものとすると、この問題のmain()関数は次のようになります(rei21-01.c)。

----- 例(21)-1 -----

```
//--- 2つの日付文字列を入力し、新しい方を表示する ---
#include <stdio.h>
#include "date.h" // date.hはこのファイル(rei21-01.c)と同一ディレクトリに配置する
main()
{
    Date Date1, Date2, Date3;

    printf("日付1 (yyyy/mm/dd): ");
    Date1 = inputDate();
    printf("日付2 (yyyy/mm/dd): ");
    Date2 = inputDate();

    Date3 = laterDate(Date1, Date2);
    printDate(Date3);
    putchar('\n');

    return(0);
}
```

ここでは、ヘッダファイル (date.h) 、ライブラリファイル (libdate.a または libdate.so) 、main()関数のソースファイル (rei21-01.c) がすべて同一ディレクトリにあるので、実行可能ファイル (a.out) は次のようにして作ります。

```
$ cc rei21-01.c -L . -l date
```

スタティックリンク・ライブラリとダイナミックリンク・ライブラリの両方が存在する場合は、ダイナミックリンク・ライブラリが優先的にリンクされます。どちらのライブラリがリンクされているかは、nmコマンドで確かめることができます。また、ダイナミックリンク・ライブラリの参照先パス (ロードパス) は、lddコマンドで確かめられます (下の実行例を参照) 。

(ダイナミックリンク・ライブラリをリンク)

```
$ nm a.out
0806094c D _DYNAMIC
08060e34 b _END_
08060904 D _GLOBAL_OFFSET_TABLE_
... (途中略) ...
0805077c t frame_dummy
      U getDate   ← U:コード未定義
      U laterDate ← U:コード未定義
080507e9 T main
08060a54 b object.2
08060a48 d p.0
      U printDate ← U:コード未定義
      U printf

$ ldd a.out
libdate.so =>      ./libdate.so
libX11.so.4 =>     /usr/lib/libX11.so.4
libm.so.2 =>       /lib/libm.so.2
libc.so.1 =>       /lib/libc.so.1
... (以下略) ...
```

(スタティックリンク・ライブラリをリンク)

```
$ nm a.out
08060a80 D _DYNAMIC
08060f5c b _END_
08060a3c D _GLOBAL_OFFSET_TABLE_
... (途中略) ...
08050798 t frame_dummy
080508a5 T getDate   ← T:定義済みコード
08050910 T laterDate ← T:定義済みコード
08050805 T main
08060b7c b object.2
08060b74 d p.0
080508f5 T printDate ← T:定義済みコード
      U printf

$ ldd a.out
... ここに、libdate.soのパスは存在しない
libX11.so.4 =>     /usr/lib/libX11.so.4
libm.so.2 =>       /lib/libm.so.2
libc.so.1 =>       /lib/libc.so.1
... (以下略) ...
```

## ポイント

### ☆スタティックリンク・ライブラリ

---

---

---

### ☆ダイナミックリンク・ライブラリ

---

---

---

問題(21)-1

例(20)-3 のプログラムを、前ページのようなmyheader.hというヘッダファイルを作り、これをインクルードする形に変更しなさい。

(※ レポート提出にあたっては、myheader.h も忘れずに提出すること。)

(toi21-01.c, myheader.h)

問題(21)-2

① トランプの1枚1枚を次のようなデータ型で表現するものとします。

```
typedef struct {  
    char  mark;    // トランプのマーク (H, D, S, C)  
    int   num;     // トランプの数字 (1~13)  
} Card;
```

② Card型の定義を「card.h」という名前のヘッダファイルに記述しなさい。card.hは、この演習を行うカレントディレクトリに置き、これ以降の設問で使います。

③ ②で作成したヘッダファイルcard.hを用いて、10枚分のカードの情報を格納する配列変数を用意したいと思います。これを動的メモリで確保しなさい。

(※ レポート提出にあたっては、card.h も忘れずに提出すること。)

(toi21-02.c, card.h)

```
---- toi21-02.c -----  
  
#include <stdio.h>  
#include <stdlib.h>  
#include <hamakou.h>  
_____ // 「card.h」 をカレントディレクトリからインクルード  
  
main()  
{  
    Card *card; // CARD型のポインタ変数  
    int i;  
  
    card = (_____)malloc(_____); // 構造体用領域の確保  
    if (card == NULL) {  
        printf("メモリを確保できませんでした。\\n");  
        return(1);  
    }  
  
    for (i = 0; i < 10; i++) {  
        _____ = getchar("マークを入力してください:"); ... 注※  
        _____ = getint("カードの数字を入力してください:");  
    }  
  
    for (i = 0; i < 10; i++) {  
        printf("%2d 枚目のカードは %c の %d です。\\n" i + 1, _____, _____);  
    }  
    free(card)  
    return(0);  
}
```

(注※: "char getchar(char \*msg)" は、浜工版の1文字入力関数です。)

### 問題(21)-3

問題(21)-2で定義した新しいCard型を利用して、52枚のカードをシャッフルする関数 `cardShuffle()` を作り、その結果を画面に表示するプログラムを作りなさい。

`main()` 関数と基本となるプログラムは、`/home/pub/practice/c-master` ディレクトリ内に、それぞれ `toi21-03.c` と `card.c` というファイル名で置いてあります。ただし、`card.c` にある次の関数のプロトタイプ宣言はヘッダファイル `card.h` に記述し、これをインクルードしなさい。また、`card.c` はライブラリ化し、`toi21-03.c` にリンクして実行プログラムを作ること。

- `cardSet()` 関数 … 52枚のカードを用意する
- `cardPrint()` 関数 … 与えられた1枚のカードを表示する
- `cardShuffle()` 関数 … 与えられた52枚のカードを指定回数シャッフルする

(※ レポート提出にあたっては、`card.h` と `card.c` も忘れずに提出すること。)

(`toi21-03.c`, `card.c`, `card.h`)

### 問題(21)-4

与えられた5枚のカードの中に、ワンペアがあるかどうか判断する `isOnePair()` 関数を作りなさい。この関数はワンペアだけがあるかを判断するものとし、ツーペア、スリーカード、フォーカードなどはワンペアとは判断しません。

外部仕様は次のように定義します。

```
int isOnePair(Card five_card[]);
```

[引 数] `five_card[]`: 5枚のカードを格納した配列

[戻り値] ワンペアがある場合 … 1  
ワンペアがない場合 … <sup>ゼロ</sup>0

次に、問題(21)-3で作ったトランプの山の任意の位置から、連続する5枚のカードを取り出し、この中にワンペアがあるかどうか判定します。判定結果を表示をするプログラムを `isOnePair()` 関数を使って作りなさい。ただし、最初に取り出すカードの位置は、必ず連続して5枚が取り出せる位置とします。

なお、画面表示は `main()` 関数内で行うこと。また、`isOnePair()` 関数のプロトタイプ宣言と関数定義は、それぞれ問題(21)-3で作った `card.h` と `card.c` に追加し、ライブラリとして `toi21-04.c` にリンクして実行プログラムを作ること。

(※ レポート提出にあたっては、`card.h` と `card.c` も忘れずに提出すること。)

(`toi21-04.c`, `card.c`, `card.h`)

### 問題(21)-5

問題(21)-4のプログラムに、ツーペア、スリーカード、フルハウス…などを判定する関数を追加し、実際のポーカーゲームに近づけなさい。

作成する関数については、問題(21)-4に準じて作成するものとし、外部仕様の詳細については各自で決定すること（ただし、各自が決めた仕様については、コメント等による説明を忘れずに記述すること）。

(`toi21-05.c`)

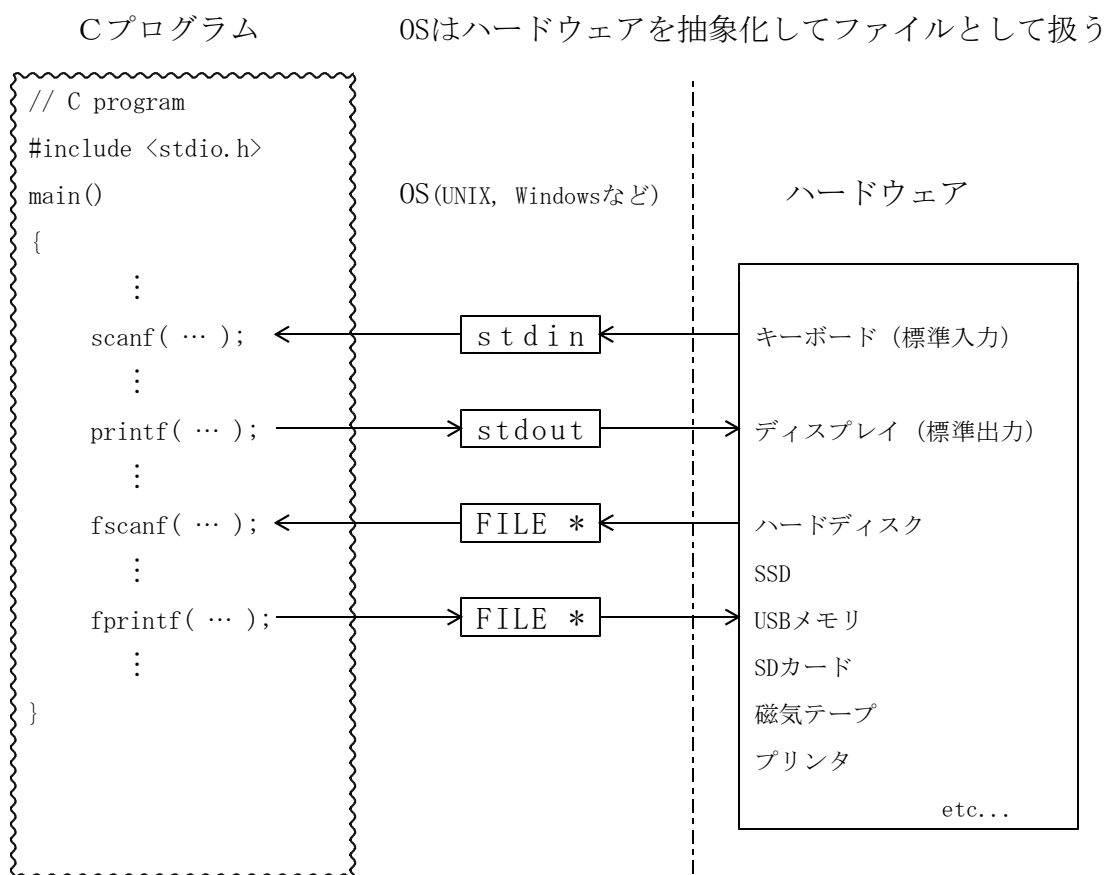
## (22) ファイル処理

これまで、入出力はキーボードや画面に対して行ってきました。しかし、キーボードから大量のデータを入力するのは大変な労力を要し、現実的ではありません。また、大量のデータをただ画面に垂れ流すのでは、後からそれを利用することもできません。

ここでは、そういった大量のデータをファイルから入力したり、ファイルへ出力したりする基礎的な知識と、ファイルに対して入出力するデータを、テキストおよびバイナリで扱う方法を学びます。

### ① ファイルとは

ファイルとは、任意のデータやプログラムなどを、物理的な記憶装置に格納するための論理的な単位のことです。C言語では、ファイルを単に連続したバイトデータの並び（バイト列：8ビットの2進データを並べたもの）として扱います。一般的なファイルは、ハードディスクやSSDなどのディスク装置、あるいはUSBメモリやSDカードなどのフラッシュメモリに記録してあるものをさす場合が多いのですが、UNIXでは、磁気テープなどの補助記憶装置や、プリンタ、ディスプレイおよびキーボード等の入出力装置もすべてファイルの一種として扱います。



## ② ファイル処理プログラムの一般形

C言語の標準ライブラリ関数の中には、ファイルを操作するための関数が数多く用意されていますが、これらファイル入出力用の関数の宣言やデータ構造は、標準ヘッダファイル `stdio.h` に記述されています。ファイルに対して何らかの操作をする時は、プログラムの冒頭で必ずこのファイルをインクルードしなくてはなりません。

```
#include <stdio.h>
```

ファイル関連の関数は、ファイル入出力を制御するために、**FILE**型というデータ型を使用します。この型の変数を指すポインタを、通常は1つのファイルに対して1つ、プログラムの中で宣言する必要があります。

```
FILE *変数名;    // コメント
```

このポインタをファイルポインタといいます。ファイルに対する入出力はこのファイルポインタを通して行われます。ファイルポインタが指すFILE型データの実体を用意する操作を、ファイルをオープンするといいます。FILE型データには、

- ・ 現在ファイル上のどこを読み書きしているかを示す位置（ファイル位置指定子）
- ・ 現在の位置からファイルの最後までバイト数
- ・ データを読み書きするバッファの位置
- ・ ファイルにエラーが起こったかどうかを示す情報
- ・ 現在のファイルが何番目にオープンされたかという情報

などの情報が格納されます。また、ファイルは使い終わったら、バッファ中に残っているデータをすべてファイル（ディスク上）に書き出し、不要になったファイル情報のためのメモリを解放しなければなりません。この操作をファイルをクローズするといいます。

ポイント

☆ファイルポインタ

---

---

☆標準入出力用のファイルポインタ

---

---

---

ファイルのオープンには**fopen()**関数により行い、ファイルのクローズは**fclose()**関数によって行います。fopen()およびfclose()関数の詳細を以下に示します。

#### ○ fopen()関数

**形 式** : FILE \*fopen(char \*filename, char \*mode);

**機 能** : 第1引数filenameが示す文字列を名前とするファイルをオープンする。

**戻り値** : オープンしたファイルを制御する**FILE型**データへのポインタを返す。オープン操作が失敗したとき、**NULL**ポインタを返す。

第2引数modeで指定する文字列は、次の文字のうちのいずれかで始まる。

mode	動 作	ファイルが存在しない場合	ファイルが存在する場合
"r"	読み取り(read)	エラー	
"w"	書き込み(write)	ファイルを新規に作成する	サイズを0にする(上書き)
"a"	追加書き込み(append)	ファイルを新規に作成する	ファイルの最後に追加

また、Windows(DOS)では、扱うファイルの種類によって、上記のモードに以下の改行文字変換の指定を、必要に応じて追加できる。

変換指定	意 味
"t"	テキストモードでオープン(入力時:[CR-LF]→[LF]/出力時:[LF]→[CR-LF])
"b"	バイナリモードでオープン(改行文字の変換を行わない)

なお、UNIXでは、テキストファイルとバイナリファイルを区別しないので、この変換は行われない。

#### ○ fclose()関数

**形 式** : FILE \*fclose(FILE \*fp);

**機 能** : 引数fpが示すファイルをクローズする。

**戻り値** : fpが示すファイルのクローズに成功した場合は0, 何らかのエラーを検出したとき**EOF**を返す

ファイル処理のプログラムは、一般的に次のような形になります。

```
#include <stdio.h>
#include <stdlib.h> // exit()関数を使用するため

int main()
{
    FILE *fp; // ファイルポインタの宣言
    ...

    fp = fopen(...); // ファイルのオープン
    if (fp == NULL) { // ファイルのオープンに失敗した場合
        printf("File Open ERROR!\n");
        exit(1); // エラーの場合は通常異常終了とする
    }
    ...

    ファイルに対する入出力処理
    (後述するfgetc(), fputc(), fgets(), fprintf()などの関数を使用する)
    ...

    fclose(fp); // ファイルのクローズ

    return(0);
}
```

またC言語には、次のような特殊なファイルポインタが用意されていて、プログラムの起動時に自動的にオープンされ、終了時にクローズされます。

標準ファイルポインタ	用 途	通常入出力先
stdin	標準入力(読み出し用にオープン)	キーボード
stdout	標準出力(書き込み用にオープン)	ディスプレイ
stderr	標準エラー出力(書き込み用にオープン)	ディスプレイ

※通常の入出力先は、パイプ (|) や I/Oリダイレクション (<, >, >>) 等により変更が可能

これらの使用方法は、前述したファイルポインタと同じです。ただオープン・クローズは自動的に行われるので、ユーザ側での操作は必要ありません。

ポイント

☆ファイルのオープン

---

---

---

☆ファイルのクローズ

---

---

---



### ③ 様々なファイル入出力関数

オープンされたファイルとの間でデータの読み書きを行うには、専用のファイル入出力関数を用います。入出力関数には、1 バイト入出力関数（1 文字入出力関数）、文字列入出力関数、書式付き入出力関数、ブロック入出力関数（バイナリ入出力関数）があり、それぞれ、次のような関数がよく使用されます。

入出力の単位	入力関数	出力関数
a. 1バイト(1文字)	fgetc()	fputc()
b. 文字列	fgets()	fputs()
c. 書式	fscanf()	fprintf()
d. ブロック(バイナリ)	fread()	fwrite()

このように、ファイル入出力関数は、関数名が **f** で始まるものがほとんどです。以下に、これらのファイル入出力関数の詳細を紹介します。

#### a. 1バイト入出力関数(1文字入出力関数)

オープンしたファイルから、1 バイト単位でデータを入力したり、結果を出力したりする関数に `fgetc()`、`fputc()`、`ungetc()` があります。バイト入出力関数は、関数名の最後が `c` になっています。

##### ○ 1バイト入力関数 `fgetc()`

**形 式** : `int fgetc(FILE *fp);`

**機 能** : `fp`が指すファイルから、次の1 バイトを`unsigned char`型の値として読み取り、`int`型に変換する。そして、そのファイルに結び付けられているファイル位置指定子を進める。

**戻り値** : `fp`が指すファイルから読み取った文字を返す。ファイルの終わりまたは読み取りエラーを検出すると、`EOF`を返す。

##### ○ 1バイト出力関数 `fputc()`

**形 式** : `int fputc(int c, FILE *fp);`

**機 能** : 第2引数`fp`が指すファイルに第1引数`c`で指定された文字を`unsigned char`型（1 バイト）に変換して書き込む。

**戻り値** : 書き込んだ文字を返す。書き込みエラーが発生すると、`EOF`を返す。

**注 意:** `fgetc()` 関数は文字単位の入力用であるのに、その戻り値は `int`型でした。  
これは `stdio.h` ファイルの中で定義されている `EOF(End Of File)` が `int`型であるために、このような仕様になっています。これに対応して、`fputc()` の出力データ用の第1引数 `c` も `int`型になっています。

## ○ 1バイト書き戻し関数 `ungetc()`

**形 式:** `int ungetc(int c, FILE *fp);`

**機 能:** `c`で指定される文字を`unsigned char`型に変換して`fp`が指すファイルに押し戻す。次の読取り処理は`c`から始まる。この関数は、ファイルに対応している外部記憶を変化させない。保証されるのは少なくとも1文字の押し戻しであり、読取りまたはファイル位置調整の操作を一度も介在させずに、この関数を何度も繰り返し呼び出すと、失敗することがある。

**戻り値:** エラーが発生すると`EOF`を返し、それ以外のときは押し戻された文字を返す。

### (参考)

これまで何気なく利用してきた、入出力関数`getchar()`や`putchar()`は、標準入出力のファイルポインタを用いて、

```
#define getchar()    fgetc(stdin)
#define putchar(c)   fputc(c, stdout)
```

と、マクロ定義したものです。

### 【 `fgetc()`, `fputc()` 関数の使用例 】

次のプログラムは、指定したファイル（この場合は、`alice.txt`というテキストファイル）をオープンし、その内容を1バイトずつ読み取り、そのまま標準出力に出力するものです。

```
----- 例(22)-1 -----
#include <stdio.h>    // ファイルに関して定義されているヘッダファイル
#include <stdlib.h>
int main()
{
    FILE *in_file;    // 入力ファイルへのファイルポインタ
    int  ch;          // データを入力する変数
    char *filename = "alice.txt"; // データを入力するファイル

    in_file = fopen(filename, "r"); // ファイルのオープン
    if (in_file == NULL) {
        printf("%sがオープンできませんでした。\\n", filename);
        exit(8); // 異常終了である非ゼロを返してプログラム終了
    }
    // エラーを検出するか、ファイルの終わりに達するまで入力が続ける
    while ((ch = fgetc(in_file)) != EOF) {
        fputc(ch, stdout); // putchar(ch); と同じ
    }
    fclose(in_file); // ファイルのクローズ
    return(0);
}
```

## b. 文字列入出力関数

オープンしたファイルから、文字列単位でデータを入力したり、結果を出力したりする関数に `fgets()`, `fputs()` があります。文字列入出力関数は、関数名の最後が `s` になっています。

### ○ 文字列入力関数 `fgets()`

**形 式** : `char *fgets(char *s, int n, FILE *fp);`

**機 能** : `fp`が指すファイルから文字列を読み取り、`s`が指す文字配列に格納する。読み取る文字数は最大`n-1`である。改行文字('\`\n`')を読み取ったとき、またはファイルの終わりを検出したとき、文字の読取りは終了する。読み取った改行文字も配列に格納し、配列に格納した文字列の直後に空文字 ('\`\0`') を書く。

**戻り値** : 入力に成功すると`s`を返す。ファイルの終わりを検出し、かつ配列に1文字も読み取っていなかった場合、配列の内容を変化させずに残し、NULLポインタを返す。読取りエラーが発生した場合もNULLポインタを返すが、この場合の配列の内容は不定である。

**注 意** : 文字列での入力の際、入力最大文字数 `n` が、文字配列 `s` の大きさより大きい場合、`s` の変数領域をはみだし、他の変数領域を侵食してしまう危険性がある。このため、`n` の部分には、`s` のサイズを指定する形をとるようにする。具体的には、`sizeof(s)` という形で文字配列`s`のサイズを取得するのがよい。

### ○ 文字列出力関数 `fputs()`

**形 式** : `int fputs(char *s, FILE *fp);`

**機 能** : `fp`が指すファイルに`s`が指す文字列を書き込む。空文字 ('\`\0`') の書込みは行わない。

**戻り値** : 書込みエラーが発生するとEOFを返し、それ以外の場合は非負の値を返す。

関数fgets(), fputs()を使用した例を次に示します。

## 【プログラム例2】 fgets(), fputs() 関数の使用例

次のプログラムは、指定したファイル（この場合は、alice.txtというテキストファイル）をオープンし、その内容を1行ずつ（最大255文字）読み取り、そのまま標準出力に出力するものです。

```
----- 例(22)-2 -----
#include <stdio.h> // ファイルに関して定義されているヘッダファイル
#include <stdlib.h>

int main()
{
    FILE *in_file; // 入力ファイルへのファイルポインタ
    char buff[256]; // 入力データを一時格納する変数
    char *in_filename = "alice.txt"; // データを入力するファイル

    in_file = fopen(in_filename, "r"); // 入力ファイルのオープン
    if (in_file == NULL) {
        printf("%sがオープンできませんでした。\n", in_filename);
        exit(8); // 異常終了である非ゼロを返してプログラム終了
    }

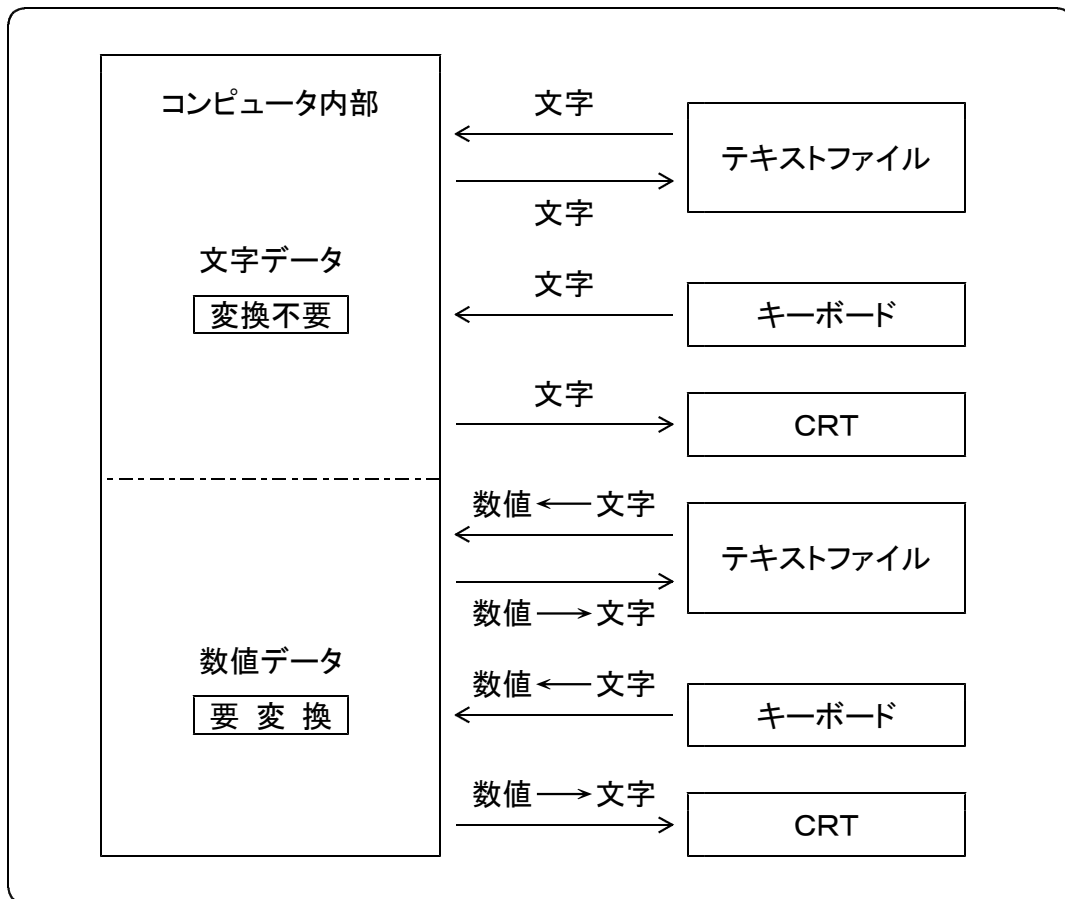
    printf("%sの内容\n", in_filename);
    // エラーを検出するか、ファイルの終わりに達するまで入力続ける
    while (fgets(buff, sizeof(buff), in_file) != NULL) {
        fputs(buff, stdout); // 標準出力へ出力
    }
    fclose(in_file); // 入力ファイルのクローズ

    return(0);
}
```

### c. 書式付き入出力関数

画面やキーボード、プリンタなどは、「文字」をデータの単位として扱います。例えば画面に「1234」という数値を表示したい時は、「1234」(2進数の10011010010)という数値ではなく、「1','2','3','4」という文字を順に出力するのです。この逆に、キーボードから「数値」を入力したいときは、まず「1','2','3','4」という文字の並びを入力し、それを「1234」(2進数の10011010010)という数値に変換しなければなりません。

ファイル入出力についても同様に、文字コードで書かれたファイル（これをテキストファイルといいます）から数値を入力したり、逆にコンピュータ内部の数値（2進数の値、これをバイナリデータといいます）をテキストファイルに出力する際には、文字列と数値の相互変換をする必要があります。



このような目的のために、ファイルから必要に応じて書式を指定し、それに基づいてデータの入出力を行う関数 `fscanf()`、`fprintf()` があります。これらの関数は、ファイルに対して入出力を行う点を除いて、第2引数以降の書式指定の方法や戻り値などは、標準入出力に対する `scanf()` および `printf()` 関数とまったく同じです。

## ○ 書式付き入力関数 `fscanf()`

**形 式** : `int fscanf(FILE *fp, char *format, ...);`

**機 能** : `fp`が指すストリームからの入力を変換し、`format`に続く実引数が指すオブジェクトに格納する。`format`が指す文字列は、入力として許される文字列とそれらの代入時の変換方法とを指定する書式とする。書式に見合うだけの実引数がないときの動作は定義されない。実引数が残っているにもかかわらず書式が尽きてしまう場合、余分の実引数は評価するだけで無視する。書式指定の詳細については、P224～230の「書式付き入力 - `scanf()`関数」を参照すること。

**戻り値** : 変換が一つでも行われなかった入力誤りが発生すると、EOFの値を返す。それ以外の場合、代入された入力項目の個数を返す。この個数は、入力中に照合誤りが発生すると、変換指定子に対応する実引数の個数よりも少なくなることがあり、0になることもある。

## ○ 書式付き出力関数 fprintf()

**形 式** : `int fprintf(FILE *fp, char *format, ... );`

**機 能** : `format`に続く実引数を、文字の並びである出力形式に変換して、`fp`が指すファイルへ出力する。その変換は、`format`が指す書式文字列内の指令にしたがう。書式文字列内には、指令が含まれていなくともよいし、複数含まれていてもよい。書式に対して実引数が不足しているときの動作は定義されない。実引数が残っているにもかかわらず書式がつきってしまう場合、余分な実引数は評価するだけで無視する。書式指定の詳細については、P216～223の「書式付き出力 - printf()関数」を参照すること。

**戻り値** : 転送された文字数を返す。出力エラーが発生したときは、負の値を返す。

### (参考)

`scanf()`や`printf()`関数は、標準入出力のファイルポインタを用いて、

```
int fscanf(stdin, char *format, ...);  
int fprintf(stdout, char *format, ...);
```

と、するのと同様です。

ポイント  
☆テキストファイル

---

---

---

ポイント  
☆1文字入出力

---

---

☆文字列入出力

---

---

☆書式指定付き入出力

---

---

---

## 【 fscanf(), fprintf() 関数の使用例 】

次のプログラムは、数字が書かれた入力ファイル“in\_num.txt”のデータを読み出し、この値を使用して計算を行い、新しいファイル“out\_num.txt”にその結果を数字で書き込むという動作をします。

なお、この例ではエラー表示を標準出力(stdout)ではなく、標準エラー出力(stderr)に出力していることに注意してください。また“in\_num.txt”は、/home/pub/practicにありますので、実行するときは自分の作業ディレクトリにコピーしてください。

----- 例 (22)-3 -----

```
#include <stdio.h> // ファイルに関して定義されているヘッダファイル
#include <stdlib.h> // 標準ライブラリ用ヘッダファイル

int main()
{
    FILE *in_fp; // 入力ファイルへのファイルポインタ
    FILE *out_fp; // 出力ファイルへのファイルポインタ
    char *in_filename = "in_num.txt"; // データを入力するファイル
    char *out_filename = "out_num.txt"; // データを出力するファイル
    int x, y; // ファイルからの入力データ用入力変数
    int z1, z2; // 計算結果を格納する変数

    in_fp = fopen(in_filename, "r"); // 入力ファイルのオープン
    if (in_fp == NULL) {
        fprintf(stderr, "%sがオープンできませんでした。\n", in_filename);
        exit(8); // 異常終了である非ゼロを返してプログラム終了
    }

    fscanf(in_fp, "%d", &x); // ファイルから数字を読み取りxへ格納
    printf("x = %d\n", x);
    fscanf(in_fp, "%d", &y); // ファイルから数字を読み取りyへ格納
    printf("y = %d\n", y);

    fclose(in_fp); // 入力ファイルのクローズ

    z1 = x + y;
    z2 = x - y;
    printf("z1 = %d\n", z1); // z1を標準出力へ表示
    printf("z2 = %d\n", z2); // z2を標準出力へ表示

    out_fp = fopen(out_filename, "w"); // 出力先ファイルのオープン
    if (out_fp == NULL) {
        fprintf(stderr, "%sがオープンできませんでした。\n", out_filename);
        exit(8); // 異常終了である非ゼロを返してプログラム終了
    }

    fprintf(out_fp, "z1 = %d\n", z1); // z1を出力ファイルへ書き込む
    fprintf(out_fp, "z2 = %d\n", z2); // z2を出力ファイルへ書き込む

    fclose(out_fp); // 出力ファイルのクローズ

    return(0);
}
```

次のようなプログラム例(22)-4を見てください。

- 160 -



#### d. ブロック入出力関数(バイナリ入出力関数)

これまで紹介してきた入出力関数では、基本的にファイルに格納するデータはすべて文字でした。また、入出力の単位も、1 バイト (1 文字) , 1 行, 1 書式といった具合に、文字を区切りとした単位です。C 言語では、文字以外のデータを入出力することももちろんできます。

ここでは、指定したバイト数分のデータを、加工せずにそのまま (バイナリで) 入出力する (例えば、500 バイト読み出せといったような) ブロック入出力を扱う関数、`fread()` および `fwrite()` を取り上げます。これらの関数を使用することによって、バイナリデータの入出力が可能になります。なお、バイナリデータを格納したファイルをバイナリファイルといいます。

#### ○ ブロック入力関数 `fread()`

**形 式** : `size_t fread(void *ptr, size_t size, size_t nmemb, FILE *fp);`

**機 能** : fp が指すファイルから、最大 nmemb 個の大きさ size の要素を、ptr が指す領域に読み取る。 ファイル位置指定子は、読取りに成功した文字数分だけ進む。エラーが発生したとき、ファイル位置指定子の値は不定とする。一つの要素の一部だけが読み取られたとき、その値は不定とする。

**戻り値** : 読取りに成功した要素の個数を返す。 その個数は、読取りエラーまたはファイルの終わりになったとき、nmemb より小さいことがある。size または nmemb が 0 のとき、0 を返す。このとき、ptr が指す領域の内容とファイルの状態は、変化しない。

#### ○ ブロック出力関数 `fwrite()`

**形 式** : `size_t fwrite(void *ptr, size_t size, size_t nmemb, FILE *fp);`

**機 能** : ptr が指す領域から、最大 nmemb 個の大きさ size の要素を、fp が指すファイルに書き込む。 ファイル位置指定子は、書込みに成功した文字数分進む。エラーが発生したとき、ファイル位置指定子の値は、不定とする。

**戻り値** : 書込みに成功した要素の個数を返す。 その個数は、書込みエラーが起きたときに限り、nmemb より小さくなる。

#### (補足) `size_t` 型について

あるデータ型がメモリ上に占める領域の大きさは、`sizeof` 演算子で求めることができます。例えば、`sizeof(配列名)` が返す値は、その配列の大きさをバイト数で表したものでしたね。ここで、扱うことのできる配列のサイズの最大値は、その処理系で指定できる非負整数、すなわち、`unsigned` 型整数の最大値になります。しかし、この値は処理系ごとに異なっていて、ある処理系では、`unsigned int` で表現できる大きさ

が上限ですが、別の処理系ではunsigned longで表現できる大きさまで扱えるかもしれません。

このように、処理系ごとの違いを吸収するためにsize\_t型が用意されているのです。size\_t型は、stddef.hというヘッダファイルで

```
typedef size_t unsigned int; や typedef size_t unsigned long;
```

などのように定義され、stdio.hなどから読み込まれるようになっています。

ポイント

☆バイナリファイル

---

---

---

☆ブロック入出力

---

---

---

## 【 fread(), fwrite() 関数の使用例 】

次のプログラムは、Point型の配列 p[] に格納されている10個のデータを、加工せずそのまま"point.dat"ファイルに出力します。その後、確認のため改めて"point.dat"ファイルから Point型配列 q[] に読み込み、それを表示するものです。

なお、このプログラムで作られたファイル "point.dat" は、バイナリファイルになっていることに注意してください ("point.dat"ファイルをlessコマンドなどで表示してみてください)。

----- 例(22)-6 -----

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    double x;
    double y;
} Point;

int main()
{
    Point p[10] = {{0.0, 0.0}, {0.0, 0.1}, {1.1, 1.2},
                  {2.2, 2.3}, {3.3, 3.4}, {4.4, 4.5},
                  {5.5, 5.6}, {6.6, 6.7}, {7.7, 7.8},
                  {8.8, 8.9}};

    Point q[10];
    FILE *fp;
    int i;

    fp = fopen("point.dat", "w");
    // 配列p[]から、Point型データを10個"point.dat"に書き出す
    fwrite(p, sizeof(Point), 10, fp);
    fclose(fp);

    fp = fopen("point.dat", "r");
    // "point.dat"から、配列q[]にPOINT型データを10個読み込む
    fread(q, sizeof(Point), 10, fp);
    fclose(fp);

    for (i = 0; i < 10; i++) {
        printf("(%g, %g)\n", q[i].x, q[i].y);
    }

    return(0);
}
```

#### ④ ファイル操作におけるエラー処理

C言語の入出力関数には、ファイルの終わりや入出力中のエラーを検出する機能が関数ごと個別に用意されています。しかし、たとえばfgetc()は、ファイルの終わりおよびエラーの検出時にEOFを返しますが、fgets()はNULLポインタを返すといった具合に、関数ごとにその情報は異なります。しかもファイルの終わりとエラーの検出に同じ値(重複情報)を返すものが多いので、それらの区別も関数ごとに行わなければなりません。そこで、このような切り分けはfeof()関数とferror()関数を用いて次のように行います。

feof()の戻り値が真(非0) → ファイル終了の検出

ferror()の戻り値が真(非0) → エラーの検出

また、ferror()関数はエラーを検出すると真(非0)を返しますが、一度検出したエラーは自動的に解除されません(ferror()は真を返し続けます)。そこで、エラー処理を行った後、再びそのファイルを利用したいときは、clearerr()関数を実行してエラーを解除する(FILE構造体中のエラーフラグやファイル終了フラグのリセット)必要があります。これらのエラー処理関数の詳細を以下に示します。

##### ○ ファイル終了検出関数 feof()

形 式 : int feof(FILE \*fp);

機 能 : fpが指すファイルに対応するファイルの終わりを判定する。この際、ファイルの最後のデータを読み取った直後は、ファイルの終了とは判定せず、もう一度データを読み取りにいったとき、終了と判定される。

戻り値 : fpが指すファイルが終わりに達している場合非0を、それ以外は0返す。

##### ○ ファイルエラー検出関数 ferror()

形 式 : int ferror(FILE \*fp);

機 能 : fpが指すファイルに対応するエラーを検出する。

戻り値 : fpが指すファイルでエラーが検出された場合非0、それ以外は0を返す。

##### ○ ファイルエラー終了解除関数 clearerr()

形 式 : void clearerr(FILE \*fp);

機 能 : fpが指すファイルの終了およびエラー状態を解除する。

戻り値 : なし。

## 【 feof(), ferror() 関数の使用例 】

----- 例(22)-7 -----

```
#include <stdio.h>
#include <stdlib.h>

main()
{
    FILE *fp;
    int c;

    fp = fopen("test.dat", "r");
    if (fp == NULL) { // 注意:fopen()のエラーはferror()では検出できない
        fprintf(stderr, "ファイルオープンエラー\n");
        exit(8);
    }
    while(1) {
        c = fgetc(fp);
        if (feof(fp)) {
            fprintf(stderr, "ファイル終了\n");
            fclose(fp);
            exit(0);
        } else if (ferror(fp)) {
            fprintf(stderr, "読み取りエラー\n");
            fclose(fp);
            exit(8);
        }
        fputc(c, stdout);
    }
    return(0);
}
```

## 【 ferror(), clearerr() 関数の使用例 】

----- 例(22)-8 -----

```
#include <stdio.h>
#include <stdlib.h>

main()
{
    FILE *fp;
    int c;

    fp = fopen("test.dat", "r"); // "test.dat"を読み取り用でオープン
    fputc(c, fp); // 読み取り用ファイルに書き込もうとした
    if (ferror(fp)) {
        fprintf(stderr, "エラー\n");
        clearerr(fp); // エラーを解除して
        c = fgetc(fp); // 改めて入力
        printf("入力した文字は%c\n", c);
    }
    fclose(fp);
    return(0);
}
```

### 問題(22)-1

あるテキストファイル内の文字数と行数をカウントし、それを画面表示するプログラムを作りなさい。カウントするテキストファイル名は、プログラムの実行時にオプションとして与えるものとし、空白文字(スペースやタブ)や改行文字('\n')も1文字としてカウントするものとします。(toi22-01.c)

(例) system.txtというテキストファイル内に含まれる文字の数をカウントする。

#### プログラム実行例

```
% ./a.out system.txt ↵  
文字数 : 151  
行数 : 2
```

#### system.txtの内容

```
The type of computer processing where the user of the system communicates  
directly with the system to input data and instruction and receive output.
```

なお、実際に実行結果を確認するためのテキストファイルには

/home/pub/practice/system.txt

/home/pub/practice/input.txt

を使用し、E-mailにはその両ファイルに対して実行した結果を添付しなさい。

### 問題(22)-2

次のように、オプションを何も指定せずに実行するとfile1をfile2へコピーし、オプション-aを指定するとfile1の内容をfile2に追加するプログラムmycpを作りなさい。

```
$ ./mycp file1 file2 ↵ ... file1をfile2へコピー  
$ ./mycp -a file1 file2 ↵ ... file1をfile2へ追加
```

(toi22-02.c)

### 問題(22)-3

いくつかの整数文字列を、1行に1つずつ格納したテキストファイル“number.txt”( /home/pub/practice/number.txt)があります。このファイルを読み込み、同じ整数値をバイナリとして格納したファイル、“number.bin”を生成するプログラムを作りなさい。

(toi22-03-01.c)

また、そのプログラムが正しいことを検証するために、この逆を行うプログラムを作りなさい。なお、バイナリファイル“number.bin”から生成したテキストファイルは、“rebmun.txt”としなさい。

(toi22-03-02.c)

(参考) 2つのファイルの内容が同じかどうかを調べるには、diffコマンドを利用します。diffコマンドは、

```
$ diff file1 file2 ↵
```

のように使います。file1とfile2に違いがあれば、異なる部分を指摘してくれます。同一内容であれば、何も表示しないで終了します。

問題(22)-4

問題(22)-3で作成したファイル、“number.bin”と“rebmun.txt”の大きさを調べ、以下の各問に答えなさい。

- (1) テキストファイルとバイナリファイルの違いを説明し、この2つのファイルの大きさが異なる理由を述べなさい。
- (2) 元々のテキストファイル“number.txt”には、1000個の整数が文字列として格納してありました。“number.bin”のファイルサイズから、君たちの使っている処理系の内部では、整数が何バイトで表現されているか考えなさい。
- (3) これらの結果から、テキストファイルとバイナリファイルは、どのように使い分けるのがよいか説明しなさい。

問題(22)-5

指定したファイルの内容を16進数で表示し、また対応するテキスト文字があればそれを表示するプログラムfdumpを作りなさい。ただし、以下の仕様をすべて満たすこと。

さらに、このプログラムで問題(22)-3で作成したファイル“number.txt”と“rebmun.txt”を表示してみなさい。(toi22-05.c)

<仕様>

- (ア) ファイルは、プログラム実行時にコマンドラインで引数として指定する。
- (イ) 1行に16バイトのデータを表示し、16バイトごとにアドレスを表示する。このアドレスは、ファイルの先頭を16進の「0000」とする。
- (ウ) 文字として画面表示できないコード(‘ ’ ~ ‘ ’ 以外)は、「.(ドット)」を表示する。
- (エ) 画面表示の書式は、次の実行例に従う。

(プログラム実行例)

<テキストファイルの場合>

```
% ./fdump system.txt ↵
addr  +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F      キャラクタ
0000   20 54 68 65 20 74 79 70 65 20 6F 66 20 63 6F 6D      The type of com
0010   70 75 74 65 72 20 70 72 6F 63 65 73 73 69 6E 67      puter processing
      :
```

<バイナリファイルの場合>

```
% ./fdump fdump ↵
addr  +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F      キャラクタ
0000   7F 45 4C 46 01 01 01 00 00 00 00 00 00 00 00 00      .ELF.....
0010   02 00 03 00 01 00 00 00 1C 85 04 08 34 00 00 00      .....4...
      :
```

#### 問題(22)-6

いくつかの整数データが記録されているテキストファイルがあります。このファイルに対して、次の処理を行うプログラムを作りなさい。

- (a) データ数、全データの合計・平均を画面に表示する
- (b) (a)の出力結果を、テキストデータとしてファイルに保存する

なお、データファイルは `/home/pub/practice/sum.dat` にあります。これをカレントディレクトリにコピーし、コピーした `sum.dat` から入力しなさい。また、結果を保存するファイルは `sum.ans` とし、カレントディレクトリに出力するものとします。

(toi22-06.c)

#### 《 実行例 》

```
$ a.out ↵
データ数: 365
合計      : 5788654
平均      : 15859.3
$ ls ↵
$ a.out      sum.ans  sum.dat  toi22-06.c
$ cat sum.ans
データ数: 365
合計      : 5788654
平均      : 15859.3
```

#### 問題(22)-7

英文で書かれたテキストファイル内の単語数をカウントするプログラムを作りなさい。入力ファイルには問題(22)-1と同じファイルを使用すること。

(toi22-07.c)

#### <ヒント>

英文における単語とは、空白文字（空白やタブなど）で囲まれた文字列のことです。  
（空白文字をカウントしただけでは、正確な単語数が求められないことに注意!!）

#### 問題(22)-8

問題(22)-1と問題(22)-7を一つに統合したwcc.c(Word & Character Counter)プログラムを作りなさい。

なお、単語・文字のどちらをカウントするかは、次のようにオプションで指定するものとします。

(toi22-08.c)

#### 《 実行例 》

```
$ wcc -c system.txt ↵
<オプション>: -c ... 文字数      -w ... 単語数
(オプションが指定されなかった場合、または両方が同時に指定された場合は「-c」を指定したものとする。)
```



## (23) 特定の範囲の整数データに名前を付ける (列挙型, *enum*)

1 週間の曜日は、"Sun"～"Sat"の 7 種類あります。これを、0～6 の整数データとして扱うとすると、これまで学習した範囲では、たとえば、次のように記述できます。

----- 例 (23)-1 -----

```
#include <stdio.h>

typedef int WeekDay;

#define Sun      0    // 日曜日
#define Mon      1    // 月曜日
#define Tue      2    // 火曜日
#define Wed      3    // 水曜日
#define Thu      4    // 木曜日
#define Fri      5    // 金曜日
#define Sat      6    // 土曜日
#define WeekNums 7    // 1 週間の日数

main()
{
    WeekDay today;          // WeekDay型の実体はint
    char *day_name[] = {
        "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"
    };

    // 日曜日から 1 週間分の曜日の番号と名前を表示
    for (today = Sun; today < WeekNums; today++) {
        printf("%d: %s\n", today, day_name[today]);
    }

    return(0);
}
```

この例のように、整数データがある範囲の値をとる場合は、多くの #define 文を書くのではなく、最近では次に挙げる列挙型を用いるのが主流となっています。

```
enum 列挙型名 {列挙子 1, 列挙子 2, ...} ; ← 列挙型の定義
enum 列挙型名 変数; ← enum 列挙型名という型の変数宣言
```

列挙型も構造体と同様に、「enum 列挙型名」という形で新しい型として使用できます。ですから、typedefを用いて新しい型名を付けることも可能です。

この列挙型を使って例 (23)-1 を変更すると次のようになります。

----- 例(23)-2 -----

```
#include <stdio.h>

typedef enum {
    Sun, Mon, Tue, Wed, Thu, Fri, Sat, WeekNums // WeekNumsは1週間の日数になる
} WeekDay;

main()
{
    WeekDay today; // WeekDay型はintではなくenum
    char *day_name[] = {
        "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Satday"
    };

    // 日曜日から1週間分の曜日の番号と名前を表示
    for (today = Sun; today < WeekNums; today++) {
        printf("%d is %s\n", today, day_name[today]);
    }
    return(0);
}
```

このプログラムの中では、enumで定義されたWeekDay型の変数todayを使用しています。この変数のとり得る値は Sun, Mon, Tue, Wed, Thu, Fri, Sat, WeekNums のいずれかになります。

ところで、それぞれの列挙子には、特に指定しなければ先頭から順に 0, 1, 2, … と連続した整数値が割り当てられます。例(23)-2 では、Sun = 0, Mon = 1, … という具合になります。ここで、例(23)-2のtodayを配列の添え字に使っている部分に注目してください。実はC言語では※注、enum型で定義された列挙子は整数と同等に扱うことができます。そのため、そのまま配列の添字として使用できるのです。同様に、switch文のcaseラベルにもよく用いられます。

※注 C++言語では事情が異なり、列挙型を整数型として扱うには、キャストが必要になります。また、C++言語では、列挙定数の大きさが int型と一致する保証はありません。

```
#include <stdio.h>

typedef enum {
    Sun, Mon, Tue, Wed, Thu, Fri, Sat, WeekNums // WeekNumsは1週間の日数になる
} WeekDay;

main()
{
    ...
    switch(today) {
        case Sun:
        case Sat:
            printf("土日はゆっくり休もう\n");
            break;
        default:
            printf("しっかり勉強\n");
    }
    ...
    return(0);
}
```

なお、列挙子ごとに値を指定すれば、その値を割り当てることもできます。次の例を見てください。

----- 例(23)-3 -----

```
#include <stdio.h>

typedef enum {
    Zero, One, Two, Three, Four, Five    // 0~5 までの定義
} Num1;
typedef enum {
    Six = 6, Seven, Eight, Nine, Ten    // 6~10 までの定義
} Num2;

main()
{
    printf("\nNum1型\n");
    printf("Zero  = %d\n", Zero);
    printf("One   = %d\n", One);
    printf("Two   = %d\n", Two);
    printf("Three = %d\n", Three);
    printf("Four  = %d\n", Four);
    printf("Five  = %d\n", Five);
    printf("\nNum2型\n");
    printf("Six   = %d\n", Six);
    printf("Seven = %d\n", Seven);
    printf("Eight = %d\n", Eight);
    printf("Nine  = %d\n", Nine);
    printf("Ten   = %d\n", Ten);

    return(0);
}
```

Num1型では、値を指定していないため自動的に割り当てられています。それに対し Num2型では、先頭の列挙子「Six」に「6」という値を指定することによって、それ以降の列挙子は順に連続した整数値が割り当てられています。

ポイント

☆enum型 (列挙型)

---

---

---

## (24) ビット単位でのデータ操作（ビット操作）

これまで学習してきたプログラムでは、char型（1バイト）やint型（4バイト）のバイト単位でデータを扱ってきました。ここからは、コンピュータで扱える最小のデータである、ビット（2進数の「1」か「0」）単位でのデータの操作について学びます。

ビット操作には次の演算子を使います。ただし、これらの演算子は、浮動小数点型のデータには使用できません。

ビットごとの演算子	
演算子	意 味
&	ビットごとの論理積 (AND)
	ビットごとの論理和 (OR)
^	ビットごとの排他的論理和 (XOR)
~	ビットごとの反転 (NOT)
<<	ビット単位での左シフト
>>	ビット単位での右シフト

### ① AND演算子（&）

2つ値の対応する各ビットどうしを比較し、両共に「1」なら結果は「1」となる。  
（少なくともどちらか一方が「0」ならば結果も「0」）

### ② OR演算子（|）

2つ値の対応する各ビットどうしを比較し、少なくともどちらか一方が「1」なら結果は「1」となる。（両者共に「0」のときだけ結果は「0」）

### ③ XOR演算子（^）

2つ値の対応する各ビットどうしを比較し、どちらか一方が「1」なら結果は「1」となる。（両方とも「1」あるいは「0」ならば結果は「0」）

### ④ NOT演算子（~）

与えられたデータの各ビットを全て反転させる。  
（「0」なら「1」、「1」なら「0」になる）

以上①～④までの演算をまとめると次表のようになります。

ビット 1	ビット 2	&		^
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

ビット	~
0	1
1	0

例)  $a = 0x0F(00001111)$ ,  $b = 0x55(01010101)$  とした場合

$a \& b$		$a \mid b$
AND) 00001111 (0x0F)		00001111 (0x0F)
01010101 (0x55)		OR) 01010101 (0x55)
00000101 (0x05)		01011111 (0x5F)
$a \wedge b$		$\sim a$
XOR) 00001111 (0x0F)		NOT) 00001111 (0x0F)
01010101 (0x55)		11110000 (0xF0)
01011010 (0x5A)		

----- 例(24)-1 -----

```
#include <stdio.h>    // ファイル操作・入出力関数用ヘッダファイル
#include <hamakou.h>  // 浜工情報技術科用ヘッダファイル

main()
{
    int x1, x2;

    x1 = 0x00FF00FF;    // 実験用データ 1
    x2 = 0x5555AAAA;    // 実験用データ 2

    putchar('\n');
    printf("    x1: %s = 0x%08X\n", int2bit(x1), x1);    // x1の表示
    printf("    x2: %s = 0x%08X\n", int2bit(x2), x2);    // x2の表示
    printf("\n-----\n\n");
    printf("x1 & x2: %s = 0x%08X\n", int2bit(x1 & x2), x1 & x2);    // AND演算子
    printf("x1 | x2: %s = 0x%08X\n", int2bit(x1 | x2), x1 | x2);    // OR演算子
    printf("x1 ^ x2: %s = 0x%08X\n", int2bit(x1 ^ x2), x1 ^ x2);    // XOR演算子
    putchar('\n');
    printf("\n-----\n\n");
    printf("    ~x1: %s = 0x%08X\n", int2bit(~x1), ~x1);    // NOT演算子
    printf("    ~x2: %s = 0x%08X\n", int2bit(~x2), ~x2);    // NOT演算子
    putchar('\n');

    return(0);
}
```

**int2bit** (Integer to Bit pattern : 本校独自関数)

〔書式〕 `char *int2bit(const unsigned int dat);`  
           *const unsigned int dat* ... 2進文字列に変換する数値

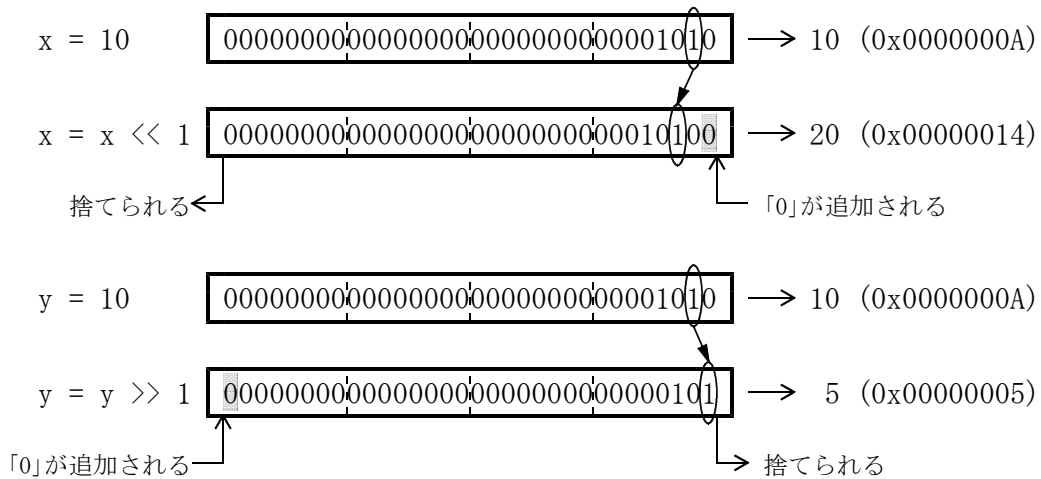
〔機能〕 与えられた整数を2進数で表現した時、そのビット列  
 (0, 1の並び) を表す文字列に変換する。

⑤ 左シフト演算子 (<<), 右シフト演算子 (>>)

左シフト演算子は、与えられたデータを指定したビット数だけ左にシフトします。この時、シフト対象が符号付きか符号なしに関わらず、左端にあったビットは捨てられ、右端には新たに「0」が追加されます（左シフトで符号ビットは保持されません）。

同様に右シフト演算子は、与えられたデータを指定したビット数だけ右にシフトし、右端にあったビットは捨てられます。右シフトでは、左端に新たに追加されるビットは、シフト対象が符号なし（unsigned型）データの場合は常に「0」ですが、符号付きの場合は処理系に依存します（本校のC言語処理系では、符号と同じビットが左端に追加され、符号ビットが保持されます）。

(x, yともunsigned型変数とします)



----- 例(24)-2 -----

```
#include <stdio.h> // ファイル操作・入出力関数用ヘッダファイル
#include <hamakou.h> // 浜工情報技術科ヘッダファイル

main()
{
    int x; // シフト演算をおこなう変数
    int y, z; // 演算結果を格納する変数
    x = 500; // 初期値設定

    printf("x : %s = %5d (0x%08X)\n", int2bit(x), x, x);
    y = x << 2; // 左シフト演算
    printf("x << 2 : %s = %5d (0x%08X)\n", int2bit(y), y, y);
    z = x >> 2; // 右シフト演算
    printf("x >> 2 : %s = %5d (0x%08X)\n", int2bit(z), z, z);

    return(0);
}
```

**bit2int** (Bit pattern to Integer : 本校独自関数)

[書式] `unsigned int bit2int(const char *string);`  
`const char *string` … 数値に変換する2進文字列

[機能] 与えられた「0」「1」の文字で表現されている2進数文字列を、相当する整数値(unsigned int型)に変換する。  
ただし、文字列中に「0」「1」以外の文字が含まれていた場合の戻り値は不定。

**bit\_check** (Bit pattern check : 本校独自関数)

[書式] `int bit_check(const char *string);`  
`const char *string` … 検査する2進文字列

[機能] 与えられた文字列中に、「0」「1」の以外の文字が含まれていないかどうかを検査する。

<戻り値>

「0」「1」の文字だけである … 0

「0」「1」以外の文字が含まれている … -1

ポイント

☆ビット演算子

---

---

---

---

---

問題(24)-1

例(24)-2 において、「x = 500;」を「x = -500;」に変更して実行してみなさい（最上位ビットが「1」の場合の実験）。

問題(24)-2

問題(24)-1 において、「int x;」を「unsigned int x;」に変更して実行してみなさい（最上位ビットが「1」で変数がunsigned int型の場合の実験）。

また、問題(24)-1 との結果の違いについて説明しなさい。

問題(24)-3

次の仕様にに基づき、指定された年月日の曜日を求めるプログラムを作りなさい。

(toi24-03.c)

<仕様>

- ① 年月日は、『年（西暦4桁）/月/日』の形式でコマンドラインで指定する。
- ② 曜日は列挙型(Week型)で表現する。それぞれの列挙子には 0～6 の値をセットし、日曜日(Sun)～土曜日(Sat)に対応させる。

（例）『1995/6/6』の曜日（火曜日）を求める手順

- a. まず、1995年が閏年かどうかを判定する。  
閏年は、その年が 4 で割り切れて かつ 100 で割り切れないか、または 400 で割り切れる年である。
- b. 1995年1月1日から6月6日までの日数を求める。
- c. 1年1月1日から1994年12月31日までの日数を求め、b. で求めた日数と加算する。
- d. c. で求めた日数を7で割った余りは2となり、1995年6月6日は火曜日となる。  
(7で割った余りは、0:日曜日, 1:月曜日, 2:火曜日, …, 6:土曜日)

問題(24)-4

次の考え方に基づき、入力された整数値（unsigned int型）を2進数表現した時の、「1」のビット数をカウントするプログラムを作りなさい。(toi24-04.c)

<考え方>

- ① キーボードから整数値を入力する。
- ② 整数値とマスク（最下位ビットのみ「1」のデータ：int型を32ビットで表現する場合は 0x00000001）との論理積(AND)をとる。
- ③ ②の結果が「1」ならば（最下位ビットが「1」の場合）、カウンタをカウントアップする。
- ④ 整数値を右へ 1 ビットシフトする。
- ⑤ ②～④を繰り返し、シフトした結果整数値が「0」になったら繰り返し終了。
- ⑥ カウンタの値を表示してプログラム終了。